Zellic

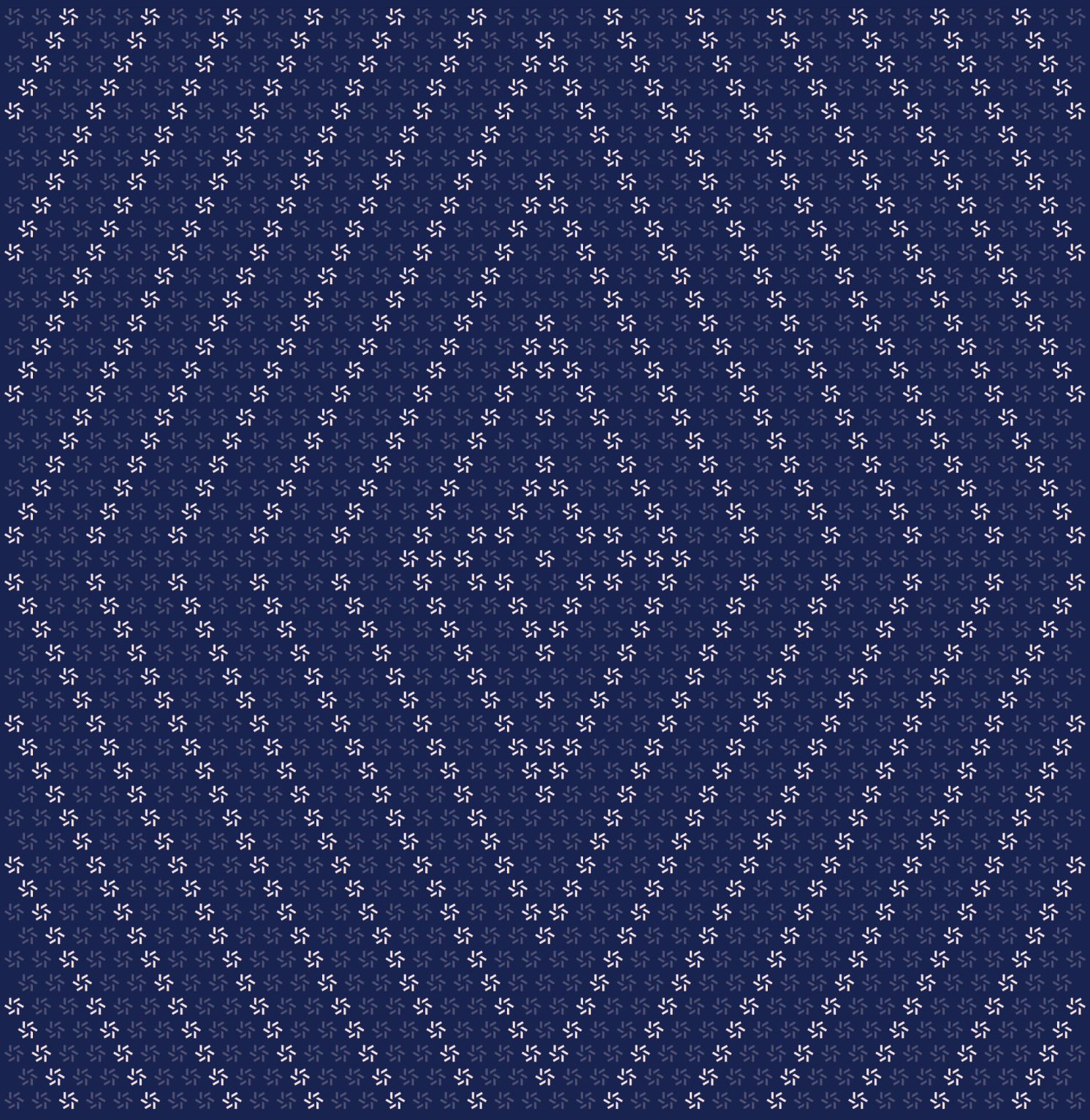
Prepared for
Christos Gkolias
Metric

Prepared by
Hojung Han
Qingying Jie
Zellic

October 6, 2026

OMM Reaudit

Smart Contract Security Assessment



Contents

About Zellic **5**

1. Overview **6**

1.1. Executive Summary 6

1.2. Goals of the Assessment 6

1.3. Non-goals and Limitations 6

1.4. Results 6

2. Introduction **8**

2.1. About OMM 8

2.2. Methodology 8

2.3. Scope 10

2.4. Project Overview 11

2.5. Project Timeline 11

3. Detailed Findings **12**

3.1. Early division misclassifies bid-ask-reference asymmetry at the boundary 12

3.2. Chainlink reports can be stored after their validity window 14

3.3. Negative margin can halt quotes when effective oracle spread is zero 16

3.4. Inactivity weakens the price velocity guard 19

3.5. Dust swaps can prolong a stop-loss freeze 21

3.6. Public payment withdrawals can take another user's unspent assets 24

3.7. Direct pool swaps do not normalize open price limits by direction 26

3.8. Insufficient test coverage 29

4. Discussion 32

4.1. The event `OwnershipTransferred` uses inconsistent first-argument semantics 32

4.2. Per-bin fees can make effective swap prices nonmonotonic 33

5. Threat Model 35

5.1. MetricOmmPool.sol 35

5.2. MetricOmmPoolFactory.sol 44

5.3. DepositAllowlistExtension.sol 64

5.4. MetricOmmPoolLiquidityAdder.sol 67

5.5. MetricOmmSimpleRouter.sol 94

5.6. OracleValueStopLossExtension.sol 124

5.7. PeripheryPayments.sol 157

5.8. PriceVelocityGuardExtension.sol 164

5.9. SelfPermit.sol 169

5.10. SwapAllowlistExtension.sol 180

5.11. AnchoredPriceProvider.sol 182

5.12. AnchoredProviderFactory.sol 186

5.13. ChainlinkOracle.sol 206

5.14. CompressedOracle.sol 216

5.15. PriceProviderFactory.sol 223

5.16. PythOracle.sol 229

5.17. compressed-OracleBase.sol 234

5.18. providers-OracleBase.sol 238

6. Assessment Results 245

6.1. Disclaimer 245

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the #1 CTF (competitive hacking) team ↗ worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io ↗ and follow @zellic_io ↗ on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io ↗.



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Metric from September 8th to September 24th, 2026. During this engagement, Zellic reviewed OMM Reaudit's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

The assessment sought to answer the following security questions:

- Can liquidity deposits and their payment callbacks be restricted to trusted pools so that an attacker-controlled pool cannot draw tokens from a depositor?
- Do pool creation and swap validation enforce safe price parameters and handle rounding correctly at the boundaries of the permitted quote range?
- Are oracle reports accepted only within their intended validity windows, and can price providers continue to return usable quotes across supported spread and margin settings?
- Do the stop-loss and price-velocity extensions protect pools as intended during small swaps and periods of inactivity without unnecessarily preventing trading?
- Do router and direct pool swaps interpret price limits consistently, and do liquidity-depth queries report accurate bid-side prices on nonuniform bin grids?

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

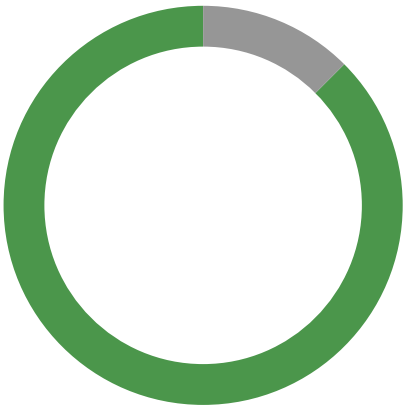
1.4. Results

During our assessment on the scoped OMM Reaudit contracts, we discovered eight findings. No critical issues were found. Seven findings were of low impact and the remaining finding was informational in nature.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of Metric in the Discussion section (4. ↗).

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	0
Low	7
Informational	1



2. Introduction

2.1. About OMM

Metric contributed the following description:

> Metric is an oracle-guided liquidity venue for majors on EVM. Pools follow external reference pricing instead of discovering price inside reserves.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood.

We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped contracts itself. These observations — found in the Discussion (4. ↗) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

OMM Reaudit Contracts

Type	Solidity
Platform	EVM-compatible
Target	Only changes between 33d0a34...4410fec
Repository	https://github.com/Metricxyz/metric-core ↗
Version	`4410fecd8997c912ae8861a337e47bfa0c5af6e3`
Programs	`contracts/**/*.sol`
Target	Only changes between cc8fef9...9b69f75
Repository	https://github.com/Metricxyz/metric-periphery ↗
Version	`9b69f7591bfdf233db1a396cf25f2dd13abb745b`
Programs	`contracts/**/*.sol`
Target	Only changes between edbc5f0...8b72555
Repository	https://github.com/Metricxyz/smart-contracts-poc ↗

Version	`8b72555ba7322b97b5c998e1cadee7100677d9a4`
Programs	`contracts/**/*.sol`

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 3.8 person-weeks. The assessment was conducted by two consultants over the course of 2.6 calendar weeks.

Contact Information

The following project manager was associated with the engagement:

Jacob Goreski
Engagement Manager
jacob@zellic.io

The following consultants were engaged to conduct the assessment:

Hojung Han
Engineer
hojung@zellic.io

Qingying Jie
Engineer
qingying@zellic.io

2.5. Project Timeline

The key dates of the engagement are detailed below.

September 8, 2026	Start of primary review period
September 9, 2026	Kick-off call
September 24, 2026	End of primary review period

3. Detailed Findings

3.1. Early division misclassifies bid-ask-reference asymmetry at the boundary

Target	MetricOmmPool	ID	ZELLIC-2KCYWD
Category	Coding Mistakes	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

The `_validateBidAskReferencePriceX64()` function checks whether the reference price splits the bid-ask spread within the configured `bidAskRefAsymmetryX8` bound. Let `N = (refX64 - bidX64) * refX64` and `D = bidX64 * (askX64 - refX64)`. The intended condition is that `N / D` lies within `[1 / M, M]`, where `M = M_X64 / ONE_X64`. The implementation divides before applying Q64 scaling and then divides again to check the reciprocal.

```
function _validateBidAskReferencePriceX64(uint256 bidX64, uint256 askX64,
    uint256 refX64) internal view {
    // [...]
    uint256 mX8 = bidAskRefAsymmetryX8;
    if (mX8 == type(uint16).max) return;
    if (refX64 == bidX64 || refX64 == askX64)
    revert BidAskRefAsymmetryOutOfBounds();

    uint256 M_X64 = ONE_X64 + (mX8 << 56);
    uint256 refMinusBid = refX64 - bidX64;
    uint256 askMinusRef = askX64 - refX64;

    uint256 ratioX64 = refMinusBid * refX64 / bidX64 * ONE_X64 / askMinusRef;

    if (ratioX64 > M_X64 || ratioX64 == 0 || ONE_X128 / ratioX64 > M_X64) {
      revert BidAskRefAsymmetryOutOfBounds();
    }
}
```

The first division discards the remainder of `N / bidX64`. Multiplication by `ONE_X64` cannot recover it, and the reciprocal division adds another truncation. As a result, the contract's integer comparison does not always agree with the exact inequalities `N * ONE_X64 <= D * M_X64` and `D * ONE_X64 <= N * M_X64`.

Impact

An in-range quote near the boundary can cause swaps and quotes to revert, while a slightly out-of-range quote can pass the asymmetry guard.

Recommendations

Compare the two cross-products directly using full-precision multiplication and division:

```
uint256 ratioX64 = refMinusBid * refX64 / bidX64 * ONE_X64 / askMinusRef;
if (ratioX64 > M_X64 || ratioX64 == 0 || ONE_X128 / ratioX64 > M_X64) {
uint256 numerator = refMinusBid * refX64;
uint256 denominator = bidX64 * askMinusRef;
if (
    numerator < Math.mulDiv(denominator, ONE_X64, M_X64, Math.Rounding.Ceil)
        || denominator < Math.mulDiv(numerator, ONE_X64, M_X64, Math.
            Rounding.Ceil)
) {
    revert BidAskRefAsymmetryOutOfBounds();
 }
```

Remediation

Metric has acknowledged this issue and provided the following comment:

> The error is small: the early division only misclassifies quotes that sit right at the asymmetry boundary, and the effect is limited to a revert or a marginally out-of-range quote passing. Fixing it would require redeploying MetricOmmPool, so for business reasons we decided not to change.

3.2. Chainlink reports can be stored after their validity window

Target	ChainlinkOracle	**ID**	ZELLIC-YF8M5W
Category	Integration Risks	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

Chainlink Data Streams reports supported by the contract ChainlinkOracle include `validFromTimestamp`, `observationsTimestamp`, and `expiresAt`. The contract ChainlinkOracle verifies the signed report through its configured VerifierProxy, but its normalization paths retain only `observationsTimestamp` in `OracleData`. For example, the v3 path assigns the observation time and discards the other two fields:

```
function _normalizeV3(bytes memory reportData)
    internal pure returns (bytes32 feedId, OracleData memory d) {
    ReportV3 memory r = abi.decode(reportData, (ReportV3));
    feedId = r.feedId;
    d.price = _toMid8(r.price);
    d.spread0 = _spreadFromBidAsk(r.price, r.bid, r.ask);
    d.spread1 = 0xFFFF;
    d.timestampMs = toTimeMs(uint256(r.observationsTimestamp) * 1000); //
    seconds → ms
}
```

After verification, `_store()` checks that the observation time is nonzero, not too far in the future, and newer than the previously stored observation. It does not check whether the transaction falls within the report's validity window.

```
function _store(bytes memory reportData) internal {
    // [...]

    d.timestampMs.revertIfZero();
    d.timestampMs.revertIfAfterBlockTimeWithDrift(MAX_TIME_DRIFT);

    if (d.timestampMs.isAfter(oracleData[feedId].timestampMs)) {
        oracleData[feedId] = d;
        emit FeedUpdated(feedId);
    }
}
```

The configured VerifierProxy ↗ delegates signature checking to the Verifier ↗; neither checks the decoded report's `expiresAt` against `block.timestamp`. Consequently, a correctly signed report can initialize an uninitialized feed, or advance a feed with an older stored observation, even after that report's `expiresAt` value. A report whose `validFromTimestamp` is still in the future is likewise not excluded by this contract's time checks.

The price providers separately enforce their own staleness bounds against `observationsTimestamp`. Thus, an arbitrarily old report may be stored but cannot normally be used for a swap. The exploitable window arises when the report has expired but its observation is still recent enough for the consuming provider.

Impact

An expired report can briefly become the pool's active price if its observation remains within the provider's staleness tolerance. If the market moved after the report expired, traders may execute against a superseded price, exposing LP inventory to adverse selection.

Recommendations

Preserve `validFromTimestamp` and `expiresAt` when decoding each supported report schema, and reject storage unless the current block time is within that report's validity window.

Remediation

Metric has acknowledged this issue and provided the following comment:

> Updated feed in the valid window of freshness must be valid too, we cannot judge onchain if there is next update existence.

3.3. Negative margin can halt quotes when effective oracle spread is zero

Target	PriceProvider	**ID**	ZELLIC-N2AP2E
Category	Business Logic	**Severity**	Low
Likelihood	Medium	**Impact**	Low

Description

The contract PriceProvider accepts a negative `marginStep` at construction as long as its magnitude is below `BPS_BASE`. This sets the bid factor above one and the ask factor below one.

```
constructor(
    // [...]
    int256 _marginStep,
    uint256 _maxTimeDelta,
    address _baseToken,
    address _quoteToken
) {
    // [...]
    if (_marginStep <= -BPS_BASE || _marginStep >= BPS_BASE) {
        revert MarginStepOutOfBounds();
    }
    marginStep = _marginStep;
    stepBidFactor = uint256(BPS_BASE - _marginStep);
    stepAskFactor = uint256(BPS_BASE + _marginStep);

    if (_maxTimeDelta == 0 || _maxTimeDelta > 7 days)
    revert MaxTimeDeltaOutOfBounds();
    MAX_TIME_DELTA = _maxTimeDelta;
}
```

The oracle spread only counteracts that narrowing after it is multiplied by `confidenceParam`,

```
function _getQuote() internal returns (uint128, uint128, uint128) {
    // [...]
    uint256 n = spread * confidenceParam;
    (uint256 bidOut, uint256 askOut) = _shapedQuote(mid, n);
```

and `confidenceParam` starts at zero. In addition, an open Chainlink v4 report is normalized to `spread0 = 0`, so its effective spread remains zero regardless of the configured confidence. With `n`

== 0 and a negative margin, `_shapedQuote()` raises the bid above the mid and lowers the ask below it.

```
function _shapedQuote(uint256 mid, uint256 n)
    internal view returns (uint256 bidOut, uint256 askOut) {
    uint256 shapedDenom = STEP_DENOM * CONFIDENCE_BASE;
    uint256 bidNum = n >= CONFIDENCE_BASE ? 0 : mid * (CONFIDENCE_BASE - n);
    uint256 askNum = mid * (CONFIDENCE_BASE + n);
    bidOut = Math.mulDiv(bidNum, Q64 * stepBidFactor, shapedDenom,
    Math.Rounding.Floor);
    askOut = Math.mulDiv(askNum, Q64 * stepAskFactor, shapedDenom,
    Math.Rounding.Ceil);
}
```

For prices at which this inversion survives Q64 rounding, the provider returns a stalled sentinel before the later quantization cover can widen the ask.

```
if (bidOut > askOut) {
    return (0, type(uint128).max, 0);
}
```

The `getQuote()` function turns the sentinel into `FeedStalled()`. A negative `marginStep` as small as `-1` reproduces this with the `mid = 100_000_000` and zero-confidence setup of `testConfidenceZeroMeansNoSpread` in PriceProvider.t.sol. The constructor accepts that value, but `marginStep` is immutable, so updating confidence cannot revive a provider whose oracle consistently reports zero spread.

Impact

Pools using the affected provider cannot swap while it returns `FeedStalled()`. The condition can arise immediately after deployment, before confidence is configured, and persist for an open Chainlink v4 feed. Recovery requires a positive effective oracle spread or replacing the provider; an immutable-provider pool cannot rotate its provider.

Recommendations

Consider rejecting or explicitly warning against negative `marginStep` for feeds that can report zero spread. For deployments that rely on negative bias, validate the provider against the chosen feed's minimum possible spread and require the confidence-adjusted spread to remain wide enough to keep `bid < ask`. A creation-time sanity quote alone is insufficient because the oracle spread can later fall to zero.

Remediation

Metric has acknowledged this issue and provided the following comment:

> Only the operator can create a forged broken pool, that becomes inoperable the instant after creation.

3.4. Inactivity weakens the price velocity guard

Target	PriceVelocityGuardExtension	ID	ZELLIC-REFHTR
Category	Business Logic	Severity	Medium
Likelihood	Low	Impact	Low

Description

The contract PriceVelocityGuardExtension compares each swap's oracle reference price with an anchor established by a previous successful swap. Its allowed deviation grows with the number of blocks since that anchor. Oracle updates without swaps do not advance the anchor, so a long period of pool inactivity can make the first subsequent price check unusually permissive.

The guard applies the following check in `beforeSwap()`:

```
function beforeSwap(
    // [...]
    uint128 referencePriceX64,
    bytes calldata
) external override returns (bytes4) {
    // [...]
    uint256 blockDiff = block.number - s.anchorBlock;

    uint64 maxChange = s.maxChangePerBlockE18;
    if (maxChange != 0) {
      uint256 delta = midPrice > anchorMid ? uint256(midPrice - anchorMid) :
    uint256(anchorMid - midPrice);
      uint256 changeE18 = (delta * 1e18) / uint256(anchorMid);
      uint256 actualSq = changeE18 * changeE18;
      uint256 allowedSq = uint256(maxChange) * uint256(maxChange)
    * (1 + blockDiff);
      if (actualSq > allowedSq) {
        revert PriceVelocityExceeded(actualSq, allowedSq);
      }
    }
    // [...]
}
```

For example, with `maxChangePerBlockE18 = 1e16`, or 1%, consider an anchor price of 100 and an abnormal oracle reference price of 10, omitting their common Q64.64 scaling. The new price differs by 90%, and its squared deviation equals the allowed value after approximately 8,100 inactive blocks: `(0.9e18)^2 = (0.01e18)^2 * 8,100`. If a swap succeeds at that price, the guard

records the price of 10 as `lastObservedMidPriceX64`. In the next block, it rolls this observation into the anchor.

If the oracle price promptly returns to 100, the increase relative to the new anchor is 900%. At the same configured rate, the guard requires approximately 810,000 blocks before that price passes, assuming no intermediate accepted prices or administrative reset. Thus, an attacker who can temporarily distort the active price source may execute the first post-inactivity swap at the distorted reference price and leave subsequent swaps blocked after the source recovers.

Impact

A sufficiently long inactive period can allow a manipulated or erroneous oracle quote to pass the velocity check. If the admitted reference price is materially below the external value, a trader can buy token0 from the pool at an unfavorable price for LPs. Once that quote becomes the anchor, a return to the ordinary price can prevent subsequent swaps for a much longer period.

Recommendations

After a configurable idle interval, consider requiring the proposed reference price to be checked against a fresh, independent price source before allowing a swap and updating the anchor.

Remediation

Metric has acknowledged this issue and provided the following comment:

> The impact is small: it needs a long period of pool inactivity together with a manipulated or erroneous oracle price. We have also decided to stop using this extension by default in our pools.

3.5. Dust swaps can prolong a stop-loss freeze

Target	OracleValueStopLossExtension		
ID	ZELLIC-TC6R33		
Category	Business Logic	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

The contract OracleValueStopLossExtension maintains token0 and token1 high watermarks for each bin, but both share a single `lastDecayTs`. It enforces a token0 watermark breach only on a `zeroForOne` swap and a token1 breach only on a `oneForZero` swap. A successful swap in the permitted direction nevertheless decays and checkpoints both watermarks then resets their shared timestamp.

```
function _checkAndUpdateWatermarks(
    address pool_,
    int16 binIdx,
    uint256 metricT0,
    uint256 metricT1,
    uint256 floorMultiplier,
    uint256 decayRate,
    bool zeroForOne
) private {
    BinHighWatermarks storage hwmS = highWatermarks[pool_][binIdx];
    uint256 dt = block.timestamp - hwmS.lastDecayTs;

    (uint256 hwm0, bool breach0) = _applyWatermark(metricT0,
    _decayed(hwmS.token0, decayRate, dt), floorMultiplier);
    if (breach0 && zeroForOne) {
      revert OracleStopLossTriggered(binIdx, true, metricT0, (hwm0
    * floorMultiplier) / E6);
    }

    (uint256 hwm1, bool breach1) = _applyWatermark(metricT1,
    _decayed(hwmS.token1, decayRate, dt), floorMultiplier);
    if (breach1 && !zeroForOne) {
      revert OracleStopLossTriggered(binIdx, false, metricT1, (hwm1
    * floorMultiplier) / E6);
    }
```

```
    // forge-lint: disable-next-line(unsafe-typecast)
    hwmS.token0 = uint104(hwm0);
    // forge-lint: disable-next-line(unsafe-typecast)
    hwmS.token1 = uint104(hwm1);
    // forge-lint: disable-next-line(unsafe-typecast)
    hwmS.lastDecayTs = uint40(block.timestamp);
}
```

The decay calculation rounds each interval's decrement down:

```
function _decayed(uint256 hwm, uint256 ratePerSecondE18, uint256 dt)
    private pure returns (uint256) {
    if (ratePerSecondE18 == 0 || dt == 0 || hwm == 0) return hwm;
    uint256 factor = ratePerSecondE18 * dt;
    if (factor >= E18) return 0;
    return hwm - (hwm * factor) / E18;
}
```

Suppose the token0 metric has fallen below its drawdown threshold while the token1 metric remains healthy — `zeroForOne` swaps revert, but an attacker can repeatedly execute small `oneForZero` swaps that touch the bin. Each successful swap advances `lastDecayTs` even though the token0 breach remains. Repeated checkpointing compounds the decay rather than applying the configured linear rate from a fixed baseline. If `hwm * ratePerSecondE18 * dt < 1e18` for every interval, each decrement rounds to zero and the breached watermark remains unchanged.

Impact

An unprivileged trader can prolong a pool-local outage of one swap direction by repeatedly trading in the other direction. The freeze can persist while the campaign continues if each decay step rounds to zero. Otherwise, recovery is delayed rather than prevented.

Recommendations

For each metric, consider retaining its stored high watermark and the timestamp when it was last established. Calculate the effective, linearly decayed watermark from the total time elapsed since that timestamp, without writing the decayed value back on every swap. Update the stored watermark and its timestamp only when the live metric establishes a new high. This prevents opposite-direction swaps from resetting the breached metric's decay clock or discarding fractional decay at each checkpoint.

Remediation

Metric has acknowledged this issue and provided the following comment:

> The impact is small. In the worst case, it can prolong the freeze — while fixing it would require an additional storage slot.

3.6. Public payment withdrawals can take another user's unspent assets

Target	PeripheryPayments, MetricOmmSimpleRouter, MetricOmmPoolLiquidityAdder		
ID	ZELLIC-GH1AR3		
Category	Business Logic	**Severity**	Medium
Likelihood	Low	**Impact**	Low

Description

The contract PeripheryPayments exposes `refundETH()`, `unwrapWETH9()`, and `sweepToken()` without restricting who may call them. Each function withdraws the contract's entire balance of the selected asset. The `refundETH()` function sends ETH to its caller, while the other two functions send WETH or tokens to a caller-selected recipient.

```
function refundETH() external payable override {
    uint256 balance = address(this).balance;
    if (balance > 0) {
      _transferETH(msg.sender, balance);
    }
}
```

The contracts MetricOmmSimpleRouter and MetricOmmPoolLiquidityAdder allow a user to send more ETH than an operation ultimately needs and retrieve the remainder through a later `refundETH()` call in the same `multicall()`. Pool creation is permissionless and accepts caller-selected extensions, which the pool invokes before returning to the periphery. At the audited revision, neither the swap nor liquidity-add entry points held a reentrancy lock shared with the withdrawal helpers. A malicious `afterSwap` or `afterAddLiquidity` extension on a pool selected by a victim can therefore call `refundETH()` after the victim's input has been paid but before the queued refund runs, and the extension receives the entire remaining ETH balance. The victim's subsequent refund sees zero, even though the two steps were submitted in one transaction.

Assets left in a periphery contract between transactions are exposed independently of reentrancy. For example, another caller can invoke `sweepToken(token, 0, recipient)` or `unwrapWETH9(0, recipient)` to take tokens or WETH that a user expected to withdraw in a separate transaction.

Impact

An attacker controlling an extension on a pool selected by a victim can take the victim's unused native funding, potentially up to the difference between the ETH supplied and the amount consumed by the swap or liquidity add. Public withdrawals can also transfer ERC-20 tokens or WETH left in a router or adder between transactions to an unrelated caller. The in-transaction attack requires surplus funds and an attacker-controlled extension; the separate-transaction exposure requires funds to remain in the periphery contract.

Recommendations

Apply a shared reentrancy lock to all swap and liquidity-add entry points and to the withdrawal helpers.

If possible, consider requiring users to execute swaps or add liquidity through `multicall()`.

Remediation

Metric has acknowledged this issue and provided the following comment:

> Users should not call the functions related to this issue in separate transactions and we plan to add an additional guard in the future to prevent this.

3.7. Direct pool swaps do not normalize open price limits by direction

Target	MetricOmmPool	ID	ZELLIC-E918HW
Category	Integration Risks	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The router accepts either endpoint value as an open `priceLimitX64` and converts it to the sentinel appropriate for the swap direction. For `zeroForOne`, an open limit is 0; for `oneForZero`, it is `type(uint128).max`.

```
function normalizePriceLimit(bool zeroForOne, uint128 priceLimitX64)
    internal pure returns (uint128) {
  if (zeroForOne) {
    if (priceLimitX64 == type(uint128).max) return 0;
    return priceLimitX64;
  }
  if (priceLimitX64 == 0) return type(uint128).max;
  return priceLimitX64;
}
```

The contract MetricOmmPool does not perform the same conversion on direct `swap()` or `simulateSwapAndRevert()` calls. Its `_swapAcrossBins()` function also treats a zero limit differently when deciding whether to skip an empty bin: the specified-input branch exempts zero, while the specified-output branch compares zero directly with the effective bin boundary.

```
function _swapAcrossBins(
    bool zeroForOne,
    bool specifiedOut,
    uint256 amountSpecifiedScaled,
    uint256 referencePriceX64,
    uint256 baseFeeX64,
    uint256 priceLimitX64
) internal returns (int256 amount0DeltaScaled, int256 amount1DeltaScaled,
    Packed2Uint128 platformFees) {
    // [...]
        bool nonEmptyBin = true;
        if (zeroForOne) {
          if (binState.token1BalanceScaled == 0 || curPosInBinCache == 0) {
            if (specifiedOut) {
```

```
            if (priceLimitX64 >= effLowerX64) break;
          } else {
            if (priceLimitX64 != 0 && priceLimitX64 >= effLowerX64) break;
            if (totalAvailableOutScaled == 0) break;
          }
          nonEmptyBin = false;
        }
      } else if (binState.tokenOBalanceScaled == 0 || curPosInBinCache
>= type(uint104).max) {
        if (specifiedOut) {
          if (priceLimitX64 <= effUpperX64) break;
        } else {
          if (priceLimitX64 != 0 && priceLimitX64 <= effUpperX64) break;
          if (totalAvailableOutScaled == 0) break;
        }
        nonEmptyBin = false;
      }
```

This conditional difference alone does not establish a distinct outcome for ordinary positive bin prices. The observable interface difference is that a direct `oneForZero` call with `priceLimitX64 == 0` cannot trade, whereas the router converts the same input to an open upper limit. For a nonempty bin, the swap math rejects the zero upper bound, and the pool's nonzero-delta check then reverts. The same issue affects both specified-input and specified-output calls.

Impact

Integrations that use the router's open-limit convention when calling the pool directly may have their `oneForZero` swaps or simulations revert unexpectedly. No funds are lost, and direct callers can avoid the issue by supplying the direction-appropriate sentinel.

Recommendations

Normalize both open-limit sentinels at the pool entry point, or document that direct pool callers must supply the direction-appropriate sentinel. In either case, use consistent empty-bin limit checks for specified-input and specified-output swaps so the accepted encoding is unambiguous.

Remediation

Metric has acknowledged this issue and provided the following comment:

> The issue is informational and no funds are lost. Direct pool callers must pass the price-limit sentinel for their swap direction, and the router already normalizes it. Fixing it would require

redeploying MetricOmmPool, so for business reasons we decided not to change it.

3.8. Insufficient test coverage

Target	Project-wide	**ID**	ZELLIC-E5HJ0T
Category	Protocol Risks	**Severity**	Low
Likelihood	N/A	**Impact**	Low

Description

The test suite at the time of writing covers the core functionality of all three repositories, but security-relevant branches remain undertested, particularly failure paths and boundary conditions. We measured coverage at the audit commits (Periphery `9b69f7591bfdf233db1a396cf25f2dd13abb745b`, Oracle `8b72555ba7322b97b5c998e1cadee7100677d9a4`, and `metric-core` `4410fecd8997c912ae8861a337e47bfa0c5af6e3`):

Repository	Tests passing	Lines	Functions	Branches
metric-core	347	68.9% (975/1416)	84.5%	63.5%
metric-periphery	177	70.5% (701/994)	75.8%	61.4%
oracle-poc	509	81.8% (865/1058)	87.9%	41.1%

While function coverage is reasonably high, branch coverage is significantly lower in every repository. This means tests reach most functions but exercise only a fraction of their conditional paths, leaving revert conditions, boundary values, and adversarial scenarios unverified. Specifically, the following areas require additional coverage:

- **Oracle failure and boundary paths** — The oracle-poc repository has the lowest branch coverage at 41.1% despite 509 passing tests. Revert paths for stale or out-of-bounds prices, confidence and spread rejections, role-transition edge cases, and verifier/Pyth verification failures are largely unexercised, so mispriced or invalid oracle data could be accepted or rejected contrary to the intended guard behavior without any test detecting it.
- **State-changing functions lacking revert-path tests** — Several state-mutating functions in the periphery extensions (stop-loss proposal/execution/cancellation paths, timelock boundaries, and allowlist mode transitions) are not fully exercised for revert paths and boundary conditions, as documented per branch in the threat-model section.

- **Fund-handling callback paths** — Router and liquidity-adder swap and modify-liquidity callbacks handle token payments whose recipient and amount depend on transient callback context; not all orderings of context set, update, and clear are tested against adversarial calldata.
- **Untested invariants and assumptions** — Key properties such as pending-proposal deletion after execution, monotonic effective price across step functions, and bid-ask-reference asymmetry at exact boundary values are not asserted as invariants.

Impact

Insufficient branch coverage increases the likelihood that

- bugs or vulnerabilities in untested revert paths and boundary conditions remain undetected until production,
- refactors or minor changes unintentionally break failure-mode behavior that no test exercises, and
- critical security assumptions, such as oracle price-guard rejections, timelock enforcement, and callback payment constraints, are violated silently.

Comprehensive testing improves code correctness, auditability, and developer confidence and reduces the risk of costly postdeployment fixes.

Recommendations

Expand the test suite to cover the gaps identified above, with emphasis on branch coverage rather than additional happy-path tests. We recommend prioritizing effort in this order:

1. **Oracle guard rejection paths (oracle-poc, branch coverage 41.1%)** — Add negative tests for every documented revert condition of the price providers and `OracleBase` implementations: staleness, confidence and spread bounds, price-guard rejections, and role-transition sequencing. These paths decide whether invalid prices enter the system.
2. **Fund-handling callbacks (metric-periphery)** — Add tests for `metricOmmSwapCallback` and `metricOmmModifyLiquidityCallback` that vary payment recipients, amounts, and transient-context ordering, including adversarial calldata from untrusted pools.
3. **Stop-loss timelock boundaries** — Add tests asserting the full life cycle of each pending proposal type: proposal, execution at the exact eligibility timestamp, cancellation, and storage deletion of the pending proposal after execution.
4. **Invariant assertions** — Convert key documented properties into assertions: balance consistency after multistep router workflows, monotonic effective price, and boundary behavior of the bid-ask-reference asymmetry check.

Specific missing test cases include but are not limited to those marked as untested in section 5. ↗.

Remediation

This issue has been acknowledged by Metric.

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. The event `OwnershipTransferred` uses inconsistent first-argument semantics

The contract Auth declares the first indexed argument of the event `OwnershipTransferred` as `user`. Its constructor and direct `transferOwnership()` implementation emit the caller in that position.

```
event OwnershipTransferred(address indexed user, address indexed newOwner);
```

```
constructor(address _owner, Authority _authority) {
    // [...]
    emit OwnershipTransferred(msg.sender, _owner);
    emit AuthorityUpdated(msg.sender, _authority);
}
```

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    owner = newOwner;

    emit OwnershipTransferred(msg.sender, newOwner);
}
```

The contract Auth2Step inherits the same event but emits the prior owner when the pending owner accepts a transfer.

```
function _transferOwnership(address newOwner) internal {
    delete _pendingOwner;
    address previousOwner = owner;
    owner = newOwner;
    emit OwnershipTransferred(previousOwner, newOwner);
}
```

For a two-step transfer, the caller is the incoming owner, so an indexer interpreting the first argument as `user` would read different meanings from the constructor and acceptance events. The event data itself remains decodable, and this discrepancy does not change ownership or authorization. However, it can make ownership history and event-based monitoring ambiguous.

Consider renaming the first event parameter to `previousOwner` to follow the intended OpenZeppelin convention.

Metric has acknowledged this issue and provided the following comment:

> One contract originates from solmate and the other from OpenZeppelin and they have different conventions. For us it should follow OZ convention: and use `previousOwner`. Event param name should be `previousOwner` not user. Not fixing to keep the code coherent with deployed one.

4.2. Per-bin fees can make effective swap prices nonmonotonic

The pool compares `priceLimitX64` against effective prices, which include the direction-specific fee of each bin. The fee calculation uses the bin's `addFeeSellE6` or `addFeeBuyE6` in addition to the base and notional fees.

```
// metric-core/contracts/MetricOmmPool.sol:L903-L907
uint256 binTotalFeeX64 = FeeMath.binTotalFeeX64(
    baseFeeX64, zeroForOne ? binState.addFeeSellE6 : binState.addFeeBuyE6,
    notionalFeeX64
);
uint256 effLowerX64 = FeeMath.effectivePriceX64(lowerReferencePriceX64,
    binTotalFeeX64, zeroForOne);
uint256 effUpperX64 = FeeMath.effectivePriceX64(upperReferencePriceX64,
    binTotalFeeX64, zeroForOne);
```

The reference-price boundaries of adjacent bins are contiguous, but their effective-price boundaries need not be. For `zeroForOne` swaps, `effectivePriceX64()` divides the reference price by `1 + binTotalFee`. If the next, lower-priced bin charges a smaller fee, its effective upper boundary can exceed the current bin's effective lower boundary. For example, suppose the current bin spans reference prices from `1` to `1.1` and has a 5% total fee, while the next bin has a zero total fee. At their shared reference boundary of `1`, the current bin's effective price is about `0.952`, while the next bin's is `1`. A `priceLimitX64` corresponding to `0.97` can therefore stop the swap within the current bin even though the next bin would offer a price above the limit.

The limit can also prevent traversal when the current bin has no output available. For a `zeroForOne` swap, the pool checks `priceLimitX64` against that bin's `effLowerX64` before marking it empty and moving to the next bin:

```
// metric-core/contracts/MetricOmmPool.sol:L911-L920
if (binState.token1BalanceScaled == 0 || curPosInBinCache == 0) {
    if (specifiedOut) {
        if (priceLimitX64 >= effLowerX64) break;
```

```
    } else {
        if (priceLimitX64 != 0 && priceLimitX64 >= effLowerX64) break;
        if (totalAvailableOutScaled == 0) break;
    }
    nonEmptyBin = false;
    }
}
```

In the example above, a limit of `0.97` stops at the empty current bin because `0.97` exceeds its effective lower boundary of about `0.952`, even though the next bin begins at an effective price of `1`. If the current bin is nonempty, the in-bin swap math can likewise stop at the limit before exhausting it, and the loop does not search later bins. Thus, both empty and nonempty bins can leave otherwise acceptable liquidity unused when per-bin fees make effective prices nonmonotonic.

Document that pool creators should choose per-bin additional fees that preserve monotonic effective prices in each swap direction, or explicitly define the price limit as a contiguous-path stopping condition rather than a search for any later acceptable bin.

Metric has acknowledged this issue and provided the following comment:

> We are aware of that and I think there is nothing we can do about it easily, also it's not so big deal. We will add to documentation for pool creators that it's best to use such additional fees so buying/selling results in monotonous prices.

5. Threat Model

This provides a full threat model description for various functions. As time permitted, we analyzed each function in the contracts and created a written threat model for some critical functions. A threat model documents a given function's externally controllable inputs and how an attacker could leverage each input to cause harm.

Not all functions in the audit scope may have been modeled. The absence of a threat model in this section does not necessarily suggest that a function is safe.

5.1. MetricOmmPool.sol

Function: `collectFees(uint256 protocolSpreadFeeE6, uint256 adminSpreadFeeE6, uint256 protocolNotionalFeeE8, uint256 adminNotionalFeeE8, address adminFeeDestination, address protocolFeeDestination)`

This function allows the factory to allocate and distribute accrued protocol and admin fees. When the spread fee sum is nonzero, it derives unallocated spread fees from token balances after subtracting bin balances, accrued notional fees, and previously allocated pending fees. It splits those fees according to the supplied spread rates. When the notional fee sum is nonzero, it similarly allocates the accrued notional fees and clears their unallocated counters. Previously allocated pending fees retain their original split. The function attempts four payouts, preserving failed payments and scaled rounding remainders in pending balances, and emits `ProtocolFeesCollected` with the amounts actually paid.

Inputs

- `protocolSpreadFeeE6`
  - **Control**: Supplied by the factory from its pool fee configuration.
  - **Constraints**: N/A.
  - **Impact**: Determines the protocol share of newly allocated spread fees.
- `adminSpreadFeeE6`
  - **Control**: Supplied by the factory from its pool fee configuration.
  - **Constraints**: N/A.
  - **Impact**: Determines the admin share of newly allocated spread fees using `adminSpreadFeeE6 / (protocolSpreadFeeE6 + adminSpreadFeeE6)`.
- `protocolNotionalFeeE8`
  - **Control**: Supplied by the factory from its pool fee configuration.
  - **Constraints**: N/A.
  - **Impact**: Determines the protocol share of newly allocated notional fees.
- `adminNotionalFeeE8`
  - **Control**: Supplied by the factory from its pool fee configuration.
  - **Constraints**: N/A.

    - **Impact**: Determines the admin share of newly allocated notional fees using `adminNotionalFeeE8 / (protocolNotionalFeeE8 + adminNotionalFeeE8)`.
- `adminFeeDestination`
    - **Control**: Supplied by the factory from the pool's admin fee destination setting.
    - **Constraints**: Successful payment depends on the token accepting the transfer.
    - **Impact**: Receives admin fees, including previously unpaid allocations.
- `protocolFeeDestination`
    - **Control**: Supplied by the factory from its `feeReceiver` setting.
    - **Constraints**: Successful payment depends on the token accepting the transfer.
    - **Impact**: Receives protocol fees, including previously unpaid allocations.

Branches and code coverage

Intended branches

- Nonzero spread and notional fee sums allocate the accrued fees between admin and protocol.
    - ☑ Test coverage
- Allocating notional fees clears the unallocated notional counters.
    - ☑ Test coverage
- Pending allocations can be paid even when both supplied fee sums are zero.
    - ☑ Test coverage
- Successful payouts reduce pending balances, and repeated collection does not pay the same allocation twice.
    - ☑ Test coverage
- Failed payouts remain pending, retain their split, and can be retried after transfer restrictions are removed.
    - ☑ Test coverage
- The `ProtocolFeesCollected` reports amounts actually transferred, including zero for failed payouts.
    - ☑ Test coverage
- Amounts below one native token unit are retained as scaled rounding remainders.
    - ☐ Test coverage

Negative behavior

- A caller other than `FACTORY` is rejected with `OnlyFactory`.
  - ☐ Negative test
- Reentry into a guarded pool operation is rejected.
  - ☐ Negative test

Function: `setBidAskRefAsymmetryX8(uint16 newBidAskRefAsymmetryX8)`

This function allows only the factory to change the pool's bid-ask reference asymmetry tolerance. A changed value updates `bidAskRefAsymmetryX8` and emits `BidAskRefAsymmetryUpdated`; an unchanged value produces no event. The setting constrains the imbalance between buy-side and sell-side returns around the oracle reference. A value of `type(uint16).max` disables that check.

Inputs

- `newBidAskRefAsymmetryX8`
  - **Control**: Supplied by the factory, following an update requested by its owner or an authorized user.
  - **Constraints**: N/A.
  - **Impact**: Changes the tolerance applied to subsequent live quotes and simulated swaps.

Branches and code coverage

Intended branches

- A changed value updates the pool's stored asymmetry tolerance, as exercised by the factory's single and batch update tests.
  - ☑ Test coverage
- A `BidAskRefAsymmetryUpdated` event is emitted when the value changes.
  - ☐ Test coverage
- An unchanged value returns without modifying the setting or emitting an event.
  - ☐ Test coverage

Negative behavior

- A caller other than `FACTORY` is rejected with `OnlyFactory`.
  - ☐ Negative test
- Reentry during another guarded pool operation is rejected.
  - ☐ Negative test

Function: `simulateSwapAndRevert(address sender, address recipient, bool zeroForOne, int128 amountSpecified, uint128 priceLimitX64, uint128 bidPriceX64, uint128 askPriceX64, uint128 referencePriceX64, bytes extensionData)`

This function simulates a swap using caller-supplied prices and the pool's shared swap logic. It validates the prices, invokes enabled before-swap and after-swap extensions, and calculates the token deltas without transferring tokens or calling a swap callback. Successful simulation ends by reverting with `SimulateSwap(amount0Delta, amount1Delta)`, rolling back the pool and extension state changes. It does not query the live price provider or enforce the pool's pause check, so a simulation result alone does not establish that a live swap will succeed.

Inputs

- `sender`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: N/A.
  - **Impact**: The sender presented to the swap extensions, allowing simulation of their checks for another address.
- `recipient`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Subject to enabled extension checks.
  - **Impact**: The recipient presented to the extensions. No tokens are transferred to it.
- `zeroForOne`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: N/A.
  - **Impact**: Selects token0 for token1 when true and the reverse direction when false.
- `amountSpecified`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must be nonzero. The scaled execution amount passed to `_getInitialStateForSwap()` must fit `uint128`, and the resulting external deltas must be nonzero and fit `int128`.
  - **Impact**: Specifies input when positive or desired output magnitude when negative, in native units of that token.
- `priceLimitX64`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Applied directly, with the same direction-dependent encoding as `swap()`.

    - **Impact**: Bounds progress along the fee-inclusive price path and may cause a partial simulated fill.
- `bidPriceX64`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: Must be nonzero and strictly less than `askPriceX64`.
    - **Impact**: Determines the sell-side base fee and is passed to the extensions.
- `askPriceX64`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: Must be strictly greater than `bidPriceX64`.
    - **Impact**: Determines the buy-side base fee and is passed to the extensions.
- `referencePriceX64`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: Must lie within the bid-ask spread and satisfy the configured asymmetry bound. Equality with either endpoint is permitted only when `bidAskRefAsymmetryX8 == type(uint16).max`.
    - **Impact**: Anchors the simulated bin prices. All three supplied prices use Q64.64 encoding.
- `extensionData`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: Must satisfy enabled extension requirements.
    - **Impact**: Forwarded to both swap extension phases.

Branches and code coverage

Intended branches

- Valid prices can complete simulation and produce the `SimulateSwap` error.
    - ☑ Test coverage
- A reference equal to the bid or ask is accepted when the asymmetry check is disabled.
    - ☑ Test coverage
- The error's token deltas match an equivalent live swap using the same prices.
    - ☐ Test coverage
- Pool and extension state changes are rolled back after a successful simulation.
    - ☐ Test coverage
- Simulation remains available while the pool is paused.
    - ☐ Test coverage
- The supplied `sender` is forwarded to both extension phases even when different from the caller.

- ☐ Test coverage

Negative behavior

- A bid greater than the ask or a zero bid is rejected.
  - ☑ Negative test
- A reference below the bid or above the ask is rejected.
  - ☑ Negative test
- Before-swap and after-swap extension reverts propagate to the caller.
  - ☑ Negative test
- An enabled extension can reject the simulated swap through its access or policy checks.
  - ☑ Negative test
- Equal bid and ask prices or an invalid reference asymmetry are rejected.
  - ☐ Negative test
- A zero amount, zero resulting external delta, or unsupported integer range reverts before the simulation result is produced.
  - ☐ Negative test
- Reentry during another guarded pool operation is rejected.
  - ☐ Negative test

Function call analysis

- `this._swapCore(...) -> this._beforeSwap(...) and this._afterSwap(...) -> configured extensions`
  - **What is controllable?** The caller controls `sender`, `recipient`, the swap parameters, all three prices, and `extensionData`.
  - **If the return value is controllable, how is it used and how can it go wrong?** Each extension must return the expected selector. A rejection prevents a quote. Caller-supplied prices and sender identity make the result a hypothetical quote rather than proof of live execution or authorization.
  - **What happens if it reverts, reenters or does other unusual control flow?** Extension failures propagate; invalid responses revert. The pool guard prevents entry into guarded functions. Even when all extensions succeed, the final `SimulateSwap` revert rolls back every state change made within the call.

Function: `swap(address recipient, bool zeroForOne, int128 amountSpecified, uint128 priceLimitX64, bytes callbackData, bytes extensionData)`

This function exchanges tokens using the active price provider's bid, ask, and reference prices. It validates the quote, calls the configured before-swap extensions, executes the swap across bins, and updates pool accounting. It transfers the output to `recipient` before invoking the caller's swap callback then checks that the input token balance increased by at least the amount owed. After settlement, it calls the after-swap extensions, emits `Swap`, and returns the signed token deltas. Positive deltas represent input owed to the pool; negative deltas represent output sent to the recipient. A price limit or insufficient available liquidity can result in a partial fill.

Inputs

- `recipient`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: No explicit address validation. The output token must accept the transfer, and enabled extensions may impose additional restrictions.
  - **Impact**: Receives the output tokens and is passed to the swap extensions.
- `zeroForOne`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: N/A.
  - **Impact**: When true, token0 is exchanged for token1; when false, token1 is exchanged for token0.
- `amountSpecified`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must be nonzero. The scaled execution amount passed to `_getInitialStateForSwap()` must fit `uint128`, and the resulting external token deltas must be nonzero and fit `int128`.
  - **Impact**: A positive value specifies input; a negative value specifies the desired output magnitude. The amount is expressed in native units of the specified token.
- `priceLimitX64`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Used directly without router-style normalization. An open limit is 0 for `zeroForOne` and `type(uint128).max` for the opposite direction.
  - **Impact**: Limits progress along the effective, fee-inclusive price path. It is a lower bound for `zeroForOne` and an upper bound for the opposite direction. Rounding can materially affect the realized price of very small trades.
- `callbackData`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: N/A.

    - **Impact**: Forwarded unchanged to the caller's `metricOmmSwapCallback()`.
- `extensionData`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: Must satisfy any requirements imposed by the enabled extensions.
    - **Impact**: Forwarded to the before-swap and after-swap extensions.

Branches and code coverage

Intended branches

- Specified-input and specified-output swaps execute in both directions and settle the corresponding token deltas.
    - ☑ Test coverage
- Swaps update bin balances and advance through bins.
    - ☑ Test coverage
- A price limit or the end of the available bin range can stop execution before the specified amount is filled.
    - ☑ Test coverage
- Enabled swap extensions are called, and after-swap extensions receive the final packed pool state.
    - ☑ Test coverage
- A `Swap` event records the caller, recipient, final position, token deltas, and platform fees.
    - ☐ Test coverage

Negative behavior

- The call reverts with `PoolPaused` when the pool is paused.
    - ☑ Negative test
- A zero specified amount reverts with `InvalidAmount`.
    - ☑ Negative test
- A swap producing a zero external token delta reverts with `FinalAmountIsZero`.
    - ☑ Negative test
- A reverting price provider prevents swap execution.
    - ☑ Negative test
- A before-swap extension can reject the transaction.
    - ☑ Negative test
- A nested swap from the callback is rejected by the pool's reentrancy guard.

- [x] Negative test

- Insufficient callback payment reverts with `IncorrectDelta`.
  - [ ] Negative test
- Invalid oracle quotes or an out-of-bounds bid-ask reference asymmetry revert.
  - [ ] Negative test
- A failed output transfer, reverting callback, or rejecting after-swap extension rolls back the transaction.
  - [ ] Negative test

Function call analysis

- `this._getBidAskReferencePriceX64() -> IPriceProvider(activePriceProvider).getQuote()`
  - **What is controllable?** N/A.
  - **If the return value is controllable, how is it used and how can it go wrong?** The returned prices determine execution prices and base fees. They are checked for a nonzero bid, `bid < ask`, a reference within the spread, and the configured asymmetry bound. These checks do not establish that the quote matches the external market.
  - **What happens if it reverts, reenters or does other unusual control flow?** A provider revert is wrapped in `PriceProviderFailed`. Reentry into guarded pool functions is blocked, and any propagated failure rolls back the swap.
- `this._swapCore(...) -> this._beforeSwap(...) and this._afterSwap(...) -> configured extensions`
  - **What is controllable?** The caller controls the swap parameters and `extensionData`; `sender` is fixed to `msg.sender`. Extension addresses and invocation orders are fixed in the pool.
  - **If the return value is controllable, how is it used and how can it go wrong?** Each extension must return the expected function selector. Extensions can reject swaps or apply their own checks; their return values do not replace the calculated token deltas.
  - **What happens if it reverts, reenters or does other unusual control flow?** Revert data is propagated, and malformed or incorrect selectors revert with `InvalidExtensionResponse`. The pool guard blocks reentry into guarded functions. Failure in either phase rolls back accounting and token settlement.
- `IMetricOmmSwapCallback(msg.sender).metricOmmSwapCallback(amount0Delta, amount1Delta, callbackData)`
  - **What is controllable?** The caller controls the callback contract and `callbackData`.
  - **If the return value is controllable, how is it used and how can it go wrong?**

N/A.

- **What happens if it reverts, reenters or does other unusual control flow?** A revert rolls back the output transfer and all swap state changes. Guarded entry into the same pool is blocked; the callback can interact with other pools.

5.2. MetricOmmPoolFactory.sol

Function: `collectPoolFees(address pool)`

This function allows anyone to trigger collection of a pool's protocol and admin fees. It loads the pool's fee configuration from `poolFeeConfig` and calls the pool's `collectFees()`, using `poolAdminFeeDestination[pool]` and `feeReceiver` as the recipients.

Inputs

- `pool`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: The factory does not verify pool registration. If the pool was not deployed through the factory, all arguments except `feeReceiver` passed to `collectFees()` will be zero. The target must accept the `collectFees()` call.
  - **Impact**: The pool from which fees are collected and the key used to load the fee configuration and admin destination.

Branches and code coverage

Intended branches

- Any caller can trigger fee collection using the stored fee configuration and destinations.
  - ☐ Test coverage
- Collectible fees are paid to the configured admin destination and protocol fee receiver.
  - ☐ Test coverage
- A pool with no collectible fees completes the call without transferring tokens.
  - ☐ Test coverage

Negative behavior

- Reentry into a factory function protected by `nonReentrant` is rejected.
  - ☐ Negative test

Function call analysis

- `this._collectFees(pool, this.poolFeeConfig[pool]) -> IMetricOmmPoolCollectFees(pool).collectFees(c.protocolSpreadFeeE6, c.adminSpreadFeeE6, c.protocolNotionalFeeE8, c.adminNotionalFeeE8, this.poolAdminFeeDestination[pool], this.feeReceiver)`
    - **What is controllable?** The caller controls `pool`. The fee rates and recipients are loaded from factory storage.
    - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
    - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts the transaction. The pool handles failed token transfers by retaining the unpaid fees, so a failed payout does not necessarily revert collection. The factory's `nonReentrant` modifier blocks reentry into its guarded functions; unguarded functions remain subject to their own access checks.

Function: `createPool(PoolParameters params)`

This function is the core factory function for deploying a new Metric OMM pool and serves as a public interface callable by anyone. The caller provides all parameters required for pool creation via the `PoolParameters` struct, including the token pair, price provider, extensions, fee configuration, bin layout, and other operational parameters. The function performs validation on the provided parameters, deploys the pool using the MetricOmmPoolDeployer, and initializes the pool with the specified configuration. The pool metadata (`poolAdmin`, `priceProviderTimelock`, `poolFeeConfig`, and `poolAdminFeeDestination`) is recorded in the factory.

Inputs

- `params.token0`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: Must not be the zero address and must differ from `params.token1`.
    - **Impact**: Set as the immutable token0 address of the pool.
- `params.token1`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: Must not be the zero address and must differ from `params.token0`.
    - **Impact**: Set as the immutable token1 address of the pool.
- `params.priceProvider`
    - **Control**: Fully controlled by the caller.

  - **Constraints**: Must not be the zero address, must implement `IPriceProvider` interface, and must return the same token0 and token1 as provided in `params`.
  - **Impact**: Set as the pool's price oracle — fixed as immutable if `params.priceProviderTimelock` is `type(uint256).max`.
- `params.extensions`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: The length of the array must not exceed `ValidateExtensionsConfig.MAX_EXTENSIONS` (7). Each extension address must be nonzero and unique.
  - **Impact**: Addresses of extensions to be attached to the pool operations.
- `params.extensionOrders`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: If `params.extensions` is empty, all fields must be zero. Otherwise, each per-action order must be a nonzero bitmask containing only indexes from 1 to `params.extensions.length`, with no duplicates.
  - **Impact**: Per-action extension call orders.
- `params.extensionInitData`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must have the same length as `params.extensions`.
  - **Impact**: Per-extension initialization calldata.
- `params.priceProviderTimelock`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: N/A.
  - **Impact**: Stored in the pool's `priceProviderTimelock` mapping and used as the waiting period when replacing the price provider.
- `params.admin`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must not be the zero address.
  - **Impact**: Stored in the pool's `poolAdmin` mapping and used for pool administration.
- `params.initialScaledAmount0PerE18Shares`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must be nonzero and less than `type(uint128).max`.
  - **Impact**: Token0 empty-bin mint density in scaled precision for 1e18 shares.
- `params.initialScaledAmount1PerE18Shares`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must be nonzero and less than `type(uint128).max`.
  - **Impact**: Token1 empty-bin mint density in scaled precision for 1e18 shares.

- `params.minimalOperationalLiquidity`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: Must not be zero and must be at least `1e6 * 1e18 / min(initialScaledAmount0PerE18Shares, initialScaledAmount1PerE18Shares)`.
    - **Impact**: Set as the minimum pre-bin share delta for add/remove and the minimum nonzero position size.
- `params.protocolSpreadFeeE6`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: Must not exceed the factory's `maxProtocolSpreadFeeE6`.
    - **Impact**: Protocol spread fee for this pool in 1e6.
- `params.adminSpreadFeeE6`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: Must not exceed the factory's `maxAdminSpreadFeeE6`.
    - **Impact**: Admin spread fee for this pool in 1e6.
- `params.protocolNotionalFeeE8`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: Must not exceed the factory's `maxProtocolNotionalFeeE8`.
    - **Impact**: Protocol notional fee for this pool in 1e8.
- `params.adminNotionalFeeE8`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: Must not exceed the factory's `maxAdminNotionalFeeE8`.
    - **Impact**: Admin notional fee for this pool in 1e8.
- `params.adminFeeDestination`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: Must not be the zero address.
    - **Impact**: The address that receives admin fees. Stored in the `poolAdminFeeDestination` mapping.
- `params.curBinDistFromProvidedPriceE6`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: Must be within the factory's `minBinDistanceE6` and `maxBinDistanceE6` limits.
    - **Impact**: Distance of the current active bin relative to the provided price.
- `params.nonNegativeBinDataArray`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: The length of the array must not be zero. There must be at least one bin with nonzero length, and the sum of all positive-side bin lengths plus `params.curBinDistFromProvidedPriceE6` must not exceed

`maxBinDistanceE6`. The total bin count must not exceed `MAX_BINS_PER_SIDE` (32,768).

- **Impact**: Determine the positive-side bin layout (length, buy/sell additional fees).

- `params.negativeBinDataArray`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: The length of the array must not be zero. There must be at least one bin with nonzero length, and the sum of all negative-side bin lengths plus `params.curBinDistFromProvidedPriceE6` must not exceed `minBinDistanceE6`. The total bin count must not exceed `MAX_BINS_PER_SIDE` (32,768).
  - **Impact**: Determine the negative-side bin layout.
- `params.salt`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: N/A.
  - **Impact**: Salt used for deterministic pool address derivation.
- `params.bidAskRefAsymmetryX8`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must be nonzero and must not exceed the factory's `onCreationMaxBidAskRefAsymmetryX8`.
  - **Impact**: Used to limit how asymmetrically the oracle reference price may split the bid-ask spread, reverting quotes that deviate too far from the geometric midpoint.

Branches and code coverage

Intended branches

- A valid call with a configured deployer deploys the pool and records the pool index and factory metadata.
  - ☑ Test coverage
- Protocol fee inputs below the factory defaults are raised to the defaults, while inputs above the defaults are retained when within caps.
  - ☑ Test coverage
- A `type(uint256).max` price-provider timelock selects immutable mode; a finite timelock selects mutable mode.
  - ☑ Test coverage
- The extension initialization loop handles zero extensions and initializes a configured extension.
  - ☑ Test coverage

- Bin unpacking accepts an empty negative-side array, exact configured distance boundaries, and grids larger than the former policy limit while enforcing the `int16` index limit.

  ☑ Test coverage

Negative behavior

- The call reverts with `PoolDeployerNotSet` if the factory has no configured deployer.

  ☐ Negative test
- The call reverts if the `params.token0` or `params.token1` is the zero address or if they are equal.

  ☐ Negative test
- The call reverts if the `params.priceProvider` is the zero address or if it returns a different token pair than `params.token0` and `params.token1`.

  ☐ Negative test
- The call reverts if the `params.admin` is the zero address.

  ☐ Negative test
- The call reverts if the `params.adminFeeDestination` is the zero address.

  ☐ Negative test
- The call reverts if the `params.protocolNotionalFeeE8` exceeds the factory's `maxProtocolNotionalFeeE8`.

  ☐ Negative test
- The call reverts if the `params.protocolSpreadFeeE6` exceeds the factory's `maxProtocolSpreadFeeE6`.

  ☑ Negative test
- The call reverts if the `params.adminSpreadFeeE6` exceeds the factory's `maxAdminSpreadFeeE6`.

  ☐ Negative test
- The call reverts if the `params.adminNotionalFeeE8` exceeds the factory's `maxAdminNotionalFeeE8`.

  ☑ Negative test
- The call reverts if the `params.bidAskRefAsymmetryX8` is zero or exceeds the factory's `onCreationMaxBidAskRefAsymmetryX8`.

  ☑ Negative test
- The call reverts if the initial scaled amounts are invalid or if the minimum liquidity is insufficient.

  ☑ Negative test
- Bin validation rejects an empty non-negative array, out-of-range current or cumulative

distances, and bin counts whose indexes exceed `int16`.

- ☑ Negative test

Function call analysis

- `MetricOmmPoolDeployer(this.poolDeployer).deploy(DeployParams(...))`
  - **What is controllable?** The `poolDeployer` can only be set by the owner or authorized users and is therefore trusted; all fields in `DeployParams` are values that have been validated earlier.
  - **If the return value is controllable, how is it used and how can it go wrong?** The returned pool address is used as a key in `poolAdmin`, `poolFeeConfig`, and so forth; it reverts on deployment failure; `CREATE2` collision occurs on reuse of the same salt.
  - **What happens if it reverts, reenters or does other unusual control flow?** On revert, the entire `createPool` call is canceled; reentrancy is not possible within the current pool constructor implementation.
- `CallExtension.callExtension(params.extensions[i], abi.encodeCall(fptr(contracts/interfaces/extensions/IMetricOmmExtensions.sol.IM pool, params.extensionInitData[i]))`
  - **What is controllable?** The caller controls the extension address and the initialization data.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** On revert, the entire `createPool` call is canceled; reentrancy is possible if the extension calls back into the factory, and this function has no reentrancy guard. During the extension initialization, the pool has not been recorded in the factory state, but if reentrancy attempts to redeploy with the same salt, the second deployment will fail due to a `CREATE2` collision.

Function: `setBinDistanceLimits(int128 newMinBinDistanceE6, int128 newMaxBinDistanceE6)`

This function allows the factory owner or authorized users to configure the bin distance range allowed for new pools. It updates `minBinDistanceE6` and `maxBinDistanceE6`, which are checked against the initial and cumulative bin distances during pool creation. Existing pools retain their bin layouts. A `BinDistanceLimitsUpdated` event is emitted when either limit changes; unchanged limits return without emitting an event.

Inputs

- `newMinBinDistanceE6`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must be at least `HARD_MIN_BIN_DISTANCE_LIMIT_E6` (`-999_999`) and less than zero.
  - **Impact**: The minimum allowed bin distance for new pools in 1e6.
- `newMaxBinDistanceE6`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must be greater than zero and must not exceed `HARD_MAX_BIN_DISTANCE_LIMIT_E6` (`type(int24).max`).
  - **Impact**: The maximum allowed bin distance for new pools in 1e6.

Branches and code coverage

Intended branches

- Valid limits update `minBinDistanceE6` and `maxBinDistanceE6`.
  - ☑ Test coverage
- A `BinDistanceLimitsUpdated` event is emitted when the limits change.
  - ☑ Test coverage
- Widening or tightening the limits changes which bin layouts are accepted by subsequent `createPool()` calls.
  - ☑ Test coverage
- If both limits equal their stored values, the function returns without changing state or emitting an event.
  - ☐ Test coverage

Negative behavior

- The call reverts with `InvalidBinDistanceLimits` if the minimum is below `HARD_MIN_BIN_DISTANCE_LIMIT_E6` or is nonnegative.
  - ☑ Negative test
- The call reverts with `InvalidBinDistanceLimits` if the maximum exceeds `HARD_MAX_BIN_DISTANCE_LIMIT_E6` or is nonpositive.
  - ☑ Negative test
- The call reverts if the caller is neither the factory owner nor authorized users.
  - ☑ Negative test

Function: `setDefaultProtocolNotionalFeeE8(uint24 newFeeE8)`

This function allows the factory owner or authorized users to set the minimum protocol notional fee for new pools. It validates the value against `maxProtocolNotionalFeeE8` and updates `protocolNotionalFeeE8`. When the value changes, a `ProtocolNotionalFeeDefaultUpdated` event is emitted; an unchanged value returns without emitting an event. New pools use the greater of this default and the caller-supplied protocol notional fee.

Inputs

- `newFeeE8`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: It must not exceed `maxProtocolNotionalFeeE8`.
  - **Impact**: The new minimum protocol notional fee for new pools in 1e8.

Branches and code coverage

Intended branches

- A valid changed value updates `protocolNotionalFeeE8` and is used as the minimum protocol notional fee for subsequently created pools.
  - ☑ Test coverage
- A `ProtocolNotionalFeeDefaultUpdated` event is emitted when the value changes.
  - ☐ Test coverage
- If `newFeeE8` equals `protocolNotionalFeeE8`, the function returns without changing state or emitting an event.
  - ☐ Test coverage

Negative behavior

- The call reverts with `ProtocolFeeTooHigh` if `newFeeE8` exceeds `maxProtocolNotionalFeeE8`.
  - ☑ Negative test
- The call reverts if the caller is neither the factory owner nor authorized users.
  - ☐ Negative test

Function: `setDefaultSpreadProtocolFeeE6(uint24 newFeeE6)`

This function allows the factory owner or authorized users to set the minimum spread protocol fee for new pools. The function updates the state variable `spreadProtocolFeeE6` with the provided value. If the new fee is less than the current `spreadProtocolFeeE6`, it is raised to the new value; if it

is greater, it is retained. A `SpreadProtocolFeeDefaultUpdated` event is emitted upon successful configuration.

Inputs

- `newFeeE6`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: It must not exceed `maxSpreadProtocolFeeE6`.
  - **Impact**: The new minimum spread protocol fee for new pools.

Branches and code coverage

Intended branches

- The state variable `spreadProtocolFeeE6` is updated to the new value if it is valid.
  - ☑ Test coverage
- A `SpreadProtocolFeeDefaultUpdated` event is emitted upon successful update.
  - ☐ Test coverage

Negative behavior

- The call reverts if the `newFeeE6` exceeds `maxSpreadProtocolFeeE6`.
  - ☑ Negative test
- The call reverts if the caller is not the factory owner or an authorized user.
  - ☐ Negative test

Function: `setFeeCaps(uint24 newMaxProtocolSpreadFeeE6, uint24 newMaxAdminSpreadFeeE6, uint24 newMaxProtocolNotionalFeeE8, uint24 newMaxAdminNotionalFeeE8)`

This function allows the factory owner or authorized users to set the maximum fee caps for protocol and admin fees. The function updates the state variables `maxProtocolSpreadFeeE6`, `maxAdminSpreadFeeE6`, `maxProtocolNotionalFeeE8`, and `maxAdminNotionalFeeE8` with the provided values. If the new max protocol spread fee is less than `spreadProtocolFeeE6`, the value of `spreadProtocolFeeE6` is set to that new value; if the new max protocol notional fee is less than `protocolNotionalFeeE8`, the value of `protocolNotionalFeeE8` is set to that new value.

Inputs

- `newMaxProtocolSpreadFeeE6`

    - **Control**: Fully controlled by the caller.
    - **Constraints**: Must not exceed the hard maximum protocol spread fee (500_000).
    - **Impact**: The new maximum protocol spread fee in 1e6.
- `newMaxAdminSpreadFeeE6`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: Must not exceed the hard maximum admin spread fee (500_000).
    - **Impact**: The new maximum admin spread fee in 1e6.
- `newMaxProtocolNotionalFeeE8`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: Must not exceed the hard maximum protocol notional fee (1_000_000).
    - **Impact**: The new maximum protocol notional fee in 1e8.
- `newMaxAdminNotionalFeeE8`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: Must not exceed the hard maximum admin notional fee (1_000_000).
    - **Impact**: The new maximum admin notional fee in 1e8.

Branches and code coverage

Intended branches

- Valid inputs update the fee caps.
    - ☑ Test coverage
- If the new max protocol spread fee is less than `spreadProtocolFeeE6`, update `spreadProtocolFeeE6` to the new value.
    - ☐ Test coverage
- If the new max protocol notional fee is less than `protocolNotionalFeeE8`, update `protocolNotionalFeeE8` to the new value.
    - ☐ Test coverage

Negative behavior

- The call reverts if any of the new max fees exceed their respective hard maximums.
    - ☑ Negative test
- The call reverts if the caller is not the factory owner or an authorized user.
    - ☑ Negative test

Function: `setFeeReceiver(address newFeeReceiver)`

This function sets the address that receives protocol fees collected by the factory. Restricted by `requiresAuth`, only the factory owner and authorized users can call it. A `FeeReceiverUpdated` event is emitted upon successful configuration.

Inputs

- `newFeeReceiver`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must not be the zero address.
  - **Impact**: The address that will receive protocol fees collected by the factory.

Branches and code coverage

Intended branches

- The state variable `feeReceiver` is updated to the new address.
  - ☑ Test coverage

Negative behavior

- The call reverts with `InvalidFeeReceiver` if the new address is the zero address.
  - ☑ Negative test
- The call reverts if the caller is not the factory owner or an authorized user.
  - ☑ Negative test

Function: `setOnCreationMaxBidAskRefAsymmetryX8(uint16 newMaxX8)`

This function allows the factory owner or authorized users to update `onCreationMaxBidAskRefAsymmetryX8`, the maximum asymmetry setting accepted during pool creation. It does not change existing pools or constrain subsequent calls to `setPoolBidAskRefAsymmetryX8()` or its batch variant. A value of `type(uint16).max` allows new pools to disable the asymmetry check. An `OnCreationMaxBidAskRefAsymmetryUpdated` event is emitted when the cap changes; an unchanged value returns without emitting an event.

Inputs

- `newMaxX8`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must be nonzero.

- **Impact**: The maximum `bidAskRefAsymmetryX8` accepted for new pools.

Branches and code coverage

Intended branches

- A valid value updates `onCreationMaxBidAskRefAsymmetryX8`.
  - ☑ Test coverage
- An `OnCreationMaxBidAskRefAsymmetryUpdated` event is emitted when the cap changes.
  - ☐ Test coverage
- If `newMaxX8` equals the stored cap, the function returns without changing state or emitting an event.
  - ☐ Test coverage

Negative behavior

- The call reverts with `BidAskRefAsymmetryTooLow` if `newMaxX8` is zero.
  - ☑ Negative test
- The call reverts if the caller is neither the factory owner nor authorized users.
  - ☐ Negative test

Function: `setPoolAdminFeeDestination(address pool, address newAdminFeeDestination)`

This function allows the pool admin to change the recipient of admin fees. It first calls `_collectFees()` using the existing fee configuration and destination. After collection returns, it updates `poolAdminFeeDestination[pool]` and emits `PoolAdminFeeDestinationUpdated`. Fees that could not be paid to the old destination remain pending in the pool and are retried using the new destination on subsequent collection calls.

Inputs

- `pool`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: The caller must equal `poolAdmin[pool]`.
  - **Impact**: The pool whose accrued fees are collected and whose admin fee destination is updated.
- `newAdminFeeDestination`
  - **Control**: Fully controlled by the caller.

- **Constraints**: Must not be the zero address.
- **Impact**: The recipient for subsequent admin fee payouts.

Branches and code coverage

Intended branches

- A valid call by the pool admin updates `poolAdminFeeDestination[pool]`.
  - ☑ Test coverage
- A `PoolAdminFeeDestinationUpdated` event is emitted after the update.
  - ☑ Test coverage
- Accrued fees are paid using the old admin destination before the destination changes.
  - ☐ Test coverage

Negative behavior

- The call reverts with `InvalidAdminFeeDestination` if the new destination is zero.
  - ☐ Negative test
- The call reverts with `NotPoolAdmin` if the caller does not equal `poolAdmin[pool]`.
  - ☐ Negative test

Function call analysis

- `this._collectFees(pool, this.poolFeeConfig[pool]) -> IMetricOmmPoolCollectFees(pool).collectFees(c.protocolSpreadFeeE6, c.adminSpreadFeeE6, c.protocolNotionalFeeE8, c.adminNotionalFeeE8, this.poolAdminFeeDestination[pool], this.feeReceiver)`
  - **What is controllable?** The caller selects `pool`, subject to the pool-admin check. The fee rates and recipients are loaded from factory storage.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert cancels the entire call, leaving the destination unchanged. During collection, callbacks observe the old destination. The factory's `nonReentrant` modifier blocks reentry into its guarded functions; unguarded functions remain subject to their own access checks.

Function: `setPoolBidAskRefAsymmetryX8(address pool, uint16 newBidAskRefAsymmetryX8)`

This function allows the factory owner or authorized users to update a pool's bid-ask reference asymmetry tolerance. It forwards the value to the pool and emits `PoolBidAskRefAsymmetryUpdated`. Setting `type(uint16).max` disables the pool's asymmetry check.

Inputs

- `pool`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: The factory does not verify pool registration. The target must accept the `setBidAskRefAsymmetryX8()` call; a MetricOmmPool requires the caller to be its factory.
  - **Impact**: The pool whose asymmetry tolerance is updated.
- `newBidAskRefAsymmetryX8`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must be nonzero.
  - **Impact**: The new tolerance for the imbalance between the buy-side and sell-side returns around the oracle reference.

Branches and code coverage

Intended branches

- A valid value updates the pool's `bidAskRefAsymmetryX8`, including a value above the creation cap.
  - ☑ Test coverage
- A `PoolBidAskRefAsymmetryUpdated` event is emitted after the pool call succeeds.
  - ☑ Test coverage

Negative behavior

- The call reverts with `BidAskRefAsymmetryTooLow` if the new value is zero.
  - ☑ Negative test
- The call reverts if the caller is neither the factory owner nor authorized users.
  - ☐ Negative test

Function call analysis

- `this._updatePoolBidAskRefAsymmetryX8(pool, newBidAskRefAsymmetryX8) -> IMetricOmmPoolFactoryActions(pool).setBidAskRefAsymmetryX8(newBidAskRefAsymmetryX8)`
    - **What is controllable?** `pool` and `newBidAskRefAsymmetryX8`.
    - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
    - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts the entire call. The factory's `nonReentrant` modifier prevents entry into its guarded functions during the pool call. Unguarded functions remain subject to their own access checks.

Function: `setPoolBidAskRefAsymmetryX8Batch(address[] pools, uint16[] newBidAskRefAsymmetryX8s)`

This function allows the factory owner or authorized users to update the asymmetry tolerance of multiple pools in one transaction.

Inputs

- `pools`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: The length must match `newBidAskRefAsymmetryX8s`. The factory does not verify pool registration or reject duplicate addresses. Each target must accept the setter call.
    - **Impact**: The pool addresses to update.
- `newBidAskRefAsymmetryX8s`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: The length must match `pools`, and every value must be nonzero.
    - **Impact**: The new asymmetry tolerance for each corresponding pool; `type(uint16).max` disables the check.

Branches and code coverage

Intended branches

- Valid arrays update all specified pools.
    - ☑ Test coverage
- A `PoolBidAskRefAsymmetryUpdated` event is emitted for each successful update.

☑ Test coverage

Negative behavior

- The call reverts with `ArrayLengthMismatch` if the array lengths differ.
  - ☑ Negative test
- A zero entry reverts with `BidAskRefAsymmetryTooLow` and rolls back earlier pool updates.
  - ☑ Negative test
- The call reverts if the caller is neither the factory owner nor authorized users.
  - ☐ Negative test

Function call analysis

- `this._updatePoolBidAskRefAsymmetryX8(pool, newBidAskRefAsymmetryX8) -> IMetricOmmPoolFactoryActions(pool).setBidAskRefAsymmetryX8(newBidAskRefAsymmetryX8)`
  - **What is controllable?** `pool` and `newBidAskRefAsymmetryX8`.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts the entire call. The factory's `nonReentrant` modifier prevents entry into its guarded functions during the pool call. Unguarded functions remain subject to their own access checks.

Function: `setPoolDeployer(address _poolDeployer)`

This function sets the MetricOmmPoolDeployer contract address used by the factory when deploying pools and can only be set once. Restricted by `requiresAuth`, only the factory owner and authorized users can call it; once set, it cannot be changed and any attempt results in a `PoolDeployerAlreadySet` error. A `PoolDeployerSet` event is emitted upon successful configuration, and while this value is `address(0)`, the `createPool` function itself does not work.

Inputs

- `_poolDeployer`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: The current value of `poolDeployer` must be `address(0)` (unset) to allow setting.
  - **Impact**: The address of the MetricOmmPoolDeployer contract used for pool deployment.

Branches and code coverage

Intended branches

- When called with a valid address and `poolDeployer == address(0)`, the deployer address is set and a `PoolDeployerSet` event is emitted.
  - ☑ Test coverage

Negative behavior

- The call reverts with `PoolDeployerAlreadySet` if the current value of `poolDeployer` is not `address(0)`.
  - ☐ Negative test
- The call reverts if the caller is not the factory owner or an authorized user.
  - ☐ Negative test

Function: `setPoolProtocolFee(address pool, uint24 newProtocolSpreadFeeE6, uint24 newProtocolNotionalFeeE8)`

This function allows the factory owner or authorized users to update the protocol fees for a specific pool. It first collects any accrued fees from the pool then updates the protocol spread and notional fees to the new values provided. If the current admin fees of the pool exceed the maximum allowed values stored in the factory, they are capped to the maximums.

Inputs

- `pool`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: N/A.
  - **Impact**: The address of the target pool whose protocol fees are to be updated.
- `newProtocolSpreadFeeE6`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must not exceed the `maxProtocolSpreadFeeE6`.
  - **Impact**: The new protocol spread fee in 1e6.
- `newProtocolNotionalFeeE8`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must not exceed the `maxProtocolNotionalFeeE8`.
  - **Impact**: The new protocol notional fee in 1e8.

Branches and code coverage

Intended branches

- Valid values update the protocol fees for the specified pool.
  - ☑ Test coverage
- If the current `adminSpreadFeeE6` of the pool exceeds `maxAdminSpreadFeeE6`, it is capped to `maxAdminSpreadFeeE6`.
  - ☑ Test coverage
- If the current `adminNotionalFeeE8` of the pool exceeds `maxAdminNotionalFeeE8`, it is capped to `maxAdminNotionalFeeE8`.
  - ☑ Test coverage
- The `spreadFeeE6` of the pool is updated to the sum of the new protocol spread fee and the capped admin spread fee.
  - ☐ Test coverage
- The `notionalFeeE8` of the pool is updated to the sum of the new protocol notional fee and the capped admin notional fee.
  - ☐ Test coverage

Negative behavior

- The call reverts if the new protocol spread fee exceeds `maxProtocolSpreadFeeE6`.
  - ☑ Negative test
- The call reverts if the new protocol notional fee exceeds `maxProtocolNotionalFeeE8`.
  - ☑ Negative test
- The call reverts if the caller is not the factory owner or an authorized user.
  - ☑ Negative test

Function call analysis

- `this._updatePoolProtocolFee(pool, newProtocolSpreadFeeE6, newProtocolNotionalFeeE8) -> this._collectFees(pool, c) -> IMetricOmmPoolCollectFees(pool).collectFees(c.protocolSpreadFeeE6, c.adminSpreadFeeE6, c.protocolNotionalFeeE8, c.adminNotionalFeeE8, this.poolAdminFeeDestination[pool], this.feeReceiver)`
  - **What is controllable?** The `pool` address is fully controlled by the caller; the other arguments are obtained from the factory storage.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.

    - **What happens if it reverts, reenters or does other unusual control flow?** On revert, the entire `setPoolProtocolFee` call is canceled. If the pool was malicious, it could reenter the factory and call functions like `createPool` or `collectPoolFees`, but this is no different from calling them directly.
- `this._updatePoolProtocolFee(pool, newProtocolSpreadFeeE6, newProtocolNotionalFeeE8) -> IMetricOmmPoolFactoryActions(pool).setPoolFees(c.protocolSpreadFeeE6 + c.adminSpreadFeeE6, c.protocolNotionalFeeE8 + c.adminNotionalFeeE8)`
    - **What is controllable?** The `pool` address and the protocol fees.
    - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
    - **What happens if it reverts, reenters or does other unusual control flow?** On revert, the entire `setPoolProtocolFee` call is canceled. If the pool was malicious, it could reenter the factory and call functions like `createPool` or `collectPoolFees`, but this is no different from calling them directly.

Function: `setPoolProtocolFeeBatch(address[] pools, uint24[] newProtocolSpreadFeeE6s, uint24[] newProtocolNotionalFeeE8s)`

This function allows the factory owner or authorized users to update the protocol fees for multiple pools in a single transaction. It iterates over the provided arrays of pool addresses and updates protocol spread and notional fees for each pool. If a pool's current admin fees exceed the maximum allowed values stored in the factory, they are capped to the maximums.

Inputs

- `pools`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: N/A.
    - **Impact**: The array of pool addresses whose protocol fees are to be updated.
- `newProtocolSpreadFeeE6s`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: The length must match the `pools` array. Each value must not exceed the `maxProtocolSpreadFeeE6`.
    - **Impact**: The new protocol spread fees for the corresponding pools.
- `newProtocolNotionalFeeE8s`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: The length must match the `pools` array. Each value must not exceed the `maxProtocolNotionalFeeE8`.
    - **Impact**: The new protocol notional fees for the corresponding pools.

Branches and code coverage

Intended branches

- Valid inputs update the protocol fees for the specified pools.
  - ☑ Test coverage
- If a pool's current `adminSpreadFeeE6` exceeds `maxAdminSpreadFeeE6`, it is capped to `maxAdminSpreadFeeE6`.
  - ☐ Test coverage
- If a pool's current `adminNotionalFeeE8` exceeds `maxAdminNotionalFeeE8`, it is capped to `maxAdminNotionalFeeE8`.
  - ☐ Test coverage
- Each pool's `spreadFeeE6` is updated to the sum of the new protocol spread fee and the capped admin spread fee.
  - ☐ Test coverage
- Each pool's `notionalFeeE8` is updated to the sum of the new protocol notional fee and the capped admin notional fee.
  - ☐ Test coverage

Negative behavior

- The call reverts if the lengths of the input arrays do not match.
  - ☑ Negative test
- The call reverts if any of the new protocol spread fees exceed `maxProtocolSpreadFeeE6`.
  - ☑ Negative test
- The call reverts if any of the new protocol notional fees exceed `maxProtocolNotionalFeeE8`.
  - ☑ Negative test
- The call reverts if the caller is not the factory owner or an authorized user.
  - ☐ Negative test

5.3. DepositAllowlistExtension.sol

Function: `function setAllowAllDepositors(address pool_, bool allowed) external onlyPoolAdmin(pool_)`

The pool administrator immediately changes the global depositor permission flag and emits a configuration event. No timelock or additional signature is required.

Inputs

- `pool_`
  - **Control**: Pool administrator.
  - **Constraints**: N/A.
  - **Impact**: Selects the pool-specific storage key to modify.
- `allowed`
  - **Control**: Pool administrator.
  - **Constraints**: N/A.
  - **Impact**: Changes the global permission flag.
- `msg.sender`
  - **Control**: Transaction caller.
  - **Constraints**: Must equal `FACTORY.poolAdmin(pool_)`.
  - **Impact**: Determines authority to change the pool-specific policy.

Branches and code coverage

Intended branches

- Setting `true` enables global access regardless of the individual allowlist.
  - ☑ Test coverage
- If `msg.sender` matches the pool admin returned by `FACTORY`, stores `allowed` unchanged in the pool's global permission flag and emits the corresponding `Set` event with `pool_` and the configured value. It does not change other permission mappings or flags.
  - ☐ Test coverage

Negative branches

- A non-administrator reverts with `OnlyPoolAdmin`.
  - ☑ Test coverage
- If the `FACTORY.poolAdmin(pool_)` external query in `onlyPoolAdmin` reverts, the original error propagates. An invalid returned address ABI value causes an ABI decoding revert.
  - ☐ Test coverage

Function: `function setAllowedToDeposit(address pool_, address depositor, bool allowed) external onlyPoolAdmin(pool_)`

The pool administrator immediately updates an individual address's permission status and emits a configuration event. No timelock or additional signature is required.

Inputs

- `pool_`
  - **Control**: Pool administrator.
  - **Constraints**: N/A.
  - **Impact**: Selects the pool-specific storage key to modify.
- `depositor`
  - **Control**: Pool administrator.
  - **Constraints**: N/A.
  - **Impact**: Records the permission status of the position owner that the next `beforeAddLiquidity` call will check. The owner may differ from the pool caller or token payer.
- `allowed`
  - **Control**: Pool administrator.
  - **Constraints**: N/A.
  - **Impact**: Overwrites the individual permission mapping.
- `msg.sender`
  - **Control**: Transaction caller.
  - **Constraints**: Must equal `FACTORY.poolAdmin(pool_)`.
  - **Impact**: Determines authority to change the pool-specific policy.

Branches and code coverage

Intended branches

- Setting `true` allows the target address.
  - ☑ Test coverage
- If `msg.sender` matches the pool admin returned by `FACTORY`, stores `allowed` unchanged in the specified address's individual permission mapping and emits the corresponding `Set` event with `pool_` and the configured value. It does not change other permission mappings or flags.
  - ☐ Test coverage

Negative branches

- A non-administrator reverts with `OnlyPoolAdmin`.
  - ☑ Test coverage
- If the `FACTORY.poolAdmin(pool_)` external query in `onlyPoolAdmin` reverts, the original error propagates. An invalid returned address ABI value causes an ABI decoding revert.
  - ☐ Test coverage

5.4. MetricOmmPoolLiquidityAdder.sol

Function: `function addLiquidityExactShares(address pool, address owner, uint80 salt, LiquidityDelta calldata deltas, uint256 maxAmountToken0, uint256 maxAmountToken1, BinPositionBounds calldata binPositionBounds, bytes calldata extensionData) external payable override returns (uint256 amount0Added, uint256 amount1Added)`

This function adds explicit per-bin shares for `owner`. The external caller remains the callback payer.

Inputs

- `pool`
  - **Control**: Arbitrary.
  - **Constraints**: Must be factory-registered.
  - **Impact**: Identifies the target pool.
- `owner`
  - **Control**: Arbitrary.
  - **Constraints**: Must be valid under the position validation.
  - **Impact**: Identifies the position owner.
- `salt`
  - **Control**: Arbitrary.
  - **Constraints**: N/A.
  - **Impact**: Identifies the position.
- `deltas`
  - **Control**: Arbitrary.
  - **Constraints**: Bin and share arrays must be nonempty and have matching lengths.
  - **Impact**: Supplies per-bin share changes.
- `maxAmountToken0`
  - **Control**: Arbitrary.
  - **Constraints**: Callback token0 payment must not exceed this value.
  - **Impact**: Caps token0 settlement.
- `maxAmountToken1`
  - **Control**: Arbitrary.
  - **Constraints**: Callback token1 payment must not exceed this value.
  - **Impact**: Caps token1 settlement.
- `binPositionBounds`

    - **Control**: Arbitrary.
    - **Constraints**: The current bin and position must satisfy these bounds.
    - **Impact**: Limits acceptable pool state.
- `extensionData`
    - **Control**: Arbitrary.
    - **Constraints**: N/A.
    - **Impact**: Is forwarded to the pool.
- `msg.value`
    - **Control**: Arbitrary.
    - **Constraints**: N/A.
    - **Impact**: May fund WETH settlement.
- `msg.sender`
    - **Control**: Arbitrary.
    - **Constraints**: N/A.
    - **Impact**: Becomes the callback payer.

Branches and code coverage

Intended branches

- Validates the pool, owner, deltas, and bin-position bounds then invokes pool liquidity addition.
    - ☑ Test coverage
- Stores transient pool, payer, and token caps before payment; the callback pays positive token deltas and clears the context.
    - ☐ Test coverage

Negative branches

- An unregistered pool, invalid owner, empty or mismatched deltas, or out-of-bounds bin position causes validation to revert.
    - ☐ Test coverage
- An active payment context, unexpected callback caller, or callback amount above either cap causes the addition to revert.
    - ☑ Test coverage
- Failed token payment, malformed pool response, or a reverting pool or extension bubbles and reverts the addition.
    - ☐ Test coverage

Function call analysis

- `_requireFactoryPool(pool)`
    - **What is controllable?** The caller selects the pool, owner, bin/share arrays. The immutable configured factory supplies registry membership.
    - **If the return value is controllable, how is it used and how can it go wrong?** `FACTORY.isPool(pool)` is a STATICCALL decoded as `bool`, with false rejected as `InvalidPool`; the pinned factory returns `poolToIdx[pool] != 0`. The local validators return no value and reject zero owner, require nonempty equal-length arrays. They do not calculate token requirements or establish that arbitrary registered code is trustworthy.
    - **What happens if it reverts, reenters, or does other unusual control flow?** Factory revert or malformed response propagates; its static context prohibits state-changing reentry. Pure validation errors revert before context setup or asset calls. Pool-specific bin, share-size, and minimum-liquidity checks execute later in the pool.
- `_validateBinAndBinPosition(pool, binPositionBounds)`
    - **What is controllable?** The caller supplies pool and inclusive bin/position bounds. The selected pool supplies the packed slot-0 word; interpretation depends on the pinned storage layout.
    - **If the return value is controllable, how is it used and how can it go wrong?** The external read is a STATICCALL decoded as `bytes32`, then `_decodeSlot0` uses `SlotOLibrary.unpack(uint256(data))`. The adder uses only current bin and position: reversed bin bounds or a value outside the bin interval/edge-position limits causes `BinPositionOutOfBounds`. A fabricated word from a nonconforming registered pool could pass the precondition without reflecting its execution state; no post-add bound check is made.
    - **What happens if it reverts, reenters, or does other unusual control flow?** Malformed return or extsload revert propagates. The static read cannot mutate through reentry. The check occurs before pool extension CALLs, so it is a snapshot of the precall state rather than a lock on all subsequent external behavior.
- `_addLiquidity(pool, owner, salt, deltas, msg.sender, maxAmountToken0, maxAmountToken1, extensionData)`
    - **What is controllable?** The entry fixes payer to its caller and supplies owner, salt, shares, caps, and extension data, and `_setPayContext` stores these settlement fields before a zero-ETH ordinary CALL from the adder to the selected registered pool.
    - **If the return value is controllable, how is it used and how can it go wrong?** `_setPayContext` uses `_tloadAddress` / `_tload` to reject a nonzero active pool (`PayContextAlreadyActive`) and `_tstoreAddress` / `_tstore` to write pool, payer, and caps. The pool's decoded `a0` / `a1` are copied directly into `amount0Added` / `amount1Added`; they are not independently checked against caps or actual shares/receipts. Caps are enforced on callback requests, not

on return values. A nonconforming accepted pool could skip the callback or return fabricated amounts.

- **What happens if it reverts, reenters, or does other unusual control flow?** On normal return, `_clearPayContext` clears all four transient slots, including when no callback was made. The `catch (bytes memory reason)` clears and reverts with the pool's original bytes; it does not swallow the error. A successful CALL with malformed ABI return fails in caller-side decoding and does not enter that catch, but the resulting invocation revert still rolls back context and nested effects. Active context blocks another paying add through `_setPayContext`; it does not guard payment utilities or every reentrant function.

- `LiquidityLib.addLiquidity(...)`

  - **What is controllable?** The selected pool's linked library, immutable token/scaling configuration, current bins, and configured extensions determine requirements. The caller's owner/salt/bins/shares and extension bytes are forwarded; the adder is the pool's sender.
  - **If the return value is controllable, how is it used and how can it go wrong?** `LiquidityLib.addLiquidity` is a linked-library DELEGATECALL in pool storage context, preserving the adder as `msg.sender`. The `_loadContext` makes STATICCALLs to the pool's `getImmutables()` and `extsload`; per-bin `_positionBinKey`, ceiling `Math.mulDiv`, SafeCast bounds, and `_deltasScaledToExternal` / `Math.ceilDiv` calculate required native amounts while updating pool shares/balances. Invalid bins, oversized shares/balances, or shares below the pool minimum revert; zero shares are skipped. When either amount is positive, token `balanceOf` STATICCALLs surround `IMetricOmmModifyLiquidityCallback(msg.sender).metricOmmModifyLiquidityCallback(amount0Added, amount1Added, callbackData)` and require the stated increases. No callback occurs when both amounts are zero. Return amounts describe computed requirements and rely on honest token balance replies.
  - **What happens if it reverts, reenters, or does other unusual control flow?** The pool's transient `nonReentrant(PoolActions.ADD_LIQUIDITY)` guard rejects overlapping guarded actions on that pool. Extension hooks use ordinary CALLs through `CallExtension.callExtension(extension, data)` — success needs at least 32 return bytes and the matching selector; nonempty revert bytes bubble, empty failure becomes `ExtensionCallFailed`, and invalid acknowledgment becomes `InvalidExtensionResponse`. A probe callback's deliberate revert unwinds library writes and before-hook effects; its after-hook is not reached. A successful paying callback clears adder context before the after-hook runs, permitting reentry to other adder operations subject to their checks; the still-active pool guard remains local to that pool. Any later uncaught failure rolls back the whole pool invocation.

- `abi.decode(callbackData, (uint8))`

  - **What is controllable?** The callback caller supplies kind and requested

amounts — `KIND_PROBE` is processed without caller authentication; `KIND_PAY` requires a nonzero stored pool, `msg.sender == expectedPool`, and each amount within the stored cap. Payer/caps are loaded from transient storage.

- **If the return value is controllable, how is it used and how can it go wrong?** The probe path returns no value and deliberately emits `LiquidityProbe(amount0Delta, amount1Delta)` as revert bytes before any pool/token call. For payment, `getImmutables()` is a STATICCALL; decoded `imm.token0` and `imm.token1` select token targets, with other fields ignored. A nonconforming accepted pool could return arbitrary tokens to pull from the stored payer. Cap checks constrain each callback's numeric amounts, not the economic identity or value of the returned tokens.
- **What happens if it reverts, reenters, or does other unusual control flow?** Malformed kind data, unknown kind, inactive/mismatched context, or cap excess reverts before payments; `getImmutables` revert/invalid ABI propagates, and static context forbids state-changing reentry. The callback does not requery factory membership; it relies on the pool stored after entry validation.

- `pay(token0, payer, msg.sender, amount0Delta)`
  - **What is controllable?** The authenticated expected pool sets amounts within the loaded caps and supplies token addresses through its immutable-data response. Only positive legs are paid, in token0-then-token1 order; the recipient is that pool.
  - **If the return value is controllable, how is it used and how can it go wrong?** These helpers return no values; `_clearPayContext` clears pool, payer, and both caps after both payments. It also executes for a zero/zero paying callback. Caps are checked per invocation and are neither decremented nor reserved before token interaction.
  - **What happens if it reverts, reenters, or does other unusual control flow?** Payment CALLs can reenter while context is still active. A direct callback from an unrelated token fails sender authentication; a nonconforming expected pool able to reenter its own callback during a payment could repeat requests before clearing. A second callback after a completed callback is rejected as inactive. If token1 fails after token0 succeeded, the callback revert rolls back token0 payment and any clearing. When the caller propagates the error, the surrounding liquidity operation also rolls back.
- `pay(token, payer, recipient, value)`
  - **What is controllable?** The entry's caller is the stored payer. The callback controls capped amounts and pool-derived token targets. Non-WETH payment calls `IERC20(token).safeTransferFrom(payer, recipient, value)`. WETH uses the adder's live ETH balance, wrapping the full amount or `nativeBalance`, transferring it to the pool, and pulling the uncovered remainder from the payer. This balance includes unrelated ETH leftovers.
  - **If the return value is controllable, how is it used and how can it go wrong?** Deposit forwards the wrapped ETH and has no return to inspect.

SafeERC20's zero-ETH token CALL accepts a successful checked word `1`, or successful empty return data from a code-bearing address, without a full ABI length check; nonaccepted responses cause `SafeERC20FailedOperation`. Neither wrapping success nor token success proves the pool received the expected asset; the conforming pool separately checks token balance increases. The helper's contract-funded `safeTransfer` branch applies if `payer == address(this)`, but the ordinary add entries store their external caller as payer.

- **What happens if it reverts, reenters, or does other unusual control flow?** WETH and token code can reenter while the paying context is active; no guard exists in `pay`. Failed token CALLs bubble revert bytes and deposit failures propagate. A later failed payment or pool check unwinds earlier wrapping and transfers in the same invocation.

Function: `function addLiquidityExactShares(address pool, uint80 salt, LiquidityDelta calldata deltas, uint256 maxAmountToken0, uint256 maxAmountToken1, BinPositionBounds calldata binPositionBounds, bytes calldata extensionData) external payable override returns (uint256 amount0Added, uint256 amount1Added)`

This function adds explicit per-bin shares for `msg.sender`, who is also the callback payer.

Inputs

- `pool`
  - **Control**: Arbitrary.
  - **Constraints**: Must be factory-registered.
  - **Impact**: Identifies the target pool.
- `salt`
  - **Control**: Arbitrary.
  - **Constraints**: N/A.
  - **Impact**: Identifies the caller-owned position.
- `deltas`
  - **Control**: Arbitrary.
  - **Constraints**: Bin and share arrays must be nonempty and have matching lengths.
  - **Impact**: Supplies per-bin share changes.
- `maxAmountToken0`
  - **Control**: Arbitrary.
  - **Constraints**: Callback token0 payment must not exceed this value.

    - **Impact**: Caps token0 settlement.
- `maxAmountToken1`
    - **Control**: Arbitrary.
    - **Constraints**: Callback token1 payment must not exceed this value.
    - **Impact**: Caps token1 settlement.
- `binPositionBounds`
    - **Control**: Arbitrary.
    - **Constraints**: The current bin and position must satisfy these bounds.
    - **Impact**: Limits acceptable pool state.
- `extensionData`
    - **Control**: Arbitrary.
    - **Constraints**: N/A.
    - **Impact**: Is forwarded to the pool.
- `msg.value`
    - **Control**: Arbitrary.
    - **Constraints**: N/A.
    - **Impact**: Adds ETH that may be wrapped for WETH settlement; payment can also use the contract's existing ETH balance.
- `msg.sender`
    - **Control**: Arbitrary.
    - **Constraints**: N/A.
    - **Impact**: Becomes both the position owner and the callback payer.

Branches and code coverage

Intended branches

- Uses `msg.sender` as owner and payer after validating the registered pool, delta arrays, and position bounds.
    - ☑ Test coverage
- Saves settlement context, executes `addLiquidity`, pays positive deltas in the callback, and clears context after payment.
    - ☐ Test coverage

Negative branches

- Invalid pool, deltas, or bin-position bounds causes validation to revert.
    - ☐ Test coverage
- A callback above either cap, an inactive context, invalid caller, or failed token transfer

causes the operation to revert.

- ☑ Test coverage

Function call analysis

- `_requireFactoryPool(pool)`
  - **What is controllable?** The caller selects the pool, bin/share arrays. The immutable configured factory supplies registry membership.
  - **If the return value is controllable, how is it used and how can it go wrong?** `FACTORY.isPool(pool)` is a STATICCALL decoded as `bool`, with false rejected as `InvalidPool`; the pinned factory returns `poolToIdx[pool] != 0`. The local validators return no value and require nonempty equal-length arrays. They do not calculate token requirements or establish that arbitrary registered code is trustworthy.
  - **What happens if it reverts, reenters, or does other unusual control flow?** Factory revert or malformed response propagates; its static context prohibits state-changing reentry. Pure validation errors revert before context setup or asset calls. Pool-specific bin, share-size, and minimum-liquidity checks execute later in the pool.
- `_validateBinAndBinPosition(pool, binPositionBounds)`
  - **What is controllable?** The caller supplies pool and inclusive bin/position bounds. The selected pool supplies the packed slot-0 word; interpretation depends on the pinned storage layout.
  - **If the return value is controllable, how is it used and how can it go wrong?** The external read is a STATICCALL decoded as `bytes32` — then `_decodeSlot0` uses `Slot0Library.unpack(uint256(data))`. The adder uses only current bin and position; reversed bin bounds or a value outside the bin interval/edge-position limits causes `BinPositionOutOfBounds`. A fabricated word from a nonconforming registered pool could pass the precondition without reflecting its execution state; no post-add bound check is made.
  - **What happens if it reverts, reenters, or does other unusual control flow?** Malformed return or extsload revert propagates. The static read cannot mutate through reentry. The check occurs before pool extension CALLs, so it is a snapshot of the precall state rather than a lock on all subsequent external behavior.
- `_addLiquidity(pool, msg.sender, salt, deltas, msg.sender, maxAmountToken0, maxAmountToken1, extensionData)`
  - **What is controllable?** The entry fixes payer to its caller and supplies owner, salt, shares, caps, and extension data, and `_setPayContext` stores these settlement fields before a zero-ETH ordinary CALL from the adder to the selected registered pool.

- **If the return value is controllable, how is it used and how can it go wrong?** `_setPayContext` uses `_tloadAddress` / `_tload` to reject a nonzero active pool (`PayContextAlreadyActive`) and `_tstoreAddress` / `_tstore` to write pool, payer, and caps. The pool's decoded `a0` / `a1` are copied directly into `amount0Added` / `amount1Added`; they are not independently checked against caps or actual shares/receipts. Caps are enforced on callback requests, not on return values. A nonconforming accepted pool could skip the callback or return fabricated amounts.
- **What happens if it reverts, reenters, or does other unusual control flow?** On normal return, `_clearPayContext` clears all four transient slots, including when no callback was made. The `catch (bytes memory reason)` clears and reverts with the pool's original bytes; it does not swallow the error. A successful CALL with malformed ABI return fails in caller-side decoding and does not enter that catch, but the resulting invocation revert still rolls back context and nested effects. Active context blocks another paying add through `_setPayContext`; it does not guard payment utilities or every reentrant function.

- `LiquidityLib.addLiquidity(...)`
  - **What is controllable?** The selected pool's linked library, immutable token/scaling configuration, current bins, and configured extensions determine requirements. The caller's owner/salt/bins/shares and extension bytes are forwarded; the adder is the pool's sender.
  - **If the return value is controllable, how is it used and how can it go wrong?** `LiquidityLib.addLiquidity` is a linked-library DELEGATECALL in pool storage context, preserving the adder as `msg.sender`. The `_loadContext` makes STATICCALLs to the pool's `getImmutables()` and `extsload`; per-bin `_positionBinKey`, ceiling `Math.mulDiv`, SafeCast bounds, and `_deltasScaledToExternal` / `Math.ceilDiv` calculate required native amounts while updating pool shares/balances. Invalid bins, oversized shares/balances, or shares below the pool minimum revert; zero shares are skipped. When either amount is positive, token `balanceOf` STATICCALLs surround `IMetricOmmModifyLiquidityCallback(msg.sender).metricOmmModifyLiquidityCallback(amount0Added, amount1Added, callbackData)` and require the stated increases. No callback occurs when both amounts are zero. Return amounts describe computed requirements and rely on honest token balance replies.
  - **What happens if it reverts, reenters, or does other unusual control flow?** The pool's transient `nonReentrant(PoolActions.ADD_LIQUIDITY)` guard rejects overlapping guarded actions on that pool. Extension hooks use ordinary CALLs through `CallExtension.callExtension(extension, data)`: success needs at least 32 return bytes and the matching selector; nonempty revert bytes bubble, empty failure becomes `ExtensionCallFailed`, and invalid acknowledgment becomes `InvalidExtensionResponse`. A probe callback's deliberate revert unwinds library writes and before-hook effects; its after-hook is not reached. A successful paying callback clears adder context before the after-hook runs,

permitting reentry to other adder operations subject to their checks; the still-active pool guard remains local to that pool. Any later uncaught failure rolls back the whole pool invocation.

- `abi.decode(callbackData, (uint8))`
  - **What is controllable?** The callback caller supplies kind and requested amounts — `KIND_PROBE` is processed without caller authentication; `KIND_PAY` requires a nonzero stored pool, `msg.sender == expectedPool`, and each amount within the stored cap. Payer/caps are loaded from transient storage.
  - **If the return value is controllable, how is it used and how can it go wrong?** The probe path returns no value and deliberately emits `LiquidityProbe(amount0Delta, amount1Delta)` as revert bytes before any pool/token call. For payment, `getImmutables()` is a STATICCALL; decoded `imm.token0` and `imm.token1` select token targets, with other fields ignored. A nonconforming accepted pool could return arbitrary tokens to pull from the stored payer. Cap checks constrain each callback's numeric amounts, not the economic identity or value of the returned tokens.
  - **What happens if it reverts, reenters, or does other unusual control flow?** Malformed kind data, unknown kind, inactive/mismatched context, or cap excess reverts before payments; `getImmutables` revert/invalid ABI propagates, and static context forbids state-changing reentry. The callback does not requery factory membership; it relies on the pool stored after entry validation.
- `pay(token0, payer, msg.sender, amount0Delta)`
  - **What is controllable?** The authenticated expected pool sets amounts within the loaded caps and supplies token addresses through its immutable-data response. Only positive legs are paid, in token0-then-token1 order; the recipient is that pool.
  - **If the return value is controllable, how is it used and how can it go wrong?** These helpers return no values; `_clearPayContext` clears pool, payer, and both caps after both payments. It also executes for a zero/zero paying callback. Caps are checked per invocation and are neither decremented nor reserved before token interaction.
  - **What happens if it reverts, reenters, or does other unusual control flow?** Payment CALLs can reenter while context is still active. A direct callback from an unrelated token fails sender authentication; a nonconforming expected pool able to reenter its own callback during a payment could repeat requests before clearing. A second callback after a completed callback is rejected as inactive. If token1 fails after token0 succeeded, the callback revert rolls back token0 payment and any clearing. When the caller propagates the error, the surrounding liquidity operation also rolls back.
- `pay(token, payer, recipient, value)`
  - **What is controllable?** The entry's caller is the stored payer. The callback controls capped amounts and pool-derived token targets. Non-WETH payment calls `IERC20(token).safeTransferFrom(payer, recipient,`

`value)`. WETH uses the adder's live ETH balance, wrapping the full amount or `nativeBalance`, transferring it to the pool, and pulling the uncovered remainder from the payer. This balance includes unrelated ETH leftovers.

- **If the return value is controllable, how is it used and how can it go wrong?** Deposit forwards the wrapped ETH and has no return to inspect. SafeERC20's zero-ETH token CALL accepts a successful checked word `1`, or successful empty return data from a code-bearing address, without a full ABI length check; nonaccepted responses cause `SafeERC20FailedOperation`. Neither wrapping success nor token success proves the pool received the expected asset; the conforming pool separately checks token balance increases. The helper's contract-funded `safeTransfer` branch applies if `payer == address(this)`, but the ordinary add entries store their external caller as payer.
- **What happens if it reverts, reenters, or does other unusual control flow?** WETH and token code can reenter while the paying context is active; no guard exists in `pay`. Failed token CALLs bubble revert bytes and deposit failures propagate. A later failed payment or pool check unwinds earlier wrapping and transfers in the same invocation.

Function: `function addLiquidityWeighted(address pool, address owner, uint80 salt, LiquidityDelta calldata weightDeltas, uint256 maxAmountToken0, uint256 maxAmountToken1, BinPositionBounds calldata binPositionBounds, bytes calldata extensionData) external payable override returns (uint256 amount0Added, uint256 amount1Added)`

This function uses positive weights as provisional shares in a reverting probe, derives required token amounts from `LiquidityProbe`, scales shares to the caps, and executes a paying addition for `owner`.

Inputs

- `pool`
  - **Control**: Arbitrary.
  - **Constraints**: Must be factory-registered.
  - **Impact**: Identifies the target pool.
- `owner`
  - **Control**: Arbitrary.
  - **Constraints**: Must be valid under the position validation.
  - **Impact**: Identifies the position owner.
- `salt`
  - **Control**: Arbitrary.

    - **Constraints**: N/A.
    - **Impact**: Identifies the position.
- `weightDeltas`
    - **Control**: Arbitrary.
    - **Constraints**: Bin and weight arrays must be nonempty and have matching lengths; every weight must be positive.
    - **Impact**: Supplies provisional per-bin share weights.
- `maxAmountToken0`
    - **Control**: Arbitrary.
    - **Constraints**: Callback token0 payment must not exceed this value.
    - **Impact**: Caps token0 settlement.
- `maxAmountToken1`
    - **Control**: Arbitrary.
    - **Constraints**: Callback token1 payment must not exceed this value.
    - **Impact**: Caps token1 settlement.
- `binPositionBounds`
    - **Control**: Arbitrary.
    - **Constraints**: The current bin and position must satisfy these bounds.
    - **Impact**: Limits acceptable pool state.
- `extensionData`
    - **Control**: Arbitrary.
    - **Constraints**: N/A.
    - **Impact**: Is forwarded to the pool.
- `msg.value`
    - **Control**: Arbitrary.
    - **Constraints**: N/A.
    - **Impact**: Adds ETH that may be wrapped for WETH settlement; payment can also use the contract's existing ETH balance.
- `msg.sender`
    - **Control**: Arbitrary.
    - **Constraints**: N/A.
    - **Impact**: Becomes the callback payer while `owner` selects the position owner.

Branches and code coverage

Intended branches

- The probe intentionally reverts with `LiquidityProbe(need0, need1)`, leaving no probe

state change; the decoded needs scale weights by the smaller token-cap ratio.

- ☐ Test coverage

- A zero required leg is unconstrained; a positive scaled share must not round to zero.
  - ☐ Test coverage
- Executes the scaled addition with `msg.sender` as payer and clears settlement context after callback payment.
  - ☐ Test coverage

Negative branches

- Invalid pool, owner, arrays, zero weights, or out-of-bounds bin state causes validation to revert.
  - ☐ Test coverage
- A normally returning probe, short or malformed revert data, or a non-`LiquidityProbe` error causes the weighted flow to revert or bubble the original error.
  - ☐ Test coverage
- The `SharesRoundedToZero`, arithmetic overflow, pool share-limit errors, callback cap failures, and failed token payment cause the operation to revert.
  - ☐ Test coverage

Function call analysis

- `_requireFactoryPool(pool)`
  - **What is controllable?** The caller selects the pool, owner, bin/share arrays (provisional weights). The immutable configured factory supplies registry membership.
  - **If the return value is controllable, how is it used and how can it go wrong?** `FACTORY.isPool(pool)` is a STATICCALL decoded as `bool`, with false rejected as `InvalidPool`; the pinned factory returns `poolToIdx[pool] != 0`. The local validators return no value and reject zero owner, require nonempty equal-length arrays and each weight to be nonzero. They do not calculate token requirements or establish that arbitrary registered code is trustworthy.
  - **What happens if it reverts, reenters, or does other unusual control flow?** Factory revert or malformed response propagates; its static context prohibits state-changing reentry. Pure validation errors revert before context setup or asset calls. Pool-specific bin, share-size, and minimum-liquidity checks execute later in the pool.
- `_validateBinAndBinPosition(pool, binPositionBounds)`
  - **What is controllable?** The caller supplies pool and inclusive bin/position bounds. The selected pool supplies the packed slot-0 word; interpretation

depends on the pinned storage layout.

- **If the return value is controllable, how is it used and how can it go wrong?** The external read is a STATICCALL decoded as `bytes32`, then `_decodeSlot0` uses `SlotOLibrary.unpack(uint256(data))`. The adder uses only current bin and position: reversed bin bounds or a value outside the bin interval/edge-position limits causes `BinPositionOutOfBounds`. A fabricated word from a nonconforming registered pool could pass the precondition without reflecting its execution state; no post-add bound check is made.
- **What happens if it reverts, reenters, or does other unusual control flow?** Malformed return or extsload revert propagates. The static read cannot mutate through reentry. The check occurs before pool extension CALLs, so it is a snapshot of the precall state rather than a lock on all subsequent external behavior.

- `IMetricOmmPoolActions(pool).addLiquidity(owner, salt, weightDeltas, abi.encode(KIND_PROBE), extensionData)`
  - **What is controllable?** The caller supplies provisional weights, pool, position identity, and extension data. The pool and its descendants control revert bytes. No paying context is established for this initial zero-ETH CALL; a conforming pool forwards the probe kind to the adder callback.
  - **If the return value is controllable, how is it used and how can it go wrong?** Decoded success values are ignored and the success block reverts `WeightedProbeInconclusive()`; that local revert is not caught by the adjacent catch. On external failure, a reason shorter than 68 bytes becomes `UnexpectedRevertLength`. Longer reasons with another selector are bubbled unchanged. Matching `LiquidityProbe.selector` causes the next two words to be read as `need0` and `need1`; trailing bytes are accepted. These amounts drive scaling, but the error's origin is not authenticated: the pool or a before-extension could fabricate the same payload without a genuine callback.
  - **What happens if it reverts, reenters, or does other unusual control flow?** The intended callback reverts immediately, rolling back the entire probe pool frame and its nested writes/transfers; the caller catches the bytes and continues to the paying call. Invalid successful-return ABI reverts in the caller and is not caught. Errors from the decode/scale/paying operations inside the catch body also propagate; the catch does not recursively catch its own body.
- `_scaleWeightsToShares(weightDeltas, maxAmountToken0, maxAmountToken1, need0, need1)`
  - **What is controllable?** The caller sets weights and caps, while the accepted probe payload sets each need. The array length determines rounding headroom.
  - **If the return value is controllable, how is it used and how can it go wrong?** Pure full-precision `mulDiv` floors its result. Each cap above the bin count is reduced by that count; each zero need gives an unconstrained `uint256.max`

scale instead of dividing. The smaller leg scale determines every output share, preserving the bin indexes. A positive input weight rounded to zero causes `SharesRoundedToZero`; an exaggerated/understated probe need can shrink/enlarge shares or make the final payment fail. The actual paying callback still applies the original caps. A fabricated zero/zero quote selects the maximum scale; it is not rejected here.

- **What happens if it reverts, reenters, or does other unusual control flow?** No external code runs. Quotients exceeding `uint256` revert; zero denominators are avoided by the need checks and the fixed nonzero WAD. Arithmetic/scaling failures abort the outer add; the earlier probe's state is already rolled back.

- `_addLiquidity(pool, owner, salt, scaled, msg.sender, maxAmountToken0, maxAmountToken1, extensionData)`

  - **What is controllable?** The entry fixes payer to its caller and supplies owner, salt, shares, caps, and extension data; `_setPayContext` stores these settlement fields before a zero-ETH ordinary CALL from the adder to the selected registered pool.
  - **If the return value is controllable, how is it used and how can it go wrong?** `_setPayContext` uses `_tloadAddress` / `_tload` to reject a nonzero active pool (`PayContextAlreadyActive`) and `_tstoreAddress` / `_tstore` to write pool, payer, and caps. The pool's decoded `a0` / `a1` are copied directly into `amount0Added` / `amount1Added`; they are not independently checked against caps or actual shares/receipts. Caps are enforced on callback requests, not on return values. A nonconforming accepted pool could skip the callback or return fabricated amounts.
  - **What happens if it reverts, reenters, or does other unusual control flow?** On normal return, `_clearPayContext` clears all four transient slots, including when no callback was made. `catch (bytes memory reason)` clears and reverts with the pool's original bytes; it does not swallow the error. A successful CALL with malformed ABI return fails in caller-side decoding and does not enter that catch, but the resulting invocation revert still rolls back context and nested effects. Active context blocks another paying add through `_setPayContext`; it does not guard payment utilities or every reentrant function.

- `LiquidityLib.addLiquidity(...)`

  - **What is controllable?** The selected pool's linked library, immutable token/scaling configuration, current bins, and configured extensions determine requirements. The caller's owner/salt/bins/shares and extension bytes are forwarded; the adder is the pool's sender.
  - **If the return value is controllable, how is it used and how can it go wrong?** `LiquidityLib.addLiquidity` is a linked-library DELEGATECALL in pool storage context, preserving the adder as `msg.sender`. The `_loadContext` makes STATICCALLs to the pool's `getImmutables()` and `extsload`; per-bin `_positionBinKey`, ceiling `Math.mulDiv`, SafeCast bounds, and `_deltasScaledToExternal` / `Math.ceilDiv` calculate required native

amounts while updating pool shares/balances. Invalid bins, oversized shares/balances, or shares below the pool minimum revert; zero shares are skipped. When either amount is positive, token `balanceOf` STATICCALLs surround `IMetricOmmModifyLiquidityCallback(msg.sender).metricOmmModifyLiquidityCallback(amount0Added, amount1Added, callbackData)` and require the stated increases. No callback occurs when both amounts are zero. Return amounts describe computed requirements and rely on honest token balance replies.

- **What happens if it reverts, reenters, or does other unusual control flow?** The pool's transient `nonReentrant(PoolActions.ADD_LIQUIDITY)` guard rejects overlapping guarded actions on that pool. Extension hooks use ordinary CALLs through `CallExtension.callExtension(extension, data)` — success needs at least 32 return bytes and the matching selector; nonempty revert bytes bubble, empty failure becomes `ExtensionCallFailed`, and invalid acknowledgment becomes `InvalidExtensionResponse`. A probe callback's deliberate revert unwinds library writes and before-hook effects; its after-hook is not reached. A successful paying callback clears adder context before the after-hook runs, permitting reentry to other adder operations subject to their checks; the still-active pool guard remains local to that pool. Any later uncaught failure rolls back the whole pool invocation.

- `abi.decode(callbackData, (uint8))`
  - **What is controllable?** The callback caller supplies kind and requested amounts — `KIND_PROBE` is processed without caller authentication; `KIND_PAY` requires a nonzero stored pool, `msg.sender == expectedPool`, and each amount within the stored cap. Payer/caps are loaded from transient storage.
  - **If the return value is controllable, how is it used and how can it go wrong?** The probe path returns no value and deliberately emits `LiquidityProbe(amount0Delta, amount1Delta)` as revert bytes before any pool/token call. For payment, `getImmutables()` is a STATICCALL; decoded `imm.token0` and `imm.token1` select token targets, with other fields ignored. A nonconforming accepted pool could return arbitrary tokens to pull from the stored payer. Cap checks constrain each callback's numeric amounts, not the economic identity or value of the returned tokens.
  - **What happens if it reverts, reenters, or does other unusual control flow?** Malformed kind data, unknown kind, inactive/mismatched context, or cap excess reverts before payments; `getImmutables` revert/invalid ABI propagates, and static context forbids state-changing reentry. The callback does not requery factory membership; it relies on the pool stored after entry validation.
- `pay(token0, payer, msg.sender, amount0Delta)`
  - **What is controllable?** The authenticated expected pool sets amounts within the loaded caps and supplies token addresses through its immutable-data response. Only positive legs are paid, in token0-then-token1 order; the recipient is that pool.

- **If the return value is controllable, how is it used and how can it go wrong?** These helpers return no values. The `_clearPayContext` clears pool, payer, and both caps after both payments. It also executes for a zero/zero paying callback. Caps are checked per invocation and are neither decremented nor reserved before token interaction.
- **What happens if it reverts, reenters, or does other unusual control flow?** Payment CALLs can reenter while context is still active. A direct callback from an unrelated token fails sender authentication; a nonconforming expected pool able to reenter its own callback during a payment could repeat requests before clearing. A second callback after a completed callback is rejected as inactive. If token1 fails after token0 succeeded, the callback revert rolls back token0 payment and any clearing. When the caller propagates the error, the surrounding liquidity operation also rolls back.

- `pay(token, payer, recipient, value)`
  - **What is controllable?** The entry's caller is the stored payer. The callback controls capped amounts and pool-derived token targets. Non-WETH payment calls `IERC20(token).safeTransferFrom(payer, recipient, value)`. WETH uses the adder's live ETH balance, wrapping the full amount or `nativeBalance`, transferring it to the pool, and pulling the uncovered remainder from the payer. This balance includes unrelated ETH leftovers.
  - **If the return value is controllable, how is it used and how can it go wrong?** Deposit forwards the wrapped ETH and has no return to inspect. SafeERC20's zero-ETH token CALL accepts a successful checked word `1`, or successful empty return data from a code-bearing address, without a full ABI length check; nonaccepted responses cause `SafeERC20FailedOperation`. Neither wrapping success nor token success proves the pool received the expected asset; the conforming pool separately checks token balance increases. The helper's contract-funded `safeTransfer` branch applies if `payer == address(this)`, but the ordinary add entries store their external caller as payer.
  - **What happens if it reverts, reenters, or does other unusual control flow?** WETH and token code can reenter while the paying context is active; no guard exists in `pay`. Failed token CALLs bubble revert bytes and deposit failures propagate. A later failed payment or pool check unwinds earlier wrapping and transfers in the same invocation.

Function: `function addLiquidityWeighted(address pool, uint80 salt, LiquidityDelta calldata weightDeltas, uint256 maxAmountToken0, uint256 maxAmountToken1, BinPositionBounds calldata binPositionBounds, bytes calldata extensionData) external payable override returns (uint256 amount0Added, uint256 amount1Added)`

This function performs weighted liquidity addition with `msg.sender` as both owner and payer. It probes, derives required amounts, scales the weights, and makes the paying call.

Inputs

- `pool`
  - **Control**: Arbitrary.
  - **Constraints**: Must be factory-registered.
  - **Impact**: Identifies the target pool.
- `salt`
  - **Control**: Arbitrary.
  - **Constraints**: N/A.
  - **Impact**: Identifies the caller-owned position.
- `weightDeltas`
  - **Control**: Arbitrary.
  - **Constraints**: Bin and weight arrays must be nonempty and have matching lengths; every weight must be positive.
  - **Impact**: Supplies provisional per-bin share weights.
- `maxAmountToken0`
  - **Control**: Arbitrary.
  - **Constraints**: Callback token0 payment must not exceed this value.
  - **Impact**: Caps token0 settlement.
- `maxAmountToken1`
  - **Control**: Arbitrary.
  - **Constraints**: Callback token1 payment must not exceed this value.
  - **Impact**: Caps token1 settlement.
- `binPositionBounds`
  - **Control**: Arbitrary.
  - **Constraints**: The current bin and position must satisfy these bounds.
  - **Impact**: Limits acceptable pool state.
- `extensionData`
  - **Control**: Arbitrary.
  - **Constraints**: N/A.
  - **Impact**: Is forwarded to the pool.
- `msg.value`
  - **Control**: Arbitrary.
  - **Constraints**: N/A.
  - **Impact**: Adds ETH that may be wrapped for WETH settlement; payment can also use the contract's existing ETH balance.
- `msg.sender`

- **Control**: Arbitrary.
- **Constraints**: N/A.
- **Impact**: Becomes both the position owner and the callback payer.

Branches and code coverage

Intended branches

- Decodes the intentional probe error, computes the smaller cap ratio, rounds scaled shares down, and executes the paying addition.
  - ☐ Test coverage
- Treats a zero token need as unconstrained when selecting the scale factor.
  - ☐ Test coverage

Negative branches

- A positive weight that scales down to zero reverts with `SharesRoundedToZero`.
  - ☐ Test coverage
- Invalid pool, delta shape, zero weight, invalid bounds, malformed probe data, arithmetic overflow, or pool share-limit errors causes the weighted flow to revert.
  - ☐ Test coverage
- Payment above a cap, an invalid callback context, or a failed transfer causes the addition to revert.
  - ☐ Test coverage

Function call analysis

- `_requireFactoryPool(pool)`
  - **What is controllable?** The caller selects the pool and bin/share arrays (provisional weights). The immutable configured factory supplies registry membership.
  - **If the return value is controllable, how is it used and how can it go wrong?** `FACTORY.isPool(pool)` is a STATICCALL decoded as `bool`, with false rejected as `InvalidPool`; the pinned factory returns `poolToIdx[pool] != 0`. The local validators return no value and require nonempty equal-length arrays and each weight to be nonzero. They do not calculate token requirements or establish that arbitrary registered code is trustworthy.
  - **What happens if it reverts, reenters, or does other unusual control flow?** Factory revert or malformed response propagates; its static context prohibits state-changing reentry. Pure validation errors revert before context setup or asset calls. Pool-specific bin, share-size, and

minimum-liquidity checks execute later in the pool.

- `_validateBinAndBinPosition(pool, binPositionBounds)`
    - **What is controllable?** The caller supplies pool and inclusive bin/position bounds. The selected pool supplies the packed slot-0 word; interpretation depends on the pinned storage layout.
    - **If the return value is controllable, how is it used and how can it go wrong?** The external read is a STATICCALL decoded as `bytes32`, and then `_decodeSlot0` uses `Slot0Library.unpack(uint256(data))`. The adder uses only current bin and position: reversed bin bounds or a value outside the bin interval/edge-position limits causes `BinPositionOutOfBounds`. A fabricated word from a nonconforming registered pool could pass the precondition without reflecting its execution state; no post-add bound check is made.
    - **What happens if it reverts, reenters, or does other unusual control flow?** Malformed return or extsload revert propagates. The static read cannot mutate through reentry. The check occurs before pool extension CALLs, so it is a snapshot of the precall state rather than a lock on all subsequent external behavior.
- `IMetricOmmPoolActions(pool).addLiquidity(msg.sender, salt, weightDeltas, abi.encode(KIND_PROBE), extensionData)`
    - **What is controllable?** The caller supplies provisional weights, pool, position identity, and extension data. The pool and its descendants control revert bytes. No paying context is established for this initial zero-ETH CALL; a conforming pool forwards the probe kind to the adder callback.
    - **If the return value is controllable, how is it used and how can it go wrong?** Decoded success values are ignored and the success block reverts `WeightedProbeInconclusive()`; that local revert is not caught by the adjacent catch. On external failure, a reason shorter than 68 bytes becomes `UnexpectedRevertLength`. Longer reasons with another selector are bubbled unchanged. Matching `LiquidityProbe.selector` causes the next two words to be read as `need0` and `need1`; trailing bytes are accepted. These amounts drive scaling, but the error's origin is not authenticated: the pool or a before-extension could fabricate the same payload without a genuine callback.
    - **What happens if it reverts, reenters, or does other unusual control flow?** The intended callback reverts immediately, rolling back the entire probe pool frame and its nested writes/transfers; the caller catches the bytes and continues to the paying call. Invalid successful-return ABI reverts in the caller and is not caught. Errors from the decode/scale/paying operations inside the catch body also propagate; the catch does not recursively catch its own body.
- `_scaleWeightsToShares(weightDeltas, maxAmountToken0, maxAmountToken1, need0, need1)`
    - **What is controllable?** The caller sets weights and caps, while the accepted

probe payload sets each need. The array length determines rounding headroom.

- **If the return value is controllable, how is it used and how can it go wrong?** Pure full-precision `mulDiv` floors its result. Each cap above the bin count is reduced by that count; each zero need gives an unconstrained `uint256.max` scale instead of dividing. The smaller leg scale determines every output share, preserving the bin indexes. A positive input weight rounded to zero causes `SharesRoundedToZero`; an exaggerated/understated probe need can shrink/enlarge shares or make the final payment fail. The actual paying callback still applies the original caps. A fabricated zero/zero quote selects the maximum scale; it is not rejected here.
- **What happens if it reverts, reenters, or does other unusual control flow?** No external code runs. Quotients exceeding `uint256` revert; zero denominators are avoided by the need checks and the fixed nonzero WAD. Arithmetic/scaling failures abort the outer add; the earlier probe's state is already rolled back.

- `_addLiquidity(pool, msg.sender, salt, scaled, msg.sender, maxAmountToken0, maxAmountToken1, extensionData)`
  - **What is controllable?** The entry fixes payer to its caller and supplies owner, salt, shares, caps, and extension data, and `_setPayContext` stores these settlement fields before a zero-ETH ordinary CALL from the adder to the selected registered pool.
  - **If the return value is controllable, how is it used and how can it go wrong?** `_setPayContext` uses `_tloadAddress` / `_tload` to reject a nonzero active pool (`PayContextAlreadyActive`) and `_tstoreAddress` / `_tstore` to write pool, payer, and caps. The pool's decoded `a0` / `a1` are copied directly into `amount0Added` / `amount1Added`; they are not independently checked against caps or actual shares/receipts. Caps are enforced on callback requests, not on return values. A nonconforming accepted pool could skip the callback or return fabricated amounts.
  - **What happens if it reverts, reenters, or does other unusual control flow?** On normal return, `_clearPayContext` clears all four transient slots, including when no callback was made. The `catch (bytes memory reason)` clears and reverts with the pool's original bytes; it does not swallow the error. A successful CALL with malformed ABI return fails in caller-side decoding and does not enter that catch, but the resulting invocation revert still rolls back context and nested effects. Active context blocks another paying add through `_setPayContext`; it does not guard payment utilities or every reentrant function.
- `LiquidityLib.addLiquidity(...)`
  - **What is controllable?** The selected pool's linked library, immutable token/scaling configuration, current bins, and configured extensions determine requirements. The caller's owner/salt/bins/shares and extension bytes are forwarded; the adder is the pool's sender.

- **If the return value is controllable, how is it used and how can it go wrong?** `LiquidityLib.addLiquidity` is a linked-library DELEGATECALL in pool storage context, preserving the adder as `msg.sender`. The `_loadContext` makes STATICCALLs to the pool's `getImmutables()` and `extsload`; per-bin `_positionBinKey`, ceiling `Math.mulDiv`, SafeCast bounds, and `_deltasScaledToExternal` / `Math.ceilDiv` calculate required native amounts while updating pool shares/balances. Invalid bins, oversized shares/balances, or shares below the pool minimum revert; zero shares are skipped. When either amount is positive, token `balanceOf` STATICCALLs surround `IMetricOmmModifyLiquidityCallback(msg.sender).metricOmmModifyLiquidityCallback(amount0Added, amount1Added, callbackData)` and require the stated increases. No callback occurs when both amounts are zero. Return amounts describe computed requirements and rely on honest token balance replies.
- **What happens if it reverts, reenters, or does other unusual control flow?** The pool's transient `nonReentrant(PoolActions.ADD_LIQUIDITY)` guard rejects overlapping guarded actions on that pool. Extension hooks use ordinary CALLs through `CallExtension.callExtension(extension, data)` — success needs at least 32 return bytes and the matching selector; nonempty revert bytes bubble, empty failure becomes `ExtensionCallFailed`, and invalid acknowledgment becomes `InvalidExtensionResponse`. A probe callback's deliberate revert unwinds library writes and before-hook effects; its after-hook is not reached. A successful paying callback clears adder context before the after-hook runs, permitting reentry to other adder operations subject to their checks; the still-active pool guard remains local to that pool. Any later uncaught failure rolls back the whole pool invocation.

- `abi.decode(callbackData, (uint8))`
  - **What is controllable?** The callback caller supplies kind and requested amounts — `KIND_PROBE` is processed without caller authentication; `KIND_PAY` requires a nonzero stored pool, `msg.sender == expectedPool`, and each amount within the stored cap. Payer/caps are loaded from transient storage.
  - **If the return value is controllable, how is it used and how can it go wrong?** The probe path returns no value and deliberately emits `LiquidityProbe(amount0Delta, amount1Delta)` as revert bytes before any pool/token call. For payment, `getImmutables()` is a STATICCALL; decoded `imm.token0` and `imm.token1` select token targets, with other fields ignored. A nonconforming accepted pool could return arbitrary tokens to pull from the stored payer. Cap checks constrain each callback's numeric amounts, not the economic identity or value of the returned tokens.
  - **What happens if it reverts, reenters, or does other unusual control flow?** Malformed kind data, unknown kind, inactive/mismatched context, or cap excess reverts before payments; `getImmutables` revert/invalid ABI propagates, and static context forbids state-changing reentry. The callback does not requery factory membership; it relies on the pool stored after entry validation.

- `pay(token0, payer, msg.sender, amount0Delta)`
    - **What is controllable?** The authenticated expected pool sets amounts within the loaded caps and supplies token addresses through its immutable-data response. Only positive legs are paid, in token0-then-token1 order; the recipient is that pool.
    - **If the return value is controllable, how is it used and how can it go wrong?** These helpers return no values; `_clearPayContext` clears pool, payer, and both caps after both payments. It also executes for a zero/zero paying callback. Caps are checked per invocation and are neither decremented nor reserved before token interaction.
    - **What happens if it reverts, reenters, or does other unusual control flow?** Payment CALLs can reenter while context is still active. A direct callback from an unrelated token fails sender authentication; a nonconforming expected pool able to reenter its own callback during a payment could repeat requests before clearing. A second callback after a completed callback is rejected as inactive. If token1 fails after token0 succeeded, the callback revert rolls back token0 payment and any clearing. When the caller propagates the error, the surrounding liquidity operation also rolls back.
- `pay(token, payer, recipient, value)`
    - **What is controllable?** The entry's caller is the stored payer. The callback controls capped amounts and pool-derived token targets. Non-WETH payment calls `IERC20(token).safeTransferFrom(payer, recipient, value)`. WETH uses the adder's live ETH balance, wrapping the full amount or `nativeBalance`, transferring it to the pool, and pulling the uncovered remainder from the payer. This balance includes unrelated ETH leftovers.
    - **If the return value is controllable, how is it used and how can it go wrong?** Deposit forwards the wrapped ETH and has no return to inspect. SafeERC20's zero-ETH token CALL accepts a successful checked word `1`, or successful empty return data from a code-bearing address, without a full ABI length check; nonaccepted responses cause `SafeERC20FailedOperation`. Neither wrapping success nor token success proves the pool received the expected asset; the conforming pool separately checks token balance increases. The helper's contract-funded `safeTransfer` branch applies if `payer == address(this)`, but the ordinary add entries store their external caller as payer.
    - **What happens if it reverts, reenters, or does other unusual control flow?** WETH and token code can reenter while the paying context is active; no guard exists in `pay`. Failed token CALLs bubble revert bytes and deposit failures propagate. A later failed payment or pool check unwinds earlier wrapping and transfers in the same invocation.

Function: `function metricOmmModifyLiquidityCallback(uint256 amount0Delta, uint256 amount1Delta, bytes calldata callbackData) external override`

This function returns token requirements through the intentional probe error or settles a paying liquidity callback with the active transient context.

Inputs

- `amount0Delta`
  - **Control**: Callback caller.
  - **Constraints**: Must not exceed the stored token0 cap in payment mode.
  - **Impact**: Specifies token0 payment or probe quote.
- `amount1Delta`
  - **Control**: Callback caller.
  - **Constraints**: Must not exceed the stored token1 cap in payment mode.
  - **Impact**: Specifies token1 payment or probe quote.
- `callbackData`
  - **Control**: Callback caller.
  - **Constraints**: Must decode to `KIND_PROBE` or `KIND_PAY`.
  - **Impact**: Selects probe or payment mode.
- `msg.sender`
  - **Control**: Callback caller.
  - **Constraints**: Must equal the active expected pool in payment mode.
  - **Impact**: Identifies the pool receiving settlement.

Branches and code coverage

Intended branches

- The `KIND_PAY` reads the stored pool, payer, and caps; it reads pool immutables, pays positive deltas in token order, and then clears context.
  - ☐ Test coverage

Negative branches

- The `KIND_PROBE` always reverts with `LiquidityProbe(amount0Delta, amount1Delta)` before caller authentication.
  - ☐ Test coverage
- Invalid decoding or callback kind, inactive context, invalid caller, cap excess, and failed

pool or token calls cause the callback to revert.

☑ Test coverage

Function call analysis

- `abi.decode(callbackData, (uint8))`
  - **What is controllable?** The callback caller supplies kind and requested amounts. The `KIND_PROBE` is processed without caller authentication; `KIND_PAY` requires a nonzero stored pool, `msg.sender == expectedPool`, and each amount within the stored cap. Payer/caps are loaded from transient storage.
  - **If the return value is controllable, how is it used and how can it go wrong?** The probe path returns no value and deliberately emits `LiquidityProbe(amount0Delta, amount1Delta)` as revert bytes before any pool/token call. For payment, `getImmutables()` is a STATICCALL; decoded `imm.token0` and `imm.token1` select token targets, with other fields ignored. A nonconforming accepted pool could return arbitrary tokens to pull from the stored payer. Cap checks constrain each callback's numeric amounts, not the economic identity or value of the returned tokens.
  - **What happens if it reverts, reenters, or does other unusual control flow?** Malformed kind data, unknown kind, inactive/mismatched context, or cap excess reverts before payments; `getImmutables` revert/invalid ABI propagates, and static context forbids state-changing reentry. The callback does not requery factory membership; it relies on the pool stored after entry validation.
- `pay(token0, payer, msg.sender, amount0Delta)`
  - **What is controllable?** The authenticated expected pool sets amounts within the loaded caps and supplies token addresses through its immutable-data response. Only positive legs are paid, in token0-then-token1 order; the recipient is that pool.
  - **If the return value is controllable, how is it used and how can it go wrong?** These helpers return no values; `_clearPayContext` clears pool, payer, and both caps after both payments. It also executes for a zero/zero paying callback. Caps are checked per invocation and are neither decremented nor reserved before token interaction.
  - **What happens if it reverts, reenters, or does other unusual control flow?** Payment CALLs can reenter while context is still active. A direct callback from an unrelated token fails sender authentication; a nonconforming expected pool able to reenter its own callback during a payment could repeat requests before clearing. A second callback after a completed callback is rejected as inactive. If token1 fails after token0 succeeded, the callback revert rolls back token0 payment and any clearing. When the caller propagates the error, the surrounding liquidity operation also rolls back.

- `pay(token, payer, recipient, value)`
  - **What is controllable?** The entry's caller is the stored payer. The callback controls capped amounts and pool-derived token targets. Non-WETH payment calls `IERC20(token).safeTransferFrom(payer, recipient, value)`. WETH uses the adder's live ETH balance, wrapping the full amount or `nativeBalance`, transferring it to the pool, and pulling the uncovered remainder from the payer. This balance includes unrelated ETH leftovers.
  - **If the return value is controllable, how is it used and how can it go wrong?** Deposit forwards the wrapped ETH and has no return to inspect. SafeERC20's zero-ETH token CALL accepts a successful checked word `1`, or successful empty return data from a code-bearing address, without a full ABI length check; nonaccepted responses cause `SafeERC20FailedOperation`. Neither wrapping success nor token success proves the pool received the expected asset; the conforming pool separately checks token balance increases. The helper's contract-funded `safeTransfer` branch applies if `payer == address(this)`, but the ordinary add entries store their external caller as payer.
  - **What happens if it reverts, reenters, or does other unusual control flow?** WETH and token code can reenter while the paying context is active; no guard exists in `pay`. Failed token CALLs bubble revert bytes and deposit failures propagate. A later failed payment or pool check unwinds earlier wrapping and transfers in the same invocation.

Function: `function multicall(bytes[] calldata data) public payable override returns (bytes[] memory results)`

This function delegatecalls this contract for every supplied payload in order. Calls share the original sender, ETH value, and storage.

Inputs

- `data`
  - **Control**: Arbitrary.
  - **Constraints**: N/A.
  - **Impact**: Defines execution order and raw results.
- `msg.value`
  - **Control**: Arbitrary.
  - **Constraints**: N/A.
  - **Impact**: Provides the shared ETH balance.
- `msg.sender`
  - **Control**: Arbitrary.

- **Constraints**: N/A.
- **Impact**: Is preserved for nested delegatecalls.

Branches and code coverage

Intended branches

- Executes sequential delegatecalls and returns each raw result, including empty return data.

  ☑ Test coverage

Negative branches

- A nested failure reverts the full batch; nonempty revert data is bubbled while empty data becomes `FailedCall()`.

  ☐ Test coverage

Function call analysis

- `Address.functionDelegateCall(address(this), data[i])`
  - **What is controllable?** The caller controls every payload, selector, argument, and ordering. The target is fixed to this contract, so these are self-DELEGATECALLs rather than delegatecalls to caller-selected foreign code. Original `msg.sender`, `msg.value`, contract balance, persistent storage, and transient storage are shared with every subcall.
  - **If the return value is controllable, how is it used and how can it go wrong?** The `LowLevelCall` returns only its success boolean while leaving return data available. The `Address` returns copied raw bytes when successful with nonempty data or a code-bearing target; each byte string becomes `results[i]` without semantic decoding, and is not passed into later entries automatically. The self target has code in this deployed entry, so the library's empty-code success rejection is not an ordinary reachable branch. A successful subcall returning a false-like value is still a successful multicall element.
  - **What happens if it reverts, reenters, or does other unusual control flow?** Nonempty failure bytes are bubbled by `LowLevelCall.bubbleRevert`; an empty revert becomes `Errors.FailedCall()`. There is no per-element catch, so any uncaught element failure unwinds previous elements' writes, transfers, and transient context in this invocation. External callbacks reached by a selected entry can reenter multicall; the loop itself adds no guard. Recursion and large byte arrays can exhaust gas, with the same rollback.
- Selector dispatch

- **What is controllable?** Caller-selected selectors can reach the four liquidity entries (exact shares with owner ↗, exact shares for caller ↗, weighted with owner ↗, weighted for caller ↗), liquidity callbacks ↗, nested multicalls, and inherited payment helpers ↗. Each route is subject to that function's validation and call analysis; arbitrary selectors do not bypass dispatcher checks.
- **If the return value is controllable, how is it used and how can it go wrong?** Each selected entry controls its own internal/library/external calls and output. Payment entries can transfer the entire live contract asset balance; selectors are not limited to operations on balances contributed by this batch caller. All subcalls see the same original `msg.value`, but DELEGATECALL does not transfer that ETH again or replenish the live balance. Nonpayable selectors reject a nonzero inherited `msg.value`.
- **What happens if it reverts, reenters, or does other unusual control flow?** A paying liquidity add requires an empty active pay context; paying callbacks require its expected pool and caps. A probe callback deliberately reverts before authentication. Those checks are not a global lock on payment functions or on calls after context clearing. Empty payloads dispatch to inherited `receive()`, whose WETH-sender check still applies. Internal errors absorbed by a selected entry, such as a valid weighted-liquidity probe revert, do not themselves fail the multicall; errors propagated out of an entry do.

5.5. MetricOmmSimpleRouter.sol

Function: `function exactInput(ExactInputParams calldata params) external payable returns (uint256 amountOut)`

This function executes an exact-input route from the first pool to the last. The caller pays the first hop; the router pays later hops from intermediate proceeds. Each hop must consume at least the requested input, and the final output must meet the requested minimum.

Inputs

- `params.tokens`
  - **Control**: Arbitrary.
  - **Constraints**: Must contain at least two entries and have length `pools.length + 1`.
  - **Impact**: Selects each hop's payment token.
- `params.pools`
  - **Control**: Arbitrary.
  - **Constraints**: Must have the same length as `extensionDatas`; at most 256 entries are allowed; and each executed pool must be registered in `FACTORY`.
  - **Impact**: Identifies the pools for each route hop.

- `params.extensionDatas`
    - **Control**: Arbitrary.
    - **Constraints**: Must have the same length as `pools`.
    - **Impact**: Supplies extension data for each route hop.
- `params.zeroForOneBitMap`
    - **Control**: Arbitrary.
    - **Constraints**: N/A.
    - **Impact**: Selects the swap direction for each route hop.
- `params.amountIn`
    - **Control**: Arbitrary.
    - **Constraints**: Must fit in `int128`.
    - **Impact**: Sets the exact input amount for the first hop.
- `params.amountOutMinimum`
    - **Control**: Arbitrary.
    - **Constraints**: The computed final output must be at least this value.
    - **Impact**: Sets the final output floor.
- `params.recipient`
    - **Control**: Arbitrary.
    - **Constraints**: N/A.
    - **Impact**: Receives the final output.
- `params.deadline`
    - **Control**: Arbitrary.
    - **Constraints**: Must not be earlier than the current block timestamp.
    - **Impact**: Limits when execution may occur.
- `msg.value`
    - **Control**: Arbitrary.
    - **Constraints**: N/A.
    - **Impact**: May fund WETH settlement.
- `msg.sender`
    - **Control**: Arbitrary.
    - **Constraints**: N/A.
    - **Impact**: Becomes the first-hop callback payer.

Branches and code coverage

Intended branches

- Executes two-hop and three-hop routes and returns the final output.
  - ☑ Test coverage
- Uses the requested direction bit for each hop and routes intermediate output to the router.
  - ☐ Test coverage
- Stores the expected pool, callback mode, payer, and payment token in transient storage before the route; clears that state on normal completion.
  - ☐ Test coverage
- Pays from router inventory for later hops. For WETH, wraps available ETH first and transfers any shortfall from the payer.
  - ☐ Test coverage
- Accepts SafeERC20 calls that return `true` or empty return data from a contract address.
  - ☐ Test coverage

Negative branches

- An expired deadline, an unregistered pool, insufficient final output, malformed route arrays, or more than 256 pools causes the appropriate route validation error to revert.
  - ☑ Test coverage
- Input above `int128.max`, a hop that consumes too little input, or invalid final deltas causes the corresponding conversion or swap-delta error to revert.
  - ☐ Test coverage
- A payment transfer failure, disallowed return value, or empty return from an address without code causes the payment operation to revert.
  - ☐ Test coverage
- A callback from an unexpected or unregistered pool, or deltas without exactly one positive side, causes callback validation to revert.
  - ☐ Test coverage
- Reentering another router swap while callback context is active overwrites that context; a successful nested call clears it and does not restore the outer context.
  - ☐ Test coverage

Function call analysis

- `_checkDeadline(params.deadline)`
  - **What is controllable?** The caller controls the deadline, amount, direction/bitmap, and route arrays.

- **If the return value is controllable, how is it used and how can it go wrong?** These internal/library helpers make no external call. The deadline comparison rejects expiry; `asAmountSpecifiedIn` calls `toInt128`, rejects values above `int128.max`, and permits zero at this layer. Path validation checks at least two tokens, matching array lengths, and at most 256 pools, making the subsequent last index valid. Bitmap bits select direction and `openLimit` supplies zero or `uint128.max`. No pool-token connectivity check or token-metadata call is made. The resulting positive amount requests exact input; the pinned pool rejects zero.
- **What happens if it reverts, reenters, or does other unusual control flow?** Expiry, invalid path shape where applicable, or out-of-range amount reverts before the pool call. These helpers cannot reenter; pool-specific amount/price constraints are enforced later.

- `_setNextCallbackContext(pool, CALLBACK_MODE_JUST_PAY, i == 0 ? msg.sender : address(this), params.tokens[i])`
  - **What is controllable?** The caller selects the pool and token; the payer is the caller for the first input payment, or this contract for forward-route intermediate payments. The immutable factory decides whether the selected pool is registered. The pinned factory returns `poolToIdx[pool] != 0`.
  - **If the return value is controllable, how is it used and how can it go wrong?** `FACTORY.isPool(pool)` is a STATICCALL decoded as `bool`; false causes `InvalidPool`. The internal transient setters return nothing and store the expected callback pool, mode, payer, token. They do not validate the selected token against pool immutables, reject an already active router context, or reset the amount-in slot at entry.
  - **What happens if it reverts, reenters, or does other unusual control flow?** Factory reverts or malformed bool replies propagate; static context prevents its read from performing a state-changing reentry. Subsequent pool/token/extension CALLs can reenter router entry points and overwrite shared transient context. This registry check is callback authentication preparation, not a router-wide reentrancy lock. An uncaught outer failure rolls back the new transient writes.
- `IMetricOmmPoolActions(pool).swap(i == last ? params.recipient : address(this), zeroForOne, amount, MetricOmmSwapPath.openLimit(zeroForOne), "", params.extensionDatas[i])`
  - **What is controllable?** The caller chooses all pools, tokens, directions, extension data, and final recipient. Each hop is a zero-ETH CALL from the router. Intermediate output is sent to the router, and the next callback pays from its holdings rather than from the original caller.
  - **If the return value is controllable, how is it used and how can it go wrong?** Each pair of returned `int128` deltas is decoded. The `extractAmountIn` selects the direction's input delta and must be at least the requested `amount`; there is no equality or upper-bound check. The `extractAmountOut` negates the other delta to become the next hop's amount. After the loop, output must

be positive, safely converted with `int128ToUint128`, and reach the minimum. There is no per-hop recipient balance measurement or token/direction consistency check. Dishonest registered-pool returns can misstate amounts; unusual signs may feed a negative request into a later hop before the final positivity check.

- **What happens if it reverts, reenters, or does other unusual control flow?** The loop replaces callback context for each hop and invokes the just-payment path below. Any pool/decode failure, selected output equal to `int128.min`, insufficient returned input, or final output failure reverts the whole route. A prior successful hop's transfers and writes also roll back. Reentrant public router calls can overwrite context before the loop resumes.

- `_getBidAskReferencePriceX64()`
  - **What is controllable?** The route selects a registered pool and supplies recipient, direction, amount, limit, callback bytes, and extension data. The pool's configured provider, extensions, tokens, liquidity, and fees determine execution. These downstream components are additional trust boundaries; registry membership alone does not validate their behavior.
  - **If the return value is controllable, how is it used and how can it go wrong?** `IPriceProvider(activePriceProvider).getQuote()` is a state-changing CALL, not STATICCALL. Decoded bid/ask/reference values pass ordering, nonzero-bid, range, and configured asymmetry checks before pricing. The `_executeSwap` scales amounts; applies FeeMath/SwapMath across bins until input/output, liquidity, price, or grid limits stop progress; updates pool accounting; and converts signed deltas to native units with ceiling division. Output token `safeTransfer` uses optional-return handling; input `balanceOf` reads use STATICCALL before/after the callback and must show the required increase. Token-reported balances remain trusted. Extension CALLs must succeed with at least 32 bytes and a matching function selector; their returned bytes do not replace swap deltas.
  - **What happens if it reverts, reenters, or does other unusual control flow?** `whenNotPaused` rejects paused swaps; the pool's transient `nonReentrant(PoolActions.SWAP)` guard blocks overlapping guarded actions on that pool, while callbacks and calls to other pools remain possible. Provider reverts are wrapped as `PriceProviderFailed(reason)`; quote-validation or return-decoding failures propagate separately. Extensions run before the swap calculation and after settlement through `CallExtension.callExtension(extension, data)`; nonempty failure bytes are bubbled, empty failures become `ExtensionCallFailed`, and bad acknowledgments become `InvalidExtensionResponse`. Any uncaught pool, token, extension, arithmetic, cast, or downstream callback error reverts that swap. These pool protections do not install a router-wide reentrancy guard.

- `_requireExpectedCallbackCaller(msg.sender)`
  - **What is controllable?** The callback caller supplies deltas and data but must match `TransientCallbackPool.getPool()` and pass

`FACTORY.isPool(caller)`. The expected pool/mode/payer/token were set by a swap entry or prior recursive hop. The configured factory's registry response is trusted.

- **If the return value is controllable, how is it used and how can it go wrong?** `TransientCallbackPool.requireCaller(caller)` compares against transient state before the factory STATICCALL. A false factory result rejects the caller. The mode selects just-payment or recursive-output handling. The `extractPositiveAmount` calls `getPositiveAmount`, accepting exactly one positive and one strictly negative delta; it does not verify that the positive side matches the stored token or direction. Its positive `int256` result becomes the payment or preceding-hop output request.
- **What happens if it reverts, reenters, or does other unusual control flow?** Invalid caller, factory failure/malformed response, or invalid deltas revert. The `InvalidCallbackMode` is a defensive check: the deployed router's public paths write only modes 0 and 1, recursive updates preserve the mode, and clearing writes zero, so an unsupported mode is not reachable through these paths. The factory read cannot mutate state through reentry. A valid callback is an expected ordinary CALL from the pool; the router has no callback-wide reentrancy guard, and authentication does not consume the payment context.

- `_justPayCallback(amount0Delta, amount1Delta)`

  - **What is controllable?** An authenticated expected pool supplies both deltas. Transient getters read the token and payer established by the router; the recipient is the callback sender.
  - **If the return value is controllable, how is it used and how can it go wrong?** The helper selects the strictly positive delta and converts it to `uint256` then calls `pay`. There is no helper return, no local cap against the entry's specified amount, and no callback count or balance receipt recorded here. Payment limits therefore rely on the conforming pool's deltas and the entry's later return-value checks; an accepted nonconforming pool is not constrained to a single payment request.
  - **What happens if it reverts, reenters, or does other unusual control flow?** Invalid sign combinations revert; payment failures propagate. Context remains active across external token/WETH execution and after a successful callback until the outer entry clears it. Direct token reentry cannot impersonate the expected pool, but other public entry points are not guarded and may overwrite context.

- `pay(token, payer, recipient, value)`

  - **What is controllable?** The callback supplies the payment amount and recipient; the active context supplies payer and token. For a contract-funded intermediate hop, `payer == address(this)` always transfers ERC-20 already held here. Otherwise, WETH payment first consumes the contract's live ETH balance, wrapping all or part of the required amount, and pulls any remainder from the payer; a non-WETH payment pulls the full amount. ETH is not attributed to individual callers.

- **If the return value is controllable, how is it used and how can it go wrong?** Neither `pay` nor WETH deposit returns a checked value. Deposit is a value-bearing CALL; success is trusted to mint WETH. SafeERC20 performs zero-ETH low-level CALLs and accepts a successful checked return word of `1`, or successful empty return data from a code-bearing token; its assembly does not perform a full ABI length check. Nonaccepted results become `SafeERC20FailedOperation`, and success alone does not prove receipt. The pinned pool separately measures input balances.
- **What happens if it reverts, reenters, or does other unusual control flow?** Deposit and token CALLs can reenter public functions while callback context remains active. The payment helper has no guard. Token revert data is bubbled by SafeERC20; WETH reverts propagate. An uncaught failure unwinds payment, wrapping, pool writes, and transient writes in the failed invocation; a catching outer caller may continue from its precall state.

- `_clearExpectedCallbackPool()`
  - **What is controllable?** The success path determines when clearing happens; callees have had access to the preceding callback window.
  - **If the return value is controllable, how is it used and how can it go wrong?** There is no external return. Four transient slots are zeroed: expected pool/mode/count, recorded input, payer, and token. This prevents later calls in the transaction from reusing context after normal completion.
  - **What happens if it reverts, reenters, or does other unusual control flow?** No external code is called. A revert before reaching this cleanup still unwinds transient writes made within the reverted invocation. Nested successful router calls can themselves clear the shared context; there is no stack of saved contexts.

Function: `function exactInputSingle(ExactInputSingleParams calldata params) external payable returns (uint256 amountOut)`

This function executes one exact-input swap through a registered pool and enforces the requested output floor. The router does not independently verify that `tokenIn` matches a pool token.

Inputs

- `params.pool`
  - **Control**: Arbitrary.
  - **Constraints**: Must be registered in `FACTORY` when the callback is validated.
  - **Impact**: Identifies the pool to call.
- `params.tokenIn`
  - **Control**: Arbitrary.

    - **Constraints**: N/A.
    - **Impact**: Identifies the token paid in the callback.
- `params.zeroForOne`
    - **Control**: Arbitrary.
    - **Constraints**: N/A.
    - **Impact**: Selects the swap direction.
- `params.priceLimitX64`
    - **Control**: Arbitrary.
    - **Constraints**: N/A.
    - **Impact**: Sets the pool price limit after direction-specific open-price sentinel normalization.
- `params.amountIn`
    - **Control**: Arbitrary.
    - **Constraints**: Must fit in `int128`.
    - **Impact**: Sets the exact input amount.
- `params.amountOutMinimum`
    - **Control**: Arbitrary.
    - **Constraints**: The computed output must be at least this value.
    - **Impact**: Sets the output floor.
- `params.recipient`
    - **Control**: Arbitrary.
    - **Constraints**: N/A.
    - **Impact**: Receives the swap output.
- `params.deadline`
    - **Control**: Arbitrary.
    - **Constraints**: Must not be earlier than the current block timestamp.
    - **Impact**: Limits when execution may occur.
- `params.extensionData`
    - **Control**: Arbitrary.
    - **Constraints**: N/A.
    - **Impact**: Is forwarded to the pool.
- `msg.value`
    - **Control**: Arbitrary.
    - **Constraints**: N/A.
    - **Impact**: May fund WETH settlement.
- `msg.sender`
    - **Control**: Arbitrary.

- **Constraints**: N/A.
- **Impact**: Becomes the callback payer.

Branches and code coverage

Intended branches

- Executes and settles both `zeroForOne` directions.
  - ☑ Test coverage
- Normalizes opposite-direction open-price sentinels.
  - ☑ Test coverage
- Stores and clears transient callback context around the swap, and wraps available ETH before drawing WETH from the payer.
  - ☐ Test coverage
- Validates the callback caller and pays the sole positive delta using the stored token and payer.
  - ☐ Test coverage

Negative branches

- Expiry, an unregistered pool, insufficient output, or `amountIn > int128.max` causes the documented validation error to revert.
  - ☑ Test coverage
- Payment failures, invalid SafeERC20 return values, invalid callback deltas, or an unexpected callback caller cause the transaction to revert.
  - ☐ Test coverage
- A selected output delta of `int128.min` panics on negation; a negative output after negation fails conversion to `uint256`.
  - ☐ Test coverage

Function call analysis

- `_checkDeadline(params.deadline)`
  - **What is controllable?** The caller controls the deadline, amount, direction/bitmap, and single-pool price limit.
  - **If the return value is controllable, how is it used and how can it go wrong?** These internal/library helpers make no external call. The deadline comparison rejects expiry; `asAmountSpecifiedIn` calls `toInt128`, rejects values above `int128.max`, and permits zero at this layer. Normalization maps the opposite-direction open-limit sentinel to the canonical limit and

otherwise preserves the caller's value. The resulting positive amount requests exact input; the pinned pool rejects zero.

- **What happens if it reverts, reenters, or does other unusual control flow?** Expiry, invalid path shape where applicable, or out-of-range amount reverts before the pool call. These helpers cannot reenter; pool-specific amount/price constraints are enforced later.

- `_setNextCallbackContext(params.pool, CALLBACK_MODE_JUST_PAY, msg.sender, params.tokenIn)`

  - **What is controllable?** The caller selects the pool and token. The payer stored for this entry is `msg.sender`, and the payment token is `params.tokenIn`. The immutable factory decides whether the selected pool is registered. The pinned factory returns `poolToIdx[pool] != 0`.
  - **If the return value is controllable, how is it used and how can it go wrong?** `FACTORY.isPool(pool)` is a STATICCALL decoded as `bool`; false causes `InvalidPool`. The internal transient setters return nothing and store the expected callback pool, mode, payer, token. They do not validate the selected token against pool immutables, reject an already active router context, or reset the amount-in slot at entry.
  - **What happens if it reverts, reenters, or does other unusual control flow?** Factory reverts or malformed bool replies propagate; static context prevents its read from performing a state-changing reentry. Subsequent pool/token/extension CALLs can reenter router entry points and overwrite shared transient context. This registry check is callback authentication preparation, not a router-wide reentrancy lock. An uncaught outer failure rolls back the new transient writes.

- `IMetricOmmPoolActions(params.pool).swap(params.recipient, params.zeroForOne, MetricOmmSwapInputs.asAmountSpecifiedIn(params.amountIn), priceLimitX64, "", params.extensionData)`

  - **What is controllable?** The caller supplies the selected registered pool, recipient, direction, amount, normalized limit, and extension data. The router supplies empty callback bytes and becomes the pool's `msg.sender`; no ETH is forwarded by this CALL.
  - **If the return value is controllable, how is it used and how can it go wrong?** The two returned `int128` values are ABI-decoded; `MetricOmmSwapResults.extractAmountOut(params.zeroForOne, amount0Delta, amount1Delta)` negates the selected output delta, and `MetricOmmSwapInputs.int128ToUint128(out)` uses SafeCast to reject a negative result. The output must reach `amountOutMinimum`. The input delta is not checked against `params.amountIn`, and actual recipient balances are not measured here. A nonconforming registered pool can therefore lie about output or request payments unrelated to its returned values.
  - **What happens if it reverts, reenters, or does other unusual control flow?** A pool revert or malformed return propagates; negating `int128.min` panics

and a negative extracted output fails the checked unsigned cast. Callback execution occurs before return and follows the authentication/payment paths below. Failing the final minimum check rolls back earlier pool/token effects.

- `_getBidAskReferencePriceX64()`
  - **What is controllable?** The route selects a registered pool and supplies recipient, direction, amount, limit, callback bytes, and extension data. The pool's configured provider, extensions, tokens, liquidity, and fees determine execution. These downstream components are additional trust boundaries; registry membership alone does not validate their behavior.
  - **If the return value is controllable, how is it used and how can it go wrong?** `IPriceProvider(activePriceProvider).getQuote()` is a state-changing CALL, not STATICCALL. Decoded bid/ask/reference values pass ordering, nonzero-bid, range, and configured asymmetry checks before pricing. The `_executeSwap` scales amounts; applies FeeMath/SwapMath across bins until input/output, liquidity, price, or grid limits stop progress; updates pool accounting; and converts signed deltas to native units with ceiling division. Output token `safeTransfer` uses optional-return handling; input `balanceOf` reads use STATICCALL before/after the callback and must show the required increase. Token-reported balances remain trusted. Extension CALLs must succeed with at least 32 bytes and a matching function selector; their returned bytes do not replace swap deltas.
  - **What happens if it reverts, reenters, or does other unusual control flow?** `whenNotPaused` rejects paused swaps; the pool's transient `nonReentrant(PoolActions.SWAP)` guard blocks overlapping guarded actions on that pool, while callbacks and calls to other pools remain possible. Provider reverts are wrapped as `PriceProviderFailed(reason)`; quote-validation or return-decoding failures propagate separately. Extensions run before the swap calculation and after settlement through `CallExtension.callExtension(extension, data)`; nonempty failure bytes are bubbled, empty failures become `ExtensionCallFailed`, and bad acknowledgments become `InvalidExtensionResponse`. Any uncaught pool, token, extension, arithmetic, cast, or downstream callback error reverts that swap. These pool protections do not install a router-wide reentrancy guard.
- `_requireExpectedCallbackCaller(msg.sender)`
  - **What is controllable?** The callback caller supplies deltas and data but must match `TransientCallbackPool.getPool()` and pass `FACTORY.isPool(caller)`. The expected pool/mode/payer/token were set by a swap entry or prior recursive hop. The configured factory's registry response is trusted.
  - **If the return value is controllable, how is it used and how can it go wrong?** `TransientCallbackPool.requireCaller(caller)` compares against transient state before the factory STATICCALL. A false factory result rejects the caller. The mode selects just-payment or recursive-output handling. The

`extractPositiveAmount` calls `getPositiveAmount`, accepting exactly one positive and one strictly negative delta; it does not verify that the positive side matches the stored token or direction. Its positive `int256` result becomes the payment or preceding-hop output request.

- **What happens if it reverts, reenters, or does other unusual control flow?** Invalid caller, factory failure/malformed response, or invalid deltas revert. The `InvalidCallbackMode` is a defensive check: the deployed router's public paths write only modes 0 and 1, recursive updates preserve the mode, and clearing writes zero, so an unsupported mode is not reachable through these paths. The factory read cannot mutate state through reentry. A valid callback is an expected ordinary CALL from the pool; the router has no callback-wide reentrancy guard, and authentication does not consume the payment context.

- `_justPayCallback(amount0Delta, amount1Delta)`

  - **What is controllable?** An authenticated expected pool supplies both deltas. Transient getters read the token and payer established by the router; the recipient is the callback sender.
  - **If the return value is controllable, how is it used and how can it go wrong?** The helper selects the strictly positive delta and converts it to `uint256` then calls `pay`. There is no helper return, no local cap against the entry's specified amount, and no callback count or balance receipt recorded here. Payment limits therefore rely on the conforming pool's deltas and the entry's later return-value checks; an accepted nonconforming pool is not constrained to a single payment request.
  - **What happens if it reverts, reenters, or does other unusual control flow?** Invalid sign combinations revert; payment failures propagate. Context remains active across external token/WETH execution and after a successful callback until the outer entry clears it. Direct token reentry cannot impersonate the expected pool, but other public entry points are not guarded and may overwrite context.

- `pay(token, payer, recipient, value)`

  - **What is controllable?** The callback supplies the payment amount and recipient; the active context supplies payer and token. For a contract-funded intermediate hop, `payer == address(this)` always transfers ERC-20 already held here. Otherwise, WETH payment first consumes the contract's live ETH balance, wrapping all or part of the required amount, and pulls any remainder from the payer; a non-WETH payment pulls the full amount. ETH is not attributed to individual callers.
  - **If the return value is controllable, how is it used and how can it go wrong?** Neither `pay` nor WETH deposit returns a checked value. Deposit is a value-bearing CALL; success is trusted to mint WETH. SafeERC20 performs zero-ETH low-level CALLs and accepts a successful checked return word of `1`, or successful empty return data from a code-bearing token; its assembly does not perform a full ABI length check. Nonaccepted results become `SafeERC20FailedOperation`, and success alone does not prove receipt. The

pinned pool separately measures input balances.

- **What happens if it reverts, reenters, or does other unusual control flow?** Deposit and token CALLs can reenter public functions while callback context remains active. The payment helper has no guard. Token revert data is bubbled by SafeERC20; WETH reverts propagate. An uncaught failure unwinds payment, wrapping, pool writes, and transient writes in the failed invocation; a catching outer caller may continue from its precall state.

- `_clearExpectedCallbackPool()`

  - **What is controllable?** The success path determines when clearing happens; callees have had access to the preceding callback window.
  - **If the return value is controllable, how is it used and how can it go wrong?** There is no external return. Four transient slots are zeroed: expected pool/mode/count, recorded input, payer, and token. This prevents later calls in the transaction from reusing context after normal completion.
  - **What happens if it reverts, reenters, or does other unusual control flow?** No external code is called. A revert before reaching this cleanup still unwinds transient writes made within the reverted invocation. Nested successful router calls can themselves clear the shared context; there is no stack of saved contexts.

Function: `function exactOutput(ExactOutputParams calldata params) external payable returns (uint256 amountIn)`

This function starts an exact-output route at the last pool and recursively invokes prior pools through callbacks. The first input token is paid by the caller; the final recorded transient input amount is returned.

Inputs

- `params.tokens`

  - **Control**: Arbitrary.
  - **Constraints**: Must contain at least two entries and have length `pools.length + 1`.
  - **Impact**: The array length participates in route validation, and `tokens[0]` sets the terminal payment token. The whole array is encoded into callback data, but recursive callbacks do not read its token entries.

- `params.pools`

  - **Control**: Arbitrary.
  - **Constraints**: Must have the same length as `extensionDatas`; at most 256 entries are allowed; and each executed pool must be registered in `FACTORY`.
  - **Impact**: Identifies pools across the reverse route.

- `params.extensionDatas`
    - **Control**: Arbitrary.
    - **Constraints**: Must have the same length as `pools`.
    - **Impact**: Supplies extension data for each route hop.
- `params.zeroForOneBitMap`
    - **Control**: Arbitrary.
    - **Constraints**: N/A.
    - **Impact**: Selects each route hop's swap direction.
- `params.amountOut`
    - **Control**: Arbitrary.
    - **Constraints**: Must fit in `int128`.
    - **Impact**: Sets the exact final output amount.
- `params.amountInMaximum`
    - **Control**: Arbitrary.
    - **Constraints**: The terminal callback payment must not exceed this value.
    - **Impact**: Sets the initial input ceiling.
- `params.recipient`
    - **Control**: Arbitrary.
    - **Constraints**: N/A.
    - **Impact**: Receives the final output.
- `params.deadline`
    - **Control**: Arbitrary.
    - **Constraints**: Must not be earlier than the current block timestamp.
    - **Impact**: Limits when execution may occur.
- `msg.value`
    - **Control**: Arbitrary.
    - **Constraints**: N/A.
    - **Impact**: May fund WETH settlement.
- `msg.sender`
    - **Control**: Arbitrary.
    - **Constraints**: N/A.
    - **Impact**: Becomes the initial callback payer.

Branches and code coverage

Intended branches

- Settles two-hop and three-hop routes and returns the recorded initial input.

  ☑ Test coverage
- Starts at the last pool, recursively swaps through prior pools, and delivers each prior output directly to the next pool.

  ☐ Test coverage
- Records terminal callback input after checking its encoded maximum then pays the initial payer's token.

  ☐ Test coverage
- Stores transient callback context before execution and clears it after normal completion.

  ☐ Test coverage
- A nonconforming registered pool can alter forwarded `amountInMax` or repeat terminal callbacks without an automatic revert; normal `MetricOmmPool` forwarding invokes the terminal callback once with unchanged data.

  ☐ Test coverage

Negative branches

- Expiry, malformed route arrays, unregistered pools, output above `int128.max`, or more than 256 hops causes route validation to revert.

  ☑ Test coverage
- A terminal payment above `amountInMax`, incorrect final output, or incorrect intermediate output causes `InputTooHigh`, `InvalidOutputAmount`, or `InvalidOutputAmountAtHop` to revert.

  ☑ Test coverage
- Invalid callback ABI, an invalid callback caller or delta sign, invalid route indexing, or failed external pool/factory calls causes the callback path to revert.

  ☐ Test coverage

Function call analysis

- `_checkDeadline(params.deadline)`
  - **What is controllable?** The caller controls the deadline, amount, direction/bitmap, and route arrays.
  - **If the return value is controllable, how is it used and how can it go wrong?** These internal/library helpers make no external call. The deadline comparison rejects expiry; `asAmountSpecifiedIn` calls `toInt128`, rejects values above `int128.max`, and permits zero at this layer. Path validation checks at least two tokens, matching array lengths, and at most 256 pools, making the subsequent last index valid. Bitmap bits select direction and

`openLimit` supplies zero or `uint128.max`. No pool-token connectivity check or token-metadata call is made. The resulting amount is negated to request exact output.

- **What happens if it reverts, reenters, or does other unusual control flow?** Expiry, invalid path shape where applicable, or out-of-range amount reverts before the pool call. These helpers cannot reenter; pool-specific amount/price constraints are enforced later.

- `_initCallbackContextforRecursiveOutput(pool, CALLBACK_MODE_EXACT_OUTPUT_ITERATE, tradesLeftAfterThis, msg.sender, params.tokens[0])`

  - **What is controllable?** The caller selects the pool and token. The initial payer is `msg.sender`, and the terminal payment token is `params.tokens[0]`; recursive context updates preserve both. Intermediate outputs go directly from the prior pool to the next pool. The immutable factory decides whether the selected pool is registered. The pinned factory returns `poolToIdx[pool] != 0`.
  - **If the return value is controllable, how is it used and how can it go wrong?** `FACTORY.isPool(pool)` is a STATICCALL decoded as `bool`; false causes `InvalidPool`. The internal transient setters return nothing and store the expected callback pool, mode, payer, token, and remaining-hop count. They do not validate the selected token against pool immutables, reject an already active router context, or reset the amount-in slot at entry.
  - **What happens if it reverts, reenters, or does other unusual control flow?** Factory reverts or malformed bool replies propagate; static context prevents its read from performing a state-changing reentry. Subsequent pool/token/extension CALLs can reenter router entry points and overwrite shared transient context. This registry check is callback authentication preparation, not a router-wide reentrancy lock. An uncaught outer failure rolls back the new transient writes.

- `IMetricOmmPoolActions(pool).swap(...)`

  - **What is controllable?** The caller supplies the route and amount-in maximum encoded into the callback struct. The router calls the last registered pool with no ETH; the stored payer and payment token are the caller and `params.tokens[0]`.
  - **If the return value is controllable, how is it used and how can it go wrong?** The returned deltas are decoded as `int128`; `extractAmountOut` must equal the requested final output. Input returned to the caller comes from `_getExactOutputAmountIn()` / `TransientCallbackPool.getAmountIn()`, set by the terminal callback, not the last pool's input delta. There is no final comparison against the original maximum, callback-occurrence check, or output balance measurement. A conforming pool forwards unchanged data and reaches one terminal callback; a nonconforming accepted pool could forge the ceiling or omit that callback, leaving a zero or reentrantly altered recorded amount.

- **What happens if it reverts, reenters, or does other unusual control flow?** External failure, bad return ABI, output mismatch, or checked-negation overflow propagates and unwinds the route. Reverse recursion and token payments occur before this CALL returns and are described below. The amount is read before the outer cleanup zeroes it.

- `_getBidAskReferencePriceX64()`
  - **What is controllable?** The route selects a registered pool and supplies recipient, direction, amount, limit, callback bytes, and extension data. The pool's configured provider, extensions, tokens, liquidity, and fees determine execution. These downstream components are additional trust boundaries; registry membership alone does not validate their behavior.
  - **If the return value is controllable, how is it used and how can it go wrong?** `IPriceProvider(activePriceProvider).getQuote()` is a state-changing CALL, not STATICCALL. Decoded bid/ask/reference values pass ordering, nonzero-bid, range, and configured asymmetry checks before pricing. THe `_executeSwap` scales amounts; applies FeeMath/SwapMath across bins until input/output, liquidity, price, or grid limits stop progress; updates pool accounting; and converts signed deltas to native units with ceiling division. Output token `safeTransfer` uses optional-return handling; input `balanceOf` reads use STATICCALL before/after the callback and must show the required increase. Token-reported balances remain trusted. Extension CALLs must succeed with at least 32 bytes and a matching function selector; their returned bytes do not replace swap deltas.
  - **What happens if it reverts, reenters, or does other unusual control flow?** `whenNotPaused` rejects paused swaps; the pool's transient `nonReentrant(PoolActions.SWAP)` guard blocks overlapping guarded actions on that pool, while callbacks and calls to other pools remain possible. Provider reverts are wrapped as `PriceProviderFailed(reason)`; quote-validation or return-decoding failures propagate separately. Extensions run before the swap calculation and after settlement through `CallExtension.callExtension(extension, data)`; nonempty failure bytes are bubbled, empty failures become `ExtensionCallFailed`, and bad acknowledgments become `InvalidExtensionResponse`. Any uncaught pool, token, extension, arithmetic, cast, or downstream callback error reverts that swap. These pool protections do not install a router-wide reentrancy guard.

- `_requireExpectedCallbackCaller(msg.sender)`
  - **What is controllable?** The callback caller supplies deltas and data but must match `TransientCallbackPool.getPool()` and pass `FACTORY.isPool(caller)`. The expected pool/mode/payer/token were set by a swap entry or prior recursive hop. The configured factory's registry response is trusted.
  - **If the return value is controllable, how is it used and how can it go wrong?** `TransientCallbackPool.requireCaller(caller)` compares against transient state before the factory STATICCALL. A false factory result rejects

the caller. The mode selects just-payment or recursive-output handling. The `extractPositiveAmount` calls `getPositiveAmount`, accepting exactly one positive and one strictly negative delta; it does not verify that the positive side matches the stored token or direction. Its positive `int256` result becomes the payment or preceding-hop output request.

- **What happens if it reverts, reenters, or does other unusual control flow?** Invalid caller, factory failure/malformed response, or invalid deltas revert. The `InvalidCallbackMode` is a defensive check: the deployed router's public paths write only modes 0 and 1, recursive updates preserve the mode, and clearing writes zero, so an unsupported mode is not reachable through these paths. The factory read cannot mutate state through reentry. A valid callback is an expected ordinary CALL from the pool; the router has no callback-wide reentrancy guard, and authentication does not consume the payment context.

- `_exactOutputIterateCallback(amount0Delta, amount1Delta, data)`

  - **What is controllable?** The authenticated pool supplies `data`, decoded as `ExactOutputIterateCallbackData`, and positive payment demand. A conforming pool forwards the router's original encoding unchanged; the callback does not compare it to a stored hash. Stored `tradesLeft` controls the reverse index; `cb.pools`, bitmap, extension data, and `amountInMax` come from those callback bytes, and `cb.tokens` is not read by this handler.
  - **If the return value is controllable, how is it used and how can it go wrong?** At zero remaining trades, the handler checks the demand against `cb.amountInMax`, stores it via `_setExactOutputAmountIn`, and pays using the stored first token/payer. Otherwise, it decrements the index, resolves direction, checks the preceding pool in the factory, and updates the expected pool before a negative exact-output swap whose recipient is the current callback sender. The `asAmountSpecifiedFromPositive` requires a positive value fitting `int128` and uses checked narrowing/negation. Returned deltas are reduced to output by `extractAmountOut` and must equal the demand. Forged callback data from a nonconforming accepted pool could change the input ceiling or route; membership and output equality are not a commitment to the initial encoded route.
  - **What happens if it reverts, reenters, or does other unusual control flow?** Malformed ABI, out-of-range array indexing, invalid amounts, failed registry checks, pool reverts, or output mismatch propagate. Recursion nests pool CALLs and callbacks until the stored count reaches zero; the pinned pool guard rejects recursively reusing an already active pool. Context is overwritten for the preceding pool and is not restored on unwind. Terminal payment does not clear/consume it, so a nonconforming expected pool could repeat terminal callbacks, with each independently checked against the maximum. Normal outer completion clears context; a revert restores transient and external state to the reverting frame's entry.

- `pay(token, payer, recipient, value)`

  - **What is controllable?** The callback supplies the payment amount and

recipient; the active context supplies payer and token. For a contract-funded intermediate hop, `payer == address(this)` always transfers ERC-20 already held here. Otherwise, WETH payment first consumes the contract's live ETH balance, wrapping all or part of the required amount, and pulls any remainder from the payer; a non-WETH payment pulls the full amount. ETH is not attributed to individual callers.

- **If the return value is controllable, how is it used and how can it go wrong?** Neither `pay` nor WETH deposit returns a checked value. Deposit is a value-bearing CALL; success is trusted to mint WETH. SafeERC20 performs zero-ETH low-level CALLs and accepts a successful checked return word of `1` or successful empty return data from a code-bearing token; its assembly does not perform a full ABI length check. Nonaccepted results become `SafeERC20FailedOperation`, and success alone does not prove receipt. The pinned pool separately measures input balances.
- **What happens if it reverts, reenters, or does other unusual control flow?** Deposit and token CALLs can reenter public functions while callback context remains active. The payment helper has no guard. Token revert data is bubbled by SafeERC20; WETH reverts propagate. An uncaught failure unwinds payment, wrapping, pool writes, and transient writes in the failed invocation; a catching outer caller may continue from its precall state.

- `_clearExpectedCallbackPool()`
  - **What is controllable?** The success path determines when clearing happens; callees have had access to the preceding callback window.
  - **If the return value is controllable, how is it used and how can it go wrong?** There is no external return. Four transient slots are zeroed: expected pool/mode/count, recorded input, payer, and token. This prevents later calls in the transaction from reusing context after normal completion.
  - **What happens if it reverts, reenters, or does other unusual control flow?** No external code is called. A revert before reaching this cleanup still unwinds transient writes made within the reverted invocation. Nested successful router calls can themselves clear the shared context; there is no stack of saved contexts.

Function: `function exactOutputSingle(ExactOutputSingleParams calldata params) external payable returns (uint256 amountIn)`

This function executes one exact-output swap through a registered pool, checks exact output, and enforces the computed input limit.

Inputs

- `params.pool`
  - **Control**: Arbitrary.

  - **Constraints**: Must be registered in `FACTORY` when the callback is validated.
  - **Impact**: Identifies the pool to call.
- `params.tokenIn`
  - **Control**: Arbitrary.
  - **Constraints**: N/A.
  - **Impact**: Identifies the token paid in the callback.
- `params.zeroForOne`
  - **Control**: Arbitrary.
  - **Constraints**: N/A.
  - **Impact**: Selects the swap direction.
- `params.priceLimitX64`
  - **Control**: Arbitrary.
  - **Constraints**: N/A.
  - **Impact**: Sets the pool price limit after direction-specific open-price sentinel normalization.
- `params.amountOut`
  - **Control**: Arbitrary.
  - **Constraints**: Must fit in `int128`.
  - **Impact**: Sets the exact output amount.
- `params.amountInMaximum`
  - **Control**: Arbitrary.
  - **Constraints**: The computed input must not exceed this value.
  - **Impact**: Sets the input ceiling.
- `params.recipient`
  - **Control**: Arbitrary.
  - **Constraints**: N/A.
  - **Impact**: Receives the swap output.
- `params.deadline`
  - **Control**: Arbitrary.
  - **Constraints**: Must not be earlier than the current block timestamp.
  - **Impact**: Limits when execution may occur.
- `params.extensionData`
  - **Control**: Arbitrary.
  - **Constraints**: N/A.
  - **Impact**: Is forwarded to the pool.
- `msg.value`
  - **Control**: Arbitrary.

- **Constraints**: N/A.
- **Impact**: May fund WETH settlement.

- `msg.sender`
  - **Control**: Arbitrary.
  - **Constraints**: N/A.
  - **Impact**: Becomes the callback payer.

Branches and code coverage

Intended branches

- Executes and settles both swap directions.
  - ☑ Test coverage
- Normalizes open-price sentinels, stores callback context, and clears it after success.
  - ☐ Test coverage
- Compares exact returned output with the request and computed returned input with `amountInMaximum`.
  - ☐ Test coverage

Negative branches

- Expiry, unregistered pools, input above the maximum, output above `int128.max`, and incorrect returned output cause their respective validation errors to revert.
  - ☑ Test coverage
- Payment failures, invalid callback callers or deltas, and SafeERC20 failures cause the call to revert.
  - ☐ Test coverage
- A registered malicious pool can report returned deltas that differ from repeated callback payment requests; normal `MetricOmmPool` behavior uses one matching callback.
  - ☐ Test coverage

Function call analysis

- `_checkDeadline(params.deadline)`
  - **What is controllable?** The caller controls the deadline, amount, direction/bitmap, and single-pool price limit.
  - **If the return value is controllable, how is it used and how can it go wrong?** These internal/library helpers make no external call. The deadline

comparison rejects expiry; `asAmountSpecifiedIn` calls `toInt128`, rejects values above `int128.max`, and permits zero at this layer. Normalization maps the opposite-direction open-limit sentinel to the canonical limit and otherwise preserves the caller's value. The resulting amount is negated to request exact output.

- **What happens if it reverts, reenters, or does other unusual control flow?** Expiry, invalid path shape where applicable, or out-of-range amount reverts before the pool call. These helpers cannot reenter; pool-specific amount/price constraints are enforced later.

- `_setNextCallbackContext(params.pool, CALLBACK_MODE_JUST_PAY, msg.sender, params.tokenIn)`

  - **What is controllable?** The caller selects the pool and token. The payer stored for this entry is `msg.sender` and the payment token is `params.tokenIn`. The immutable factory decides whether the selected pool is registered. The pinned factory returns `poolToIdx[pool] != 0`.
  - **If the return value is controllable, how is it used and how can it go wrong?** `FACTORY.isPool(pool)` is a STATICCALL decoded as `bool`; false causes `InvalidPool`. The internal transient setters return nothing and store the expected callback pool, mode, payer, token. They do not validate the selected token against pool immutables, reject an already active router context, or reset the amount-in slot at entry.
  - **What happens if it reverts, reenters, or does other unusual control flow?** Factory reverts or malformed bool replies propagate; static context prevents its read from performing a state-changing reentry. Subsequent pool/token/extension CALLs can reenter router entry points and overwrite shared transient context. This registry check is callback authentication preparation, not a router-wide reentrancy lock. An uncaught outer failure rolls back the new transient writes.

- `IMetricOmmPoolActions(params.pool).swap(params.recipient, params.zeroForOne, -expectedAmountOut, priceLimitX64, "", params.extensionData)`

  - **What is controllable?** The caller controls the registered pool and swap parameters; the router sets empty callback bytes and forwards no ETH.
  - **If the return value is controllable, how is it used and how can it go wrong?** The `extractAmountOut` negates the chosen returned delta and must equal `expectedAmountOut`. The `extractAmountIn` selects the other delta then `int128ToUint128` checks its sign through SafeCast; the result must not exceed `params.amountInMaximum`. These checks use pool-returned values, not a tally of callback payments or recipient balance changes. An accepted nonconforming pool could report compliant deltas after making excessive or repeated payment requests.
  - **What happens if it reverts, reenters, or does other unusual control flow?** Pool reverts and return-decoding failures propagate. Selected output `int128.min` causes checked-negation failure; a negative input causes the

checked unsigned conversion to revert. Callback payment runs before these checks, so any final output/max-input failure rolls back the callback and pool effects.

- `_getBidAskReferencePriceX64()`
  - **What is controllable?** The route selects a registered pool and supplies recipient, direction, amount, limit, callback bytes, and extension data. The pool's configured provider, extensions, tokens, liquidity, and fees determine execution. These downstream components are additional trust boundaries; registry membership alone does not validate their behavior.
  - **If the return value is controllable, how is it used and how can it go wrong?** `IPriceProvider(activePriceProvider).getQuote()` is a state-changing CALL, not STATICCALL. Decoded bid/ask/reference values pass ordering, nonzero-bid, range, and configured asymmetry checks before pricing. The `_executeSwap` scales amounts; applies FeeMath/SwapMath across bins until input/output, liquidity, price, or grid limits stop progress; updates pool accounting; and converts signed deltas to native units with ceiling division. Output token `safeTransfer` uses optional-return handling; input `balanceOf` reads use STATICCALL before/after the callback and must show the required increase. Token-reported balances remain trusted. Extension CALLs must succeed with at least 32 bytes and a matching function selector; their returned bytes do not replace swap deltas.
  - **What happens if it reverts, reenters, or does other unusual control flow?** `whenNotPaused` rejects paused swaps; the pool's transient `nonReentrant(PoolActions.SWAP)` guard blocks overlapping guarded actions on that pool, while callbacks and calls to other pools remain possible. Provider reverts are wrapped as `PriceProviderFailed(reason)`; quote-validation or return-decoding failures propagate separately. Extensions run before the swap calculation and after settlement through `CallExtension.callExtension(extension, data)`; nonempty failure bytes are bubbled, empty failures become `ExtensionCallFailed`, and bad acknowledgments become `InvalidExtensionResponse`. Any uncaught pool, token, extension, arithmetic, cast, or downstream callback error reverts that swap. These pool protections do not install a router-wide reentrancy guard.
- `_requireExpectedCallbackCaller(msg.sender)`
  - **What is controllable?** The callback caller supplies deltas and data but must match `TransientCallbackPool.getPool()` and pass `FACTORY.isPool(caller)`. The expected pool/mode/payer/token were set by a swap entry or prior recursive hop. The configured factory's registry response is trusted.
  - **If the return value is controllable, how is it used and how can it go wrong?** `TransientCallbackPool.requireCaller(caller)` compares against transient state before the factory STATICCALL. A false factory result rejects the caller. The mode selects just-payment or recursive-output handling. The `extractPositiveAmount` calls `getPositiveAmount`, accepting exactly one

positive and one strictly negative delta; it does not verify that the positive side matches the stored token or direction. Its positive `int256` result becomes the payment or preceding-hop output request.

- **What happens if it reverts, reenters, or does other unusual control flow?** Invalid caller, factory failure/malformed response, or invalid deltas revert. The `InvalidCallbackMode` is a defensive check: the deployed router's public paths write only modes 0 and 1, recursive updates preserve the mode, and clearing writes zero, so an unsupported mode is not reachable through these paths. The factory read cannot mutate state through reentry. A valid callback is an expected ordinary CALL from the pool; the router has no callback-wide reentrancy guard, and authentication does not consume the payment context.

- `_justPayCallback(amount0Delta, amount1Delta)`
  - **What is controllable?** An authenticated expected pool supplies both deltas. Transient getters read the token and payer established by the router; the recipient is the callback sender.
  - **If the return value is controllable, how is it used and how can it go wrong?** The helper selects the strictly positive delta and converts it to `uint256` then calls `pay`. There is no helper return, no local cap against the entry's specified amount, and no callback count or balance receipt recorded here. Payment limits therefore rely on the conforming pool's deltas and the entry's later return-value checks; an accepted nonconforming pool is not constrained to a single payment request.
  - **What happens if it reverts, reenters, or does other unusual control flow?** Invalid sign combinations revert; payment failures propagate. Context remains active across external token/WETH execution and after a successful callback until the outer entry clears it. Direct token reentry cannot impersonate the expected pool, but other public entry points are not guarded and may overwrite context.
- `pay(token, payer, recipient, value)`
  - **What is controllable?** The callback supplies the payment amount and recipient; the active context supplies payer and token. For a contract-funded intermediate hop, `payer == address(this)` always transfers ERC-20 already held here. Otherwise, WETH payment first consumes the contract's live ETH balance, wrapping all or part of the required amount, and pulls any remainder from the payer; a non-WETH payment pulls the full amount. ETH is not attributed to individual callers.
  - **If the return value is controllable, how is it used and how can it go wrong?** Neither `pay` nor WETH deposit returns a checked value. Deposit is a value-bearing CALL; success is trusted to mint WETH. SafeERC20 performs zero-ETH low-level CALLs and accepts a successful checked return word of `1`, or successful empty return data from a code-bearing token; its assembly does not perform a full ABI length check. Nonaccepted results become `SafeERC20FailedOperation`, and success alone does not prove receipt. The pinned pool separately measures input balances.

- **What happens if it reverts, reenters, or does other unusual control flow?** Deposit and token CALLs can reenter public functions while callback context remains active. The payment helper has no guard. Token revert data is bubbled by SafeERC20; WETH reverts propagate. An uncaught failure unwinds payment, wrapping, pool writes, and transient writes in the failed invocation; a catching outer caller may continue from its precall state.

- `_clearExpectedCallbackPool()`
  - **What is controllable?** The success path determines when clearing happens; callees have had access to the preceding callback window.
  - **If the return value is controllable, how is it used and how can it go wrong?** There is no external return. Four transient slots are zeroed: expected pool/mode/count, recorded input, payer, and token. This prevents later calls in the transaction from reusing context after normal completion.
  - **What happens if it reverts, reenters, or does other unusual control flow?** No external code is called. A revert before reaching this cleanup still unwinds transient writes made within the reverted invocation. Nested successful router calls can themselves clear the shared context; there is no stack of saved contexts.

Function: `function metricOmmSwapCallback(int256 amount0Delta, int256 amount1Delta, bytes calldata data) external override`

This function processes a callback from the expected pool — `JUST_PAY` pays the stored token from the stored payer to `msg.sender`; `EXACT_OUTPUT_ITERATE` either recursively executes the prior hop or pays the terminal input.

Inputs

- `amount0Delta`
  - **Control**: Callback caller.
  - **Constraints**: Together with `amount1Delta`, one delta must be strictly positive and the other strictly negative in either mode.
  - **Impact**: May specify the token0 amount to pay or recursively fund.
- `amount1Delta`
  - **Control**: Callback caller.
  - **Constraints**: Together with `amount0Delta`, one delta must be strictly positive and the other strictly negative in either mode.
  - **Impact**: May specify the token1 amount to pay or recursively fund.
- `data`
  - **Control**: Router-generated callback data for exact output, otherwise callback caller.

- **Constraints**: In `EXACT_OUTPUT_ITERATE`, must ABI-decode as `ExactOutputIterateCallbackData`; `JUST_PAY` does not read it.
- **Impact**: Supplies the route arrays, direction bitmap, and input maximum for exact output. The remaining hop count is read from transient storage.

- `msg.sender`
  - **Control**: Callback caller.
  - **Constraints**: Must equal the stored expected pool and remain registered in `FACTORY`.
  - **Impact**: Identifies the pool permitted to receive payment.

Branches and code coverage

Intended branches

- In `JUST_PAY`, validates the caller and pays the one positive delta with the stored payer and token.
  - ☐ Test coverage
- In `EXACT_OUTPUT_ITERATE`, either starts the prior hop or records and pays the terminal input.
  - ☐ Test coverage

Negative branches

- No active context, an unexpected caller, an unregistered caller, malformed callback data, or invalid delta signs causes the callback to revert.
  - ☐ Test coverage
- An input above the encoded limit or an intermediate amount above `int128.max` causes the exact-output callback to revert.
  - ☐ Test coverage

Function call analysis

- `_requireExpectedCallbackCaller(msg.sender)`
  - **What is controllable?** The callback caller supplies deltas and data but must match `TransientCallbackPool.getPool()` and pass `FACTORY.isPool(caller)`. The expected pool/mode/payer/token were set by a swap entry or prior recursive hop. The configured factory's registry response is trusted.
  - **If the return value is controllable, how is it used and how can it go wrong?** `TransientCallbackPool.requireCaller(caller)` compares against transient state before the factory STATICCALL. A false factory result rejects

the caller. The mode selects just-payment or recursive-output handling. The `extractPositiveAmount` calls `getPositiveAmount`, accepting exactly one positive and one strictly negative delta; it does not verify that the positive side matches the stored token or direction. Its positive `int256` result becomes the payment or preceding-hop output request.

- **What happens if it reverts, reenters, or does other unusual control flow?** Invalid caller, factory failure/malformed response, or invalid deltas revert. The `InvalidCallbackMode` is a defensive check: the deployed router's public paths write only modes 0 and 1, recursive updates preserve the mode, and clearing writes zero, so an unsupported mode is not reachable through these paths. The factory read cannot mutate state through reentry. A valid callback is an expected ordinary CALL from the pool; the router has no callback-wide reentrancy guard, and authentication does not consume the payment context.

- `_justPayCallback(amount0Delta, amount1Delta)`

  - **What is controllable?** An authenticated expected pool supplies both deltas. Transient getters read the token and payer established by the router; the recipient is the callback sender.
  - **If the return value is controllable, how is it used and how can it go wrong?** The helper selects the strictly positive delta and converts it to `uint256` then calls `pay`. There is no helper return, no local cap against the entry's specified amount, and no callback count or balance receipt recorded here. Payment limits therefore rely on the conforming pool's deltas and the entry's later return-value checks; an accepted nonconforming pool is not constrained to a single payment request.
  - **What happens if it reverts, reenters, or does other unusual control flow?** Invalid sign combinations revert; payment failures propagate. Context remains active across external token/WETH execution and after a successful callback until the outer entry clears it. Direct token reentry cannot impersonate the expected pool, but other public entry points are not guarded and may overwrite context.

- `_exactOutputIterateCallback(amount0Delta, amount1Delta, data)`

  - **What is controllable?** The authenticated pool supplies `data`, decoded as `ExactOutputIterateCallbackData`, and positive payment demand. A conforming pool forwards the router's original encoding unchanged; the callback does not compare it to a stored hash. Stored `tradesLeft` controls the reverse index; `cb.pools`, bitmap, extension data, and `amountInMax` come from those callback bytes, and `cb.tokens` is not read by this handler.
  - **If the return value is controllable, how is it used and how can it go wrong?** At zero remaining trades, the handler checks the demand against `cb.amountInMax`, stores it via `_setExactOutputAmountIn`, and pays using the stored first token/payer. Otherwise, it decrements the index, resolves direction, checks the preceding pool in the factory, and updates the expected pool before a negative exact-output swap whose recipient is the current callback sender. The `asAmountSpecifiedFromPositive` requires a

positive value fitting `int128` and uses checked narrowing/negation. Returned deltas are reduced to output by `extractAmountOut` and must equal the demand. Forged callback data from a nonconforming accepted pool could change the input ceiling or route; membership and output equality are not a commitment to the initial encoded route.

- **What happens if it reverts, reenters, or does other unusual control flow?** Malformed ABI, out-of-range array indexing, invalid amounts, failed registry checks, pool reverts, or output mismatch propagate. Recursion nests pool CALLs and callbacks until the stored count reaches zero; the pinned pool guard rejects recursively reusing an already active pool. Context is overwritten for the preceding pool and is not restored on unwind. Terminal payment does not clear/consume it, so a nonconforming expected pool could repeat terminal callbacks, with each independently checked against the maximum. Normal outer completion clears context; a revert restores transient and external state to the reverting frame's entry.

- `_getBidAskReferencePriceX64()`

  - **What is controllable?** The route selects a registered pool and supplies recipient, direction, amount, limit, callback bytes, and extension data. The pool's configured provider, extensions, tokens, liquidity, and fees determine execution. These downstream components are additional trust boundaries; registry membership alone does not validate their behavior.
  - **If the return value is controllable, how is it used and how can it go wrong?** `IPriceProvider(activePriceProvider).getQuote()` is a state-changing CALL, not STATICCALL. Decoded bid/ask/reference values pass ordering, nonzero-bid, range, and configured asymmetry checks before pricing. The `_executeSwap` scales amounts; applies FeeMath/SwapMath across bins until input/output, liquidity, price, or grid limits stop progress; updates pool accounting; and converts signed deltas to native units with ceiling division. Output token `safeTransfer` uses optional-return handling; input `balanceOf` reads use STATICCALL before/after the callback and must show the required increase. Token-reported balances remain trusted. Extension CALLs must succeed with at least 32 bytes and a matching function selector; their returned bytes do not replace swap deltas.
  - **What happens if it reverts, reenters, or does other unusual control flow?** `whenNotPaused` rejects paused swaps; the pool's transient `nonReentrant(PoolActions.SWAP)` guard blocks overlapping guarded actions on that pool, while callbacks and calls to other pools remain possible. Provider reverts are wrapped as `PriceProviderFailed(reason)`; quote-validation or return-decoding failures propagate separately. Extensions run before the swap calculation and after settlement through `CallExtension.callExtension(extension, data)`; nonempty failure bytes are bubbled, empty failures become `ExtensionCallFailed`, and bad acknowledgments become `InvalidExtensionResponse`. Any uncaught pool, token, extension, arithmetic, cast, or downstream callback error reverts that swap. These pool protections do not install a router-wide reentrancy guard.

- `pay(token, payer, recipient, value)`
    - **What is controllable?** The callback supplies the payment amount and recipient; the active context supplies payer and token. For a contract-funded intermediate hop, `payer == address(this)` always transfers ERC-20 already held here. Otherwise, WETH payment first consumes the contract's live ETH balance, wrapping all or part of the required amount, and pulls any remainder from the payer; a non-WETH payment pulls the full amount. ETH is not attributed to individual callers.
    - **If the return value is controllable, how is it used and how can it go wrong?** Neither `pay` nor WETH deposit returns a checked value. Deposit is a value-bearing CALL; success is trusted to mint WETH. SafeERC20 performs zero-ETH low-level CALLs and accepts a successful checked return word of `1` or successful empty return data from a code-bearing token; its assembly does not perform a full ABI length check. Nonaccepted results become `SafeERC20FailedOperation`, and success alone does not prove receipt. The pinned pool separately measures input balances.
    - **What happens if it reverts, reenters, or does other unusual control flow?** Deposit and token CALLs can reenter public functions while callback context remains active. The payment helper has no guard. Token revert data is bubbled by SafeERC20; WETH reverts propagate. An uncaught failure unwinds payment, wrapping, pool writes, and transient writes in the failed invocation; a catching outer caller may continue from its precall state.

Function: `function multicall(bytes[] calldata data) public payable returns (bytes[] memory results)`

This function delegatecalls this router for each supplied payload in order. Every nested call shares the original sender, value, and storage.

Inputs

- `data`
    - **Control**: Arbitrary.
    - **Constraints**: N/A.
    - **Impact**: Defines call order and returned byte arrays.
- `msg.value`
    - **Control**: Arbitrary.
    - **Constraints**: N/A.
    - **Impact**: Provides the shared ETH balance.
- `msg.sender`
    - **Control**: Arbitrary.

- **Constraints**: N/A.
- **Impact**: Is preserved for nested delegatecalls.

Branches and code coverage

Intended branches

- Executes deposit and refund operations in sequence.
  - ☑ Test coverage
- An empty array succeeds with an empty result; each successful delegatecall stores its raw return bytes.
  - ☐ Test coverage

Negative branches

- A nested revert reverts the entire batch. Nonempty revert data is bubbled unchanged, while empty revert data becomes `FailedCall()`.
  - ☐ Test coverage

Function call analysis

- `Address.functionDelegateCall(address(this), data[i])`
  - **What is controllable?** The caller controls every payload, selector, argument, and ordering. The target is fixed to this contract, so these are self-DELEGATECALLs rather than delegatecalls to caller-selected foreign code. Original `msg.sender`, `msg.value`, contract balance, persistent storage, and transient storage are shared with every subcall.
  - **If the return value is controllable, how is it used and how can it go wrong?** The `LowLevelCall` returns only its success boolean while leaving return data available. The `Address` returns copied raw bytes when successful with nonempty data or a code-bearing target; each byte string becomes `results[i]` without semantic decoding and is not passed into later entries automatically. The self target has code in this deployed entry, so the library's empty-code success rejection is not an ordinary reachable branch. A successful subcall returning a false-like value is still a successful multicall element.
  - **What happens if it reverts, reenters, or does other unusual control flow?** Nonempty failure bytes are bubbled by `LowLevelCall.bubbleRevert`; an empty revert becomes `Errors.FailedCall()`. There is no per-element catch, so any uncaught element failure unwinds previous elements' writes, transfers, and transient context in this invocation. External callbacks reached by a selected entry can reenter multicall; the loop itself adds no guard. Recursion and large byte arrays can exhaust gas, with the same

rollback.

- Selector dispatch
  - **What is controllable?** Caller-selected selectors can reach swap entries (`exactInputSingle` ↗, `exactInput` ↗, `exactOutputSingle` ↗, `exactOutput` ↗), swap callbacks ↗, nested multicalls, and inherited permit helpers ↗ and payment helpers ↗. Each route is subject to that function's validation and call analysis; arbitrary selectors do not bypass dispatcher checks.
  - **If the return value is controllable, how is it used and how can it go wrong?** Each selected entry controls its own internal/library/external calls and output. Payment entries can transfer the entire live contract asset balance; selectors are not limited to operations on balances contributed by this batch caller. All subcalls see the same original `msg.value`, but DELEGATECALL does not transfer that ETH again or replenish the live balance. Nonpayable selectors reject a nonzero inherited `msg.value`.
  - **What happens if it reverts, reenters, or does other unusual control flow?** Swap callbacks still require the expected factory-registered pool; swap entry reentry can overwrite shared transient context because its setters lack an active-context guard. Permit token CALLs and payment token/ETH CALLs introduce their own reentry windows. Empty payloads dispatch to inherited `receive()`, whose WETH-sender check still applies. An error propagated out of any selected router entry fails the batch. A downstream callee may catch its own nested error and return successfully; that does not itself fail the element.

5.6. OracleValueStopLossExtension.sol

Function: `function afterSwap( address, address, bool zeroForOne, int128, uint128, uint256 packedSlot0Initial, uint256 packedSlot0Final, uint128 bidPriceX64, uint128 askPriceX64, uint128, int128, int128, uint256, bytes calldata ) external override returns (bytes4)`

For an initialized calling pool, this function checks the range of bins traversed by the trade and determines by direction whether bid-/ask-derived per-share value violates the permitted drawdown from the decayed high-water mark (HWM). On success, it updates the HWM and last-decay timestamp and returns the selector. If this hook errors in the pool's actual `afterSwap` path, earlier swap state changes and token settlement are also reverted.

Inputs

- `zeroForOne`
  - **Control**: Calling pool.
  - **Constraints**: N/A.

    - **Impact**: When `true`, blocks token0-denominated value drawdowns — when `false`, blocks token1-denominated value drawdowns.
- `packedSlot0Initial`
    - **Control**: Calling pool.
    - **Constraints**: N/A.
    - **Impact**: Provides the starting bin index and, with the final index, determines the inclusive traversal range.
- `packedSlot0Final`
    - **Control**: Calling pool.
    - **Constraints**: N/A.
    - **Impact**: Provides the final bin index and, with the initial index, determines the inclusive traversal range.
- `bidPriceX64`
    - **Control**: Calling pool./oracle.
    - **Constraints**: Must be nonzero when token1 value is converted to token0 units.
    - **Impact**: Used as the denominator when converting token1 per-share value to token0 units.
- `askPriceX64`
    - **Control**: Calling pool./oracle.
    - **Constraints**: N/A.
    - **Impact**: Multiplies token0 per-share value when converting it to token1 units.
- `msg.sender`
    - **Control**: Caller.
    - **Constraints**: The caller address's stop-loss configuration must be initialized.
    - **Impact**: Determines the pool to check and keys its pool-specific configuration and watermarks.

Branches and code coverage

Intended branches

- With `drawdown = 0`, the hook returns successfully.
    - ☑ Test coverage
- This path skips pool external queries and all HWM updates.
    - ☐ Test coverage
- On first contact, a zero HWM is initialized to the current metric. It cannot protect against the loss before that first trade.

☑ Test coverage

- Raises the HWM to a new high when the current metric exceeds it.

☑ Test coverage

- Passes when the current metric is lower than the HWM but remains within the permitted drawdown.

☑ Test coverage

- Passes when exactly at the threshold.

☑ Test coverage

- Deletes the HWM when traversing an empty bin (`totalShares = 0`) and initializes it again on first contact after redeposit.

☑ Test coverage

- Deletion also includes the token1 HWM and `lastDecayTs`.

☐ Test coverage

- Saturates the metric at `uint104.max` for extreme balances and dust shares.

☑ Test coverage

- Floors the shares used as the calculation denominator at `minimalOperationalLiquidity`. The saturation test above does not independently demonstrate this denominator adjustment.

☐ Test coverage

- After `uint40` timestamp rollover, the difference between wrapped `lastDecayTs` and the current timestamp can be very large. With positive decay, it can decay the HWM to zero and then update it to the current metric, effectively reinitializing the protection baseline.

☐ Test coverage

- Checks multiple bins, including both endpoints.

☑ Test coverage

- When decay rate times' elapsed time is at least `1e18`, an existing HWM falls to zero and can rise again to the current value.

☑ Test coverage

- Even where a bin-balance decrease at the same price is blocked, the trade can later pass in that direction after the HWM decays.

☑ Test coverage

- Allows a trade in the opposite direction when only the token0 metric breaches the drawdown limit and `zeroForOne = false`.

☑ Test coverage

- Allows a trade in the opposite direction when only the token1 metric breaches the drawdown limit and `zeroForOne = true`.

☑ Test coverage

- Replaces `minimalOperationalLiquidity = 0` with one.

☐ Test coverage

- Uses the ask price only in the conversion term for the token1-denominated metric. The token0-denominated metric uses the bid, independently reflecting price effects in both directions.

☐ Test coverage

- The calculation first produces `t0ps=floor(t0×1e18/shares)` and `t1ps=floor(t1×1e18/shares)`. It then produces `metric0=clamp(t0ps+floor(t1ps×2^64/bid))` and `metric1=clamp(floor(t0ps×ask/2^64)+t1ps)`. No test was confirmed to validate intermediate rounding directly.

☐ Test coverage

- `SlotOLibrary.unpack` sign-extends 16 bits beginning at bit 8 of the two packed Slot0 values to obtain the start and end bins. It sets the smaller value to `lo`, the larger to `hi`, and constructs `count = hi - lo + 1` indexes in ascending order, so it checks the same bin once and includes both endpoints regardless of movement direction.

☐ Test coverage

- For each bin, `PoolStateLibrary` computes the keys for state mapping slot 4 and share mapping slot 5 with `keccak256`. It queries all state slots and all share slots through two `extsload(bytes32[])` calls then traverses them paired by matching array index.

☐ Test coverage

- Interprets the share word as the full `uint256`. For a bin with shares, extracts the low 104 bits as `t0` and the next 104 bits as `t1`; length and fee fields are not used in metric calculation. For a bin with no shares, deletes the watermarks and continues, skipping balance, price, and decay calculations.

☐ Test coverage

- `_metrics` selects `minShares` as the denominator only when `totalShares < minShares`; otherwise, it uses `totalShares`. Both `t0ps` / `t1ps` products and the bid conversion fit in 256 bits and round down through `Math.mulDiv`'s simple-division path.

☐ Test coverage

- If `t0ps × ask` exceeds 256 bits in the ask conversion, `Math.mulDiv` calculates the quotient through its 512-bit path. It obtains high and low products with `mul512`, subtracts the remainder, and removes powers of two from the denominator to obtain the exact quotient, so intermediate product size alone does not revert. The final quotient fits in `uint256` for the input type ranges.

☐ Test coverage

- `_clampMetric` caps each metric exceeding `uint104.max`; otherwise, it preserves the original value. This capping is applied to both metrics.

- ☐ Test coverage
- For each live bin, calculates `dt = block.timestamp - lastDecayTs` once; `_decayed` returns the HWM unchanged when the rate is zero, when the rate is positive but `dt` is zero, or when both are positive but the HWM is zero.
  - ☐ Test coverage
- Outside those early returns, calculates `factor = rate × dt`. For `factor < 1e18`, returns `HWM - floor(HWM × factor / 1e18)`; for `factor ≥ 1e18`, returns zero. It decays token0 and token1 HWMs using the same rate and `dt`.
  - ☐ Test coverage
- In `_applyWatermark`, if `metric ≥` the decayed HWM, selects the metric as the new HWM and sets `breach=false`. If `metric < HWM`, preserves the existing HWM and calculates whether `metric < floor(HWM × (1e6 - drawdown) / 1e6)`.
  - ☐ Test coverage
- First checks the token0 breach and `zeroForOne` — only if it passes does it calculate and direction-check token1. If both direction checks pass, stores the selected two HWMs together with `uint40(block.timestamp)`. Both HWMs and the decay time are updated even when a breach in the opposite direction is allowed.
  - ☐ Test coverage
- Returns the `afterSwap` selector after processing all bins. Traversal, including empty bins, makes no token-transfer or external state-changing calls; pool queries are view calls.
  - ☐ Test coverage

Negative branches

- Reverts with `OracleStopLossNotInitialized` for an uninitialized caller.
  - ☐ Test coverage
- Reverts with `OracleStopLossTriggered` for `zeroForOne=true` when only the token0 metric breaches the drawdown limit.
  - ☑ Test coverage
- Reverts with `OracleStopLossTriggered` for `zeroForOne=false` when only the token1 metric breaches the drawdown limit.
  - ☑ Test coverage
- Both directions revert when both metrics breach.
  - ☑ Test coverage
- Reverts with `OracleStopLossTriggered` when an earlier HWM remains after removing and redepositing without an empty-state hook traversal, and the current metric in that direction is below that HWM's permitted drawdown limit.
  - ☑ Test coverage

- Reverts through `mulDiv`'s zero denominator if `bid = 0` and execution reaches live-bin calculation.
  - ☐ Test coverage
- The entire call reverts if `getImmutables`, `extsload`, or returned ABI decoding reverts.
  - ☐ Test coverage
- Reverts with an array-bounds panic if the returned shares array lacks the current `i` or the states array lacks the current `i` for a bin with `totalShares > 0`.
  - ☐ Test coverage
- Reverts with an arithmetic-overflow panic if `rate × dt` exceeds `uint256` in a bin that does not take a decay early return.
  - ☐ Test coverage
- The entire hook reverts if a wide bin range exhausts gas while constructing index or slot arrays, performing the two external queries, or calculating each bin.
  - ☐ Test coverage
- If a later bin errors, earlier bin HWM deletions, updates, and decay-time changes are all reverted, and the entire hook call reverts.
  - ☐ Test coverage

Function call analysis

- `_requireInitialized(msg.sender)`
  - **What is controllable?** The hook caller supplies its own address implicitly; the initialization flag at that key can only be set by the immutable factory.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A — no return value. Success permits analysis of the caller's own pool state; it does not perform a separate factory registration lookup.
  - **What happens if it reverts, reenters, or does other unusual control flow?** The `OracleStopLossNotInitialized` propagates before any pool query. This helper is an internal storage check without an external callback.
- `_afterSwapOracleStopLoss(msg.sender, packedSlot0Initial, packedSlot0Final, bidPriceX64, askPriceX64, zeroForOne)`
  - **What is controllable?** The immediate caller is the initialized pool. In the supplied core's `swap`, the swap caller chooses direction, amount, and price limit; the pool obtains bid/ask from its configured provider and validates them before passing them to this hook. The creator selects the initial provider, and the pool admin can replace a mutable provider through the factory's delayed configuration path. The initial and final packed words come from the pool's swap state, so users influence the traversed bin range through trades rather than supplying arbitrary packed words on the normal

swap path; `simulateSwapAndRevert` instead accepts caller-selected prices, validates them, invokes the same hook, and ultimately reverts all simulation effects. The admin controls the hook's stored drawdown and decay through its configuration paths.

- **If the return value is controllable, how is it used and how can it go wrong?** N/A — this internal helper has no return value. A zero drawdown returns before all pool queries and watermark updates; otherwise, the calls below determine which bins are checked and updated.
- **What happens if it reverts, reenters, or does other unusual control flow?** Any uncaught error in a nested query or calculation propagates through `afterSwap`, reverting all watermark changes in that hook call. In the supplied pool's extension path, the failure also reverts the surrounding swap.

- `IMetricOmmPool(pool_).getImmutables()`

  - **What is controllable?** The `pool_` is the initialized hook caller. Its implementation supplies the returned struct. In the supplied factory/core path, the permissionless pool creator chooses `params.minimalOperationalLiquidity`; the factory requires it to be nonzero and at least the precision floor derived from the initial per-share token amounts and then the pool stores it immutably. The creator thus selects this metric denominator floor within those bounds; a later swap caller cannot replace it.
  - **If the return value is controllable, how is it used and how can it go wrong?** The `minimalOperationalLiquidity` floors the share denominator in `_metrics`, with zero replaced by one. A dishonest initialized implementation can distort per-share values by returning a false floor. The extension does not obtain this value from a separately authenticated source.
  - **What happens if it reverts, reenters, or does other unusual control flow?** The view call uses `STATICCALL`. A revert or malformed struct ABI response propagates. Callbacks inherit static restrictions and cannot modify state. This query occurs before any watermark write.

- `SlotOLibrary.unpack(packedSlotOInitial)`

  - **What is controllable?** The pool supplies both packed words; the library and bit layout are fixed. In the supplied core flow, these words are derived from the initial swap state and `SlotOLibrary.loadPackedSlotO()` after the swap, so they are not raw user-selected inputs to the normal swap entry point.
  - **If the return value is controllable, how is it used and how can it go wrong?** The two returned `curBinIdx` fields define the inclusive low-to-high bin range. The hook does not compare them to an independent record of the swap. A dishonest initialized caller can select the range, and a wide range increases gas consumption.
  - **What happens if it reverts, reenters, or does other unusual control flow?** These internal pure decoders only unpack bits and do not call other contracts. Array allocation or later traversal of a large derived range can exhaust gas and revert the hook.

- `PoolStateLibrary._multipleBinStates(pool_, binIdxs)`
    - **What is controllable?** The range fixes `binIdxs`; `_calculateBinStateSlot` computes the slot keys using the bin-state mapping base slot. The initialized pool controls the external implementation and response. Here the library's `pool` argument receives the hook's `pool_`.
    - **If the return value is controllable, how is it used and how can it go wrong?** The returned `bytes32[]` becomes `states`. Its indexed words are decoded as scaled balances for nonempty bins and drive both metrics. Forged words can therefore distort watermark checks for that caller's key. Extra array elements are unused; a missing element reverts if execution indexes it.
    - **What happens if it reverts, reenters, or does other unusual control flow?** The external view call is a `STATICCALL`; a callee revert, invalid ABI response, or exhausted gas propagates. A callback cannot write state under static restrictions. Both batch reads finish before the bin loop makes any watermark changes.
- `PoolStateLibrary._multipleBinTotalShares(pool_, binIdxs)`
    - **What is controllable?** The range fixes the bin indexes, and `_calculateBinTotalSharesSlot` fixes their share-slot keys. The same initialized pool supplies the returned storage words.
    - **If the return value is controllable, how is it used and how can it go wrong?** The resulting `shares` array supplies each bin's share count. A zero count deletes that bin's watermark and skips its state-word access; a nonzero count affects the metric denominator. A dishonest return can thus reset or distort that caller's protection. Missing indexed elements revert; surplus elements are ignored.
    - **What happens if it reverts, reenters, or does other unusual control flow?** The external read uses `STATICCALL`, so state-changing reentrancy is prohibited throughout its call tree. A failed read or ABI decode aborts the hook; an array-bound error later in the loop also rolls back earlier bin writes.
- `PoolStateLibrary._decodeBinTotalShares(shares[i])`
    - **What is controllable?** The pool-provided response words are the inputs; the decode code is fixed.
    - **If the return value is controllable, how is it used and how can it go wrong?** The first returns the full `uint256` share word. If it is nonzero, the second extracts the two `uint104` scaled balances; the fee fields it also returns are unused here. The decoders do not authenticate the data.
    - **What happens if it reverts, reenters, or does other unusual control flow?** These are internal pure operations with no callbacks. Solidity array indexing can revert before a decoder is entered when a response is too short; the error propagates.
- `_metrics(t0, t1, totalShares, minShares, bidPriceX64, askPriceX64)`
    - **What is controllable?** Balances and shares come from the pool, `minShares` comes from its immutables response, and bid/ask come from the hook

arguments.

- **If the return value is controllable, how is it used and how can it go wrong?** The helper uses `max(totalShares, minShares)` as the denominator, computes `tOps = floor(t0 * 1e18 / shares)` and `t1ps` likewise, and then computes `tOps + floor(t1ps * 2^64 / bid)` and `floor(tOps * ask / 2^64) + t1ps`. The `_clampMetric` caps each result at `uint104.max`. These rounded values feed the watermark comparison; price or balance distortion can change whether a trade passes. The per-share rounding occurs before price conversion.
- **What happens if it reverts, reenters, or does other unusual control flow?** All helpers are internal pure code. The share denominator is positive on this path, and the fixed denominator is 2^{64}; a zero bid causes division-by-zero panic even when `t1ps` is zero. Under the declared input widths and positive share floor, the nonzero-denominator metric results fit in `uint256`; saturation is applied afterward. Any error propagates without a callback.

- `_checkAndUpdateWatermarks(...)`
  - **What is controllable?** The live metrics and direction derive from the caller's data; stored watermarks, the configured drawdown floor, and decay rate derive from prior successful hooks or admin configuration. Elapsed time uses execution environment time.
  - **If the return value is controllable, how is it used and how can it go wrong?** `_decayed` supplies the linearly reduced watermark, returning the old value for zero rate, zero elapsed time, or zero watermark — and zero once `rate * dt >= 1e18`. The `_applyWatermark` returns the greater watermark and a strict-below-floor breach flag. The caller checks token0 breaches only for `zeroForOne`, and token1 breaches only for the opposite direction; it then stores both resulting watermarks and the truncated `uint40` decay timestamp. A zero watermark seeds from the current metric and provides no prior-loss protection.
  - **What happens if it reverts, reenters, or does other unusual control flow?** `OracleStopLossTriggered` propagates on the applicable directional breach. Multiplication of rate by elapsed time can panic at arithmetic limits. Timestamp subtraction cannot underflow for reachable stored state under nondecreasing chain time; each writer stores a truncated current timestamp, and deletion stores zero. These are internal calls with no external reentrancy. A later bin failure reverts updates and deletions already made for earlier bins.

Function: `function cancelOracleStopLossDecay(address pool_) external onlyPoolAdmin(pool_)`

This function deletes a pending proposal and emits a cancellation event. The active configuration remains unchanged.

Inputs

- `pool_`
  - **Control**: Pool administrator.
  - **Constraints**: N/A.
  - **Impact**: Selects the pool-specific storage key to modify.
- `msg.sender`
  - **Control**: Transaction caller.
  - **Constraints**: Must equal `FACTORY.poolAdmin(pool_)`.
  - **Impact**: Determines authority to change the pool-specific policy.

Branches and code coverage

Intended branches

- Cancels an existing pending proposal regardless of whether its delay has elapsed.
  - ☐ Test coverage
- After confirming the administrator, verifies initialization through `_initializedSchedule`. Clears only this proposal type's pending value and execution time and emits a `Cancelled` event, preserving the applied configuration and other pending proposal types.
  - ☐ Test coverage

Negative branches

- Reverts with `OnlyPoolAdmin` if the caller is not the current pool administrator reported by the factory.
  - ☐ Test coverage
- Reverts with `OracleStopLossNotInitialized` for an uninitialized pool.
  - ☐ Test coverage
- Reverts with the corresponding `NoPending` error if the proposal is absent or already deleted.
  - ☐ Test coverage
- If the `FACTORY.poolAdmin(pool_)` external query in `onlyPoolAdmin` reverts, the original error propagates. An invalid returned address ABI value causes an ABI decoding revert.
  - ☐ Test coverage

Function call analysis

- `_initializedSchedule(pool_)`

- **What is controllable?** The authorized caller selects `pool_`; initialization state and the schedule reside in this extension. Only the immutable factory can initialize that key.
- **If the return value is controllable, how is it used and how can it go wrong?** The helper returns a storage reference to `poolStopLossSchedule[pool_]` after validating initialization. The caller cannot supply an arbitrary storage reference; subsequent schedule reads and writes use the selected pool's slot.
- **What happens if it reverts, reenters, or does other unusual control flow?** `OracleStopLossNotInitialized(pool_)` propagates before any schedule mutation. Both helpers execute internally and do not transfer control to another contract.

Function: `function cancelOracleStopLossDrawdown(address pool_) external onlyPoolAdmin(pool_)`

This function deletes a pending proposal and emits a cancellation event. The active configuration remains unchanged.

Inputs

- `pool_`
  - **Control**: Pool administrator.
  - **Constraints**: N/A.
  - **Impact**: Selects the pool-specific storage key to modify.
- `msg.sender`
  - **Control**: Transaction caller.
  - **Constraints**: Must equal `FACTORY.poolAdmin(pool_)`.
  - **Impact**: Determines authority to change the pool-specific policy.

Branches and code coverage

Intended branches

- Cancels an existing pending proposal regardless of whether its delay has elapsed.

  ☑ Test coverage
- After confirming the administrator, verifies initialization through `_initializedSchedule`. Clears only this proposal type's pending value and execution time and emits a `Cancelled` event, preserving the applied configuration and other pending proposal types.

  ☐ Test coverage

Negative branches

- Reverts with `OnlyPoolAdmin` if the caller is not the current pool administrator reported by the factory.
  - ☐ Test coverage
- Reverts with `OracleStopLossNotInitialized` for an uninitialized pool.
  - ☐ Test coverage
- Reverts with the corresponding `NoPending` error if the proposal is absent or already deleted.
  - ☐ Test coverage
- If the `FACTORY.poolAdmin(pool_)` external query in `onlyPoolAdmin` reverts, the original error propagates. An invalid returned address ABI value causes an ABI decoding revert.
  - ☐ Test coverage

Function call analysis

- `_initializedSchedule(pool_)`
  - **What is controllable?** The authorized caller selects `pool_`; initialization state and the schedule reside in this extension. Only the immutable factory can initialize that key.
  - **If the return value is controllable, how is it used and how can it go wrong?** The helper returns a storage reference to `poolStopLossSchedule[pool_]` after validating initialization. The caller cannot supply an arbitrary storage reference; subsequent schedule reads and writes use the selected pool's slot.
  - **What happens if it reverts, reenters, or does other unusual control flow?** `OracleStopLossNotInitialized(pool_)` propagates before any schedule mutation. Both helpers execute internally and do not transfer control to another contract.

Function: `function cancelOracleStopLossHighWatermarks(address pool_) external onlyPoolAdmin(pool_)`

This function deletes a pending proposal and emits a cancellation event. The active configuration remains unchanged.

Inputs

- `pool_`
  - **Control**: Pool administrator.

    - **Constraints**: N/A.
    - **Impact**: Selects the pool-specific storage key to modify.
- `msg.sender`
    - **Control**: Transaction caller.
    - **Constraints**: Must equal `FACTORY.poolAdmin(pool_)`.
    - **Impact**: Determines authority to change the pool-specific policy.

Branches and code coverage

Intended branches

- Cancels an existing pending proposal.
    - ☑ Test coverage
- The cancellation path does not read the delay expiry time, so it can cancel before expiry.
    - ☐ Test coverage
- Deletes the pool's entire `pendingHighWatermark` struct without changing the current configuration or watermarks then emits `OracleStopLossHighWatermarkCancelled`. It does not recheck initialization or query delay expiry.
    - ☐ Test coverage

Negative branches

- Reverts with `OnlyPoolAdmin` if the caller is not the current pool administrator reported by the factory.
    - ☐ Test coverage
- Reverts with the corresponding NoPending error if the proposal is absent or already deleted.
    - ☐ Test coverage
- If the `FACTORY.poolAdmin(pool_)` external query in `onlyPoolAdmin` reverts, the original error propagates. An invalid returned address ABI value causes an ABI decoding revert.
    - ☐ Test coverage

Function: `function cancelOracleStopLossTimelock(address pool_) external onlyPoolAdmin(pool_)`

This function deletes a pending proposal and emits a cancellation event. The active configuration remains unchanged.

Inputs

- `pool_`
  - **Control**: Pool administrator.
  - **Constraints**: N/A.
  - **Impact**: Selects the pool-specific storage key to modify.
- `msg.sender`
  - **Control**: Transaction caller.
  - **Constraints**: Must equal `FACTORY.poolAdmin(pool_)`.
  - **Impact**: Determines authority to change the pool-specific policy.

Branches and code coverage

Intended branches

- Cancels an existing pending proposal regardless of whether its delay has elapsed.
  - ☐ Test coverage
- After confirming the administrator, verifies initialization through `_initializedSchedule`. Clears only this proposal type's pending value and execution time and emits a `Cancelled` event, preserving the applied configuration and other pending proposal types.
  - ☐ Test coverage

Negative branches

- Reverts with `OnlyPoolAdmin` if the caller is not the current pool administrator reported by the factory.
  - ☐ Test coverage
- Reverts with `OracleStopLossNotInitialized` for an uninitialized pool.
  - ☐ Test coverage
- Reverts with the corresponding `NoPending` error if the proposal is absent or already deleted.
  - ☐ Test coverage
- If the `FACTORY.poolAdmin(pool_)` external query in `onlyPoolAdmin` reverts, the original error propagates. An invalid returned address ABI value causes an ABI decoding revert.
  - ☐ Test coverage

Function call analysis

- `_initializedSchedule(pool_)`

- **What is controllable?** The authorized caller selects `pool_`; initialization state and the schedule reside in this extension. Only the immutable factory can initialize that key.
- **If the return value is controllable, how is it used and how can it go wrong?** The helper returns a storage reference to `poolStopLossSchedule[pool_]` after validating initialization. The caller cannot supply an arbitrary storage reference; subsequent schedule reads and writes use the selected pool's slot.
- **What happens if it reverts, reenters, or does other unusual control flow?** `OracleStopLossNotInitialized(pool_)` propagates before any schedule mutation. Both helpers execute internally and do not transfer control to another contract.

Function: `function executeOracleStopLossDecay(address pool_) external onlyPoolAdmin(pool_)`

This function applies an elapsed proposal to the active configuration, clears the pending proposal, and emits an event.

Inputs

- `pool_`
  - **Control**: Pool administrator.
  - **Constraints**: N/A.
  - **Impact**: Selects the pool-specific storage key to modify.
- `msg.sender`
  - **Control**: Transaction caller.
  - **Constraints**: Must equal `FACTORY.poolAdmin(pool_)`.
  - **Impact**: Determines authority to change the pool-specific policy.

Branches and code coverage

Intended branches

- Updates `decayPerSecondE18` when the timestamp is at least the execution time.
  - ☑ Test coverage
- Changing decay to zero uses the new rate without locking the prior rate for past elapsed time, so recovery can stop.
  - ☑ Test coverage
- On success, clears the pending value and execution time to zero, preventing

reexecution.

- ☐ Test coverage

- After confirming the administrator, `_initializedSchedule` checks initialization and selects the pool's schedule slot. Pending existence and `_requireElapsed` must succeed in order before `decayPerSecondE18` is applied, that pending proposal is deleted, and the `Set` event is emitted. Other pending proposal types are preserved.

  - ☐ Test coverage

Negative branches

- Reverts with `OnlyPoolAdmin` if the caller is not the current pool administrator reported by the factory.

  - ☐ Test coverage

- Reverts with `OracleStopLossNotInitialized` for an uninitialized pool.

  - ☐ Test coverage

- Reverts with `OracleStopLossNoPendingDecay` if the pending execution time is zero.

  - ☐ Test coverage

- Reverts with `OracleStopLossTimelockNotElapsed` if the timestamp is earlier than the execution time.

  - ☐ Test coverage

- If the `FACTORY.poolAdmin(pool_)` external query in `onlyPoolAdmin` reverts, the original error propagates. An invalid returned address ABI value causes an ABI decoding revert.

  - ☐ Test coverage

Function call analysis

- `_initializedSchedule(pool_)`
  - **What is controllable?** The authorized caller selects `pool_`; initialization state and the schedule reside in this extension. Only the immutable factory can initialize that key.
  - **If the return value is controllable, how is it used and how can it go wrong?** The helper returns a storage reference to `poolStopLossSchedule[pool_]` after validating initialization. The caller cannot supply an arbitrary storage reference; subsequent schedule reads and writes use the selected pool's slot.
  - **What happens if it reverts, reenters, or does other unusual control flow?** `OracleStopLossNotInitialized(pool_)` propagates before any schedule mutation. Both helpers execute internally and do not transfer control to another contract.
- `_requireElapsed(sched.pendingDecayExecuteAfter)`

- **What is controllable?** The execution time was stored by an authorized proposal; this execution call supplies only the pool key, not a replacement execution time.
- **If the return value is controllable, how is it used and how can it go wrong?** N/A — the helper returns no value. Passing it allows the pending configuration to be applied and cleared.
- **What happens if it reverts, reenters, or does other unusual control flow?** `OracleStopLossTimelockNotElapsed` propagates if the stored execution time is still in the future. It is an internal check, so it cannot reenter; a failure leaves the pending item and active configuration unchanged.

Function: `function executeOracleStopLossDrawdown(address pool_) external onlyPoolAdmin(pool_)`

This function applies an elapsed proposal to the active configuration, clears the pending proposal, and emits an event.

Inputs

- `pool_`
  - **Control**: Pool administrator.
  - **Constraints**: N/A.
  - **Impact**: Selects the pool-specific storage key to modify.
- `msg.sender`
  - **Control**: Transaction caller.
  - **Constraints**: Must equal `FACTORY.poolAdmin(pool_)`.
  - **Impact**: Determines authority to change the pool-specific policy.

Branches and code coverage

Intended branches

- Updates `drawdownE6` when the timestamp is at least the execution time.
  - ☑ Test coverage
- On success, clears the pending value and execution time to zero, preventing reexecution.
  - ☐ Test coverage
- After confirming the administrator, `_initializedSchedule` checks initialization and selects the pool's schedule slot. Pending existence and `_requireElapsed` must succeed in order before `drawdownE6` is applied, that pending proposal is deleted, and

the `Set` event is emitted. Other pending proposal types are preserved.

☐ Test coverage

Negative branches

- Reverts with `OnlyPoolAdmin` if the caller is not the current pool administrator reported by the factory.

  ☐ Test coverage
- Reverts with `OracleStopLossNotInitialized` for an uninitialized pool.

  ☐ Test coverage
- Reverts with `OracleStopLossNoPendingDrawdown` if the pending execution time is zero.

  ☑ Test coverage
- Reverts with `OracleStopLossTimelockNotElapsed` if the timestamp is earlier than the execution time.

  ☑ Test coverage
- If the `FACTORY.poolAdmin(pool_)` external query in `onlyPoolAdmin` reverts, the original error propagates. An invalid returned address ABI value causes an ABI decoding revert.

  ☐ Test coverage

Function call analysis

- `_initializedSchedule(pool_)`
  - **What is controllable?** The authorized caller selects `pool_`; initialization state and the schedule reside in this extension. Only the immutable factory can initialize that key.
  - **If the return value is controllable, how is it used and how can it go wrong?** The helper returns a storage reference to `poolStopLossSchedule[pool_]` after validating initialization. The caller cannot supply an arbitrary storage reference; subsequent schedule reads and writes use the selected pool's slot.
  - **What happens if it reverts, reenters, or does other unusual control flow?** `OracleStopLossNotInitialized(pool_)` propagates before any schedule mutation. Both helpers execute internally and do not transfer control to another contract.
- `_requireElapsed(sched.pendingDrawdownExecuteAfter)`
  - **What is controllable?** The execution time was stored by an authorized proposal; this execution call supplies only the pool key, not a replacement execution time.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A — the helper returns no value. Passing it allows the pending

configuration to be applied and cleared.

- **What happens if it reverts, reenters, or does other unusual control flow?** `OracleStopLossTimelockNotElapsed` propagates if the stored execution time is still in the future. It is an internal check, so it cannot reenter; a failure leaves the pending item and active configuration unchanged.

Function: `function executeOracleStopLossHighWatermarks(address pool_) external onlyPoolAdmin(pool_)`

This function applies an elapsed proposal to the active configuration, clears the pending proposal, and emits an event.

Inputs

- `pool_`
  - **Control**: Pool administrator.
  - **Constraints**: N/A.
  - **Impact**: Selects the pool-specific storage key to modify.
- `msg.sender`
  - **Control**: Transaction caller.
  - **Constraints**: Must equal `FACTORY.poolAdmin(pool_)`.
  - **Impact**: Determines authority to change the pool-specific policy.

Branches and code coverage

Intended branches

- Overwrites both HWMs for the specified bin when the timestamp is at least the execution time.
  - ☑ Test coverage
- Resets `lastDecayTs` to the current timestamp converted to `uint40` and clears the pending proposal. The test above queries only the two HWMs with decay set to zero, so it does not directly validate the timestamp reset or pending deletion.
  - ☐ Test coverage
- Writes both watermarks and the decay timestamp for the stored pending `binIdx`, deletes the entire pool-level pending proposal, and emits `OracleStopLossHighWatermarkUpdated`. This path does not call `_initializedSchedule` or recheck initialization.
  - ☐ Test coverage

Negative branches

- Reverts with `OnlyPoolAdmin` if the caller is not the current pool administrator reported by the factory.
  - ☐ Test coverage
- Reverts with `OracleStopLossNoPendingHighWatermark` if the pending execution time is zero.
  - ☑ Test coverage
- Reverts with `OracleStopLossTimelockNotElapsed` if the timestamp is earlier than the execution time.
  - ☑ Test coverage
- If the `FACTORY.poolAdmin(pool_)` external query in `onlyPoolAdmin` reverts, the original error propagates. An invalid returned address ABI value causes an ABI decoding revert.
  - ☐ Test coverage

Function call analysis

- `_requireElapsed(pending.executeAfter)`
  - **What is controllable?** The execution time was stored by an authorized proposal; this execution call supplies only the pool key, not a replacement execution time.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A — the helper returns no value. Passing it allows the pending configuration to be applied and cleared.
  - **What happens if it reverts, reenters, or does other unusual control flow?** `OracleStopLossTimelockNotElapsed` propagates if the stored execution time is still in the future. It is an internal check, so it cannot reenter; a failure leaves the pending item and active configuration unchanged.

Function: `function executeOracleStopLossTimelock(address pool_) external onlyPoolAdmin(pool_)`

This function applies an elapsed proposal to the active configuration, clears the pending proposal, and emits an event.

Inputs

- `pool_`
  - **Control**: Pool administrator.
  - **Constraints**: N/A.

    - **Impact**: Selects the pool-specific storage key to modify.
- `msg.sender`
    - **Control**: Transaction caller.
    - **Constraints**: Must equal `FACTORY.poolAdmin(pool_)`.
    - **Impact**: Determines authority to change the pool-specific policy.

Branches and code coverage

Intended branches

- Updates the timelock when the timestamp is at least the execution time.
    - ☑ Test coverage
- A timelock-change proposal waits for the existing timelock at proposal time, not the new value.
    - ☑ Test coverage
- On success, clears the pending value and execution time to zero, preventing reexecution. The configuration-value validation test above does not directly assert deletion of the entire pending proposal.
    - ☐ Test coverage
- After confirming the administrator, `_initializedSchedule` checks initialization and selects the pool's schedule slot. Pending existence and `_requireElapsed` must succeed in order before the timelock is applied, that pending proposal is deleted, and the Set event is emitted. Other pending proposal types are preserved.
    - ☐ Test coverage

Negative branches

- Reverts with `OnlyPoolAdmin` if the caller is not the current pool administrator reported by the factory.
    - ☐ Test coverage
- Reverts with `OracleStopLossNotInitialized` for an uninitialized pool.
    - ☐ Test coverage
- Reverts with `OracleStopLossNoPendingTimelock` if the pending execution time is zero.
    - ☐ Test coverage
- Reverts with `OracleStopLossTimelockNotElapsed` if the timestamp is earlier than the execution time.
    - ☑ Test coverage
- If the `FACTORY.poolAdmin(pool_)` external query in `onlyPoolAdmin` reverts, the original error propagates. An invalid returned address ABI value causes an ABI decoding revert.

- ☐ Test coverage

Function call analysis

- `_initializedSchedule(pool_)`
  - **What is controllable?** The authorized caller selects `pool_`; initialization state and the schedule reside in this extension. Only the immutable factory can initialize that key.
  - **If the return value is controllable, how is it used and how can it go wrong?** The helper returns a storage reference to `poolStopLossSchedule[pool_]` after validating initialization. The caller cannot supply an arbitrary storage reference; subsequent schedule reads and writes use the selected pool's slot.
  - **What happens if it reverts, reenters, or does other unusual control flow?** `OracleStopLossNotInitialized(pool_)` propagates before any schedule mutation. Both helpers execute internally and do not transfer control to another contract.
- `_requireElapsed(sched.pendingTimelockExecuteAfter)`
  - **What is controllable?** The execution time was stored by an authorized proposal; this execution call supplies only the pool key, not a replacement execution time.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A — the helper returns no value. Passing it allows the pending configuration to be applied and cleared.
  - **What happens if it reverts, reenters, or does other unusual control flow?** `OracleStopLossTimelockNotElapsed` propagates if the stored execution time is still in the future. It is an internal check, so it cannot reenter; a failure leaves the pending item and active configuration unchanged.

Function: `function initialize(address pool, bytes calldata data) external override(BaseMetricExtension, IOracleValueStopLossExtension) onlyFactory returns (bytes4)`

This function ABI-decodes the factory's pool-specific drawdown, decay, and timelock settings; stores them once; emits three configuration events; and returns the initialization selector.

Inputs

- `pool`
  - **Control**: Factory.
  - **Constraints**: Must not yet be initialized in this extension.

    - **Impact**: The configuration key to initialize.
- `data`
    - **Control**: Factory.
    - **Constraints**: The `abi.decode` must succeed using `(uint32 drawdownE6, uint64 decayPerSecondE18, uint40 timelock)`.
    - **Impact**: The decoded values set the pool-specific loss limit, decay, and delay configuration:
        - `drawdownE6`: A `uint32` no greater than `1e6`. Zero disables the loss check, and `1e6` makes the drawdown threshold zero.
        - `decayPerSecondE18`: A `uint64` no greater than `1e18`. Zero disables decay.
        - `timelock`: A `uint40` with no minimum. Zero allows a proposal to execute in the same timestamp.
- `msg.sender`
    - **Control**: Caller.
    - **Constraints**: Only `FACTORY` is allowed.
    - **Impact**: Authorizes configuration initialization.

Branches and code coverage

Intended branches

- Stores a valid configuration with `initialized = true`.
    - ☑ Test coverage
- Allows trailing bytes when the ABI decoding required by the function is valid.
    - ☐ Test coverage
- After confirming factory authority and no duplicate initialization, decodes `data` as `(uint32,uint64,uint40)`. If the drawdown and then decay range checks pass, it stores the four configuration fields; emits `DrawdownSet`, `DecaySet`, and `TimelockSet` in order; and returns the initialization selector.
    - ☐ Test coverage

Negative branches

- Reverts with `OracleStopLossAlreadyInitialized` for an already initialized pool.
    - ☑ Test coverage
- A non-factory call reverts with `OnlyFactory`.
    - ☐ Test coverage
- Short or invalidly encoded `data` reverts during ABI decoding.

- ☐ Test coverage

- Reverts with `OracleStopLossDrawdownTooLarge` in `_validateDrawdown` if `drawdown > 1e6`.
  - ☐ Test coverage
- After passing the drawdown check, reverts with `OracleStopLossDecayTooLarge` in `_validateDecay` if `decay > 1e18`.
  - ☐ Test coverage

Function call analysis

- `abi.decode(data, (uint32, uint64, uint40))`
  - **What is controllable?** Only the immutable factory may call directly. In the supplied factory flow, a permissionless pool creator selects `params.extensionInitData[i]`, which the factory forwards unchanged for the newly deployed pool. The creator therefore controls the initial drawdown, decay, and timelock within ABI and extension validation bounds. Neither operation invokes the factory or the selected pool.
  - **If the return value is controllable, how is it used and how can it go wrong?** The decoded drawdown, decay rate, and timelock populate the configuration after validation. A successful decode alone does not enforce the drawdown or decay bounds.
  - **What happens if it reverts, reenters, or does other unusual control flow?** The `OnlyFactory` rejects other callers; an already initialized key is rejected before decoding. Malformed ABI data reverts. These operations introduce no external call or reentrancy boundary.
- `_validateDrawdown(drawdownE6)`
  - **What is controllable?** The creator selects the drawdown in the initialization bytes forwarded by the factory.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A — no return value. The helper enforces `drawdownE6 <= E6 (1e6)` before the initialization struct is stored.
  - **What happens if it reverts, reenters, or does other unusual control flow?** `OracleStopLossDrawdownTooLarge` propagates and leaves the key uninitialized. This is an internal pure check with no callback.
- `_validateDecay(decayPerSecondE18)`
  - **What is controllable?** The creator selects the decay rate in the initialization bytes forwarded by the factory.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A — no return value. The helper enforces `decayPerSecondE18 <= E18 (1e18)` before storage.
  - **What happens if it reverts, reenters, or does other unusual control flow?**

`OracleStopLossDecayTooLarge` propagates and rolls back initialization. This internal pure check cannot reenter.

Function: `function proposeOracleStopLossDecay(address pool_, uint256 newDecayPerSecondE18) external onlyPoolAdmin(pool_)`

This function calculates an execution time from the current timelock, stores the proposal, and emits an event. It does not immediately change the active configuration.

Inputs

- `pool_`
  - **Control**: Pool administrator.
  - **Constraints**: N/A.
  - **Impact**: Selects the pool-specific storage key to modify.
- `newDecayPerSecondE18`
  - **Control**: Pool administrator.
  - **Constraints**: Must be at most 1e18.
  - **Impact**: Sets the `decayPerSecondE18` value to apply later.
- `msg.sender`
  - **Control**: Transaction caller.
  - **Constraints**: Must equal `FACTORY.poolAdmin(pool_)`.
  - **Impact**: Determines authority to change the pool-specific policy.

Branches and code coverage

Intended branches

- Stores the new value and an execution time equal to the current configuration's timelock added to `block.timestamp` as `uint256` in the pending proposal.
  - ☑ Test coverage
- Overwrites an existing proposal rather than rejecting it and restarts the delay.
  - ☐ Test coverage
- When the current timelock is zero, execution can occur in the same timestamp.
  - ☑ Test coverage
- When the timestamp and timelock are both zero, storing the proposal succeeds but `executeAfter = 0` collides with the no-proposal sentinel.
  - ☐ Test coverage

- After confirming the administrator, first checks the proposal-value limit and then updates the pending proposal and emits the corresponding `Proposed` event only if initialization and the execution time's `uint40` range checks pass. The active configuration remains unchanged until execution succeeds.

  - ☐ Test coverage

Negative branches

- Reverts with `OnlyPoolAdmin` if the caller is not the current pool administrator reported by the factory.

  - ☐ Test coverage
- Reverts with `OracleStopLossNotInitialized` for an uninitialized pool.

  - ☐ Test coverage
- Reverts with `OracleStopLossTimelockOverflow` if adding the timestamp and current timelock as `uint256` succeeds but the result exceeds `uint40.max`.

  - ☐ Test coverage
- Reverts with `OracleStopLossDecayTooLarge` for a value greater than 1e18.

  - ☑ Test coverage
- If the `FACTORY.poolAdmin(pool_)` external query in `onlyPoolAdmin` reverts, the original error propagates. An invalid returned address ABI value causes an ABI decoding revert.

  - ☐ Test coverage
- Reverts with an arithmetic panic in the extreme environment where adding the timestamp and current timelock as `uint256` overflows.

  - ☐ Test coverage

Function call analysis

- `_validateDecay(newDecayPerSecondE18)`

  - **What is controllable?** The `newDecayPerSecondE18` is supplied by the authorized caller.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A — this pure helper returns no value. It accepts values at most `E18` `(1e18)`, allowing the subsequent narrowing cast and configuration storage within that bound.
  - **What happens if it reverts, reenters, or does other unusual control flow?** An excessive value reverts with `OracleStopLossDecayTooLarge` before the proposal is written. The helper has no external calls or reentrancy boundary.
- `_initializedSchedule(pool_)`

  - **What is controllable?** The authorized caller selects `pool_`; initialization state

and the schedule reside in this extension. Only the immutable factory can initialize that key.

- **If the return value is controllable, how is it used and how can it go wrong?** The helper returns a storage reference to `poolStopLossSchedule[pool_]` after validating initialization. The caller cannot supply an arbitrary storage reference; subsequent schedule reads and writes use the selected pool's slot.
- **What happens if it reverts, reenters, or does other unusual control flow?** `OracleStopLossNotInitialized(pool_)` propagates before any schedule mutation. Both helpers execute internally and do not transfer control to another contract.

- `_afterTimelock(pool_)`
  - **What is controllable?** The caller selects the pool; the delay is its stored `timelock`, initially factory-configured and subsequently changed through the admin's delayed execution path. The current timestamp is execution environment state.
  - **If the return value is controllable, how is it used and how can it go wrong?** The returned `uint40` becomes the pending proposal's execution time. It is computed from the currently active delay, so a proposed new delay cannot shorten its own waiting period. A zero active delay permits execution at the current timestamp.
  - **What happens if it reverts, reenters, or does other unusual control flow?** The helper checks that the sum fits in `uint40` and reverts with `OracleStopLossTimelockOverflow` before storing the proposal if it does not. Checked addition can also revert at the `uint256` arithmetic limit. It performs no external calls.

Function: `function proposeOracleStopLossDrawdown(address pool_, uint256 newMaxDrawdownE6) external onlyPoolAdmin(pool_)`

This function calculates an execution time from the current timelock, stores the proposal, and emits an event. It does not immediately change the active configuration.

Inputs

- `pool_`
  - **Control**: Pool administrator.
  - **Constraints**: N/A.
  - **Impact**: Selects the pool-specific storage key to modify.
- `newMaxDrawdownE6`
  - **Control**: Pool administrator.
  - **Constraints**: Must be at most 1e6.

    - **Impact**: Sets the `drawdownE6` value to apply later.
- `msg.sender`
    - **Control**: Transaction caller.
    - **Constraints**: Must equal `FACTORY.poolAdmin(pool_)`.
    - **Impact**: Determines authority to change the pool-specific policy.

Branches and code coverage

Intended branches

- Stores the new value and an execution time equal to the current configuration's timelock added to `block.timestamp` as `uint256` in the pending proposal.
    - ☑ Test coverage
- Overwrites an existing proposal rather than rejecting it and restarts the delay.
    - ☐ Test coverage
- When the current timelock is zero, execution can occur in the same timestamp. In a hypothetical scenario where the timestamp is zero, `executeAfter = 0` collides with the no-proposal sentinel.
    - ☐ Test coverage
- After confirming the administrator, first checks the proposal-value limit and then updates the pending proposal and emits the corresponding `Proposed` event only if initialization and the execution time's `uint40` range checks pass. The active configuration remains unchanged until execution succeeds.
    - ☐ Test coverage

Negative branches

- Reverts with `OnlyPoolAdmin` if the caller is not the current pool administrator reported by the factory.
    - ☑ Test coverage
- Reverts with `OracleStopLossNotInitialized` for an uninitialized pool.
    - ☐ Test coverage
- Reverts with `OracleStopLossTimelockOverflow` if adding the timestamp and current timelock as `uint256` succeeds but the result exceeds `uint40.max`.
    - ☑ Test coverage
- Reverts with an arithmetic panic in the extreme environment where adding the timestamp and current timelock as `uint256` overflows.
    - ☐ Test coverage
- Reverts with `OracleStopLossDrawdownTooLarge` for a value greater than 1e6.

- [x] Test coverage

- If the `FACTORY.poolAdmin(pool_)` external query in `onlyPoolAdmin` reverts, the original error propagates. An invalid returned address ABI value causes an ABI decoding revert.
  - [ ] Test coverage

Function call analysis

- `_validateDrawdown(newMaxDrawdownE6)`
  - **What is controllable?** The `newMaxDrawdownE6` is supplied by the authorized caller.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A — this pure helper returns no value. It accepts values at most `E6 (1e6)`, allowing the subsequent narrowing cast and configuration storage within that bound.
  - **What happens if it reverts, reenters, or does other unusual control flow?** An excessive value reverts with `OracleStopLossDrawdownTooLarge` before the proposal is written. The helper has no external calls or reentrancy boundary.
- `_initializedSchedule(pool_)`
  - **What is controllable?** The authorized caller selects `pool_`; initialization state and the schedule reside in this extension. Only the immutable factory can initialize that key.
  - **If the return value is controllable, how is it used and how can it go wrong?** The helper returns a storage reference to `poolStopLossSchedule[pool_]` after validating initialization. The caller cannot supply an arbitrary storage reference; subsequent schedule reads and writes use the selected pool's slot.
  - **What happens if it reverts, reenters, or does other unusual control flow?** `OracleStopLossNotInitialized(pool_)` propagates before any schedule mutation. Both helpers execute internally and do not transfer control to another contract.
- `_afterTimelock(pool_)`
  - **What is controllable?** The caller selects the pool; the delay is its stored `timelock`, initially factory configured and subsequently changed through the admin's delayed execution path. The current timestamp is execution environment state.
  - **If the return value is controllable, how is it used and how can it go wrong?** The returned `uint40` becomes the pending proposal's execution time. It is computed from the currently active delay, so a proposed new delay cannot shorten its own waiting period. A zero active delay permits execution at the current timestamp.
  - **What happens if it reverts, reenters, or does other unusual control flow?**

The helper checks that the sum fits in `uint40` and reverts with `OracleStopLossTimelockOverflow` before storing the proposal if it does not. Checked addition can also revert at the `uint256` arithmetic limit. It performs no external calls.

Function: `function proposeOracleStopLossHighWatermarks(address pool_, int16 binIdx, uint104 newHwmToken0, uint104 newHwmToken1) external onlyPoolAdmin(pool_)`

This function calculates an execution time from the current timelock, stores the proposal, and emits an event. It does not immediately change the active configuration.

Inputs

- `pool_`
  - **Control**: Pool administrator.
  - **Constraints**: N/A.
  - **Impact**: Selects the pool-specific storage key to modify.
- `binIdx`
  - **Control**: Pool administrator.
  - **Constraints**: N/A.
  - **Impact**: Selects the bin whose watermarks are applied after delayed execution.
- `newHwmToken0`
  - **Control**: Pool administrator.
  - **Constraints**: N/A.
  - **Impact**: Sets the token0-denominated watermark applied after delayed execution.
- `newHwmToken1`
  - **Control**: Pool administrator.
  - **Constraints**: N/A.
  - **Impact**: Sets the token1-denominated watermark applied after delayed execution.
- `msg.sender`
  - **Control**: Transaction caller.
  - **Constraints**: Must equal `FACTORY.poolAdmin(pool_)`.
  - **Impact**: Determines authority to change the pool-specific policy.

Branches and code coverage

Intended branches

- Stores the new values and an execution time equal to the current configuration's timelock added to `block.timestamp` as `uint256` in the pending proposal.
  - ☑ Test coverage
- Overwrites an existing proposal rather than rejecting it and restarts the delay. Only one proposal exists per pool, so a proposal for another bin is also replaced.
  - ☐ Test coverage
- When the current timelock is zero, execution can occur in the same timestamp. In a hypothetical scenario where the timestamp is zero, `executeAfter = 0` collides with the no-proposal sentinel.
  - ☐ Test coverage
- After confirming the administrator, updates the pending proposal and emits the corresponding `Proposed` event only if initialization and the execution time's `uint40` range checks pass. The active configuration remains unchanged until execution succeeds.
  - ☐ Test coverage

Negative branches

- Reverts with `OnlyPoolAdmin` if the caller is not the current pool administrator reported by the factory.
  - ☑ Test coverage
- Reverts with `OracleStopLossNotInitialized` for an uninitialized pool.
  - ☐ Test coverage
- Reverts with `OracleStopLossTimelockOverflow` if adding the timestamp and current timelock as `uint256` succeeds but the result exceeds `uint40.max`.
  - ☐ Test coverage
- If the `FACTORY.poolAdmin(pool_)` external query in `onlyPoolAdmin` reverts, the original error propagates. An invalid returned address ABI value causes an ABI decoding revert.
  - ☐ Test coverage
- Reverts with an arithmetic panic in the extreme environment where adding the timestamp and current timelock as `uint256` overflows.
  - ☐ Test coverage

Function call analysis

- `_requireInitialized(pool_)`

- **What is controllable?** The `pool_` is selected by the authorized caller; `oracleStopLossConfig[pool_].initialized` is written by factory-only initialization.
- **If the return value is controllable, how is it used and how can it go wrong?** N/A — the helper has no return value. Its success gates creation of the pending watermark proposal for that pool.
- **What happens if it reverts, reenters, or does other unusual control flow?** `OracleStopLossNotInitialized(pool_)` aborts the proposal. This is an internal storage check with no external call or reentrancy boundary.

- `_afterTimelock(pool_)`
  - **What is controllable?** The caller selects the pool; the delay is its stored `timelock`, initially factory configured and subsequently changed through the admin's delayed execution path. The current timestamp is execution environment state.
  - **If the return value is controllable, how is it used and how can it go wrong?** The returned `uint40` becomes the pending proposal's execution time. It is computed from the currently active delay, so a proposed new delay cannot shorten its own waiting period. A zero active delay permits execution at the current timestamp.
  - **What happens if it reverts, reenters, or does other unusual control flow?** The helper checks that the sum fits in `uint40` and reverts with `OracleStopLossTimelockOverflow` before storing the proposal if it does not. Checked addition can also revert at the `uint256` arithmetic limit. It performs no external calls.

Function: `function proposeOracleStopLossTimelock(address pool_, uint40 newTimelock) external onlyPoolAdmin(pool_)`

This function calculates an execution time from the current timelock, stores the proposal, and emits an event. It does not immediately change the active configuration.

Inputs

- `pool_`
  - **Control**: Pool administrator.
  - **Constraints**: N/A.
  - **Impact**: Selects the pool-specific storage key to modify.
- `newTimelock`
  - **Control**: Pool administrator.
  - **Constraints**: N/A.
  - **Impact**: Sets the timelock value to apply later.

- `msg.sender`
  - **Control**: Transaction caller.
  - **Constraints**: Must equal `FACTORY.poolAdmin(pool_)`.
  - **Impact**: Determines authority to change the pool-specific policy.

Branches and code coverage

Intended branches

- Stores the new value and an execution time equal to the current configuration's timelock added to `block.timestamp` as `uint256` in the pending proposal.
  - ☑ Test coverage
- Overwrites an existing proposal rather than rejecting it and restarts the delay.
  - ☐ Test coverage
- When the current timelock is zero, execution can occur in the same timestamp. In a hypothetical scenario where the timestamp is zero, `executeAfter = 0` collides with the no-proposal sentinel.
  - ☐ Test coverage
- After confirming the administrator, updates the pending proposal and emits the corresponding `Proposed` event only if initialization and the execution time's `uint40` range checks pass. The active configuration remains unchanged until execution succeeds.
  - ☐ Test coverage

Negative branches

- Reverts with `OnlyPoolAdmin` if the caller is not the current pool administrator reported by the factory.
  - ☐ Test coverage
- Reverts with `OracleStopLossNotInitialized` for an uninitialized pool.
  - ☐ Test coverage
- Reverts with `OracleStopLossTimelockOverflow` if adding the timestamp and current timelock as `uint256` succeeds but the result exceeds `uint40.max`.
  - ☐ Test coverage
- If the `FACTORY.poolAdmin(pool_)` external query in `onlyPoolAdmin` reverts, the original error propagates. An invalid returned address ABI value causes an ABI decoding revert.
  - ☐ Test coverage
- Reverts with an arithmetic panic in the extreme environment where adding the timestamp and current timelock as `uint256` overflows.

- ☐ Test coverage

Function call analysis

- `_initializedSchedule(pool_)`
  - **What is controllable?** The authorized caller selects `pool_`; initialization state and the schedule reside in this extension. Only the immutable factory can initialize that key.
  - **If the return value is controllable, how is it used and how can it go wrong?** The helper returns a storage reference to `poolStopLossSchedule[pool_]` after validating initialization. The caller cannot supply an arbitrary storage reference; subsequent schedule reads and writes use the selected pool's slot.
  - **What happens if it reverts, reenters, or does other unusual control flow?** `OracleStopLossNotInitialized(pool_)` propagates before any schedule mutation. Both helpers execute internally and do not transfer control to another contract.
- `_afterTimelock(pool_)`
  - **What is controllable?** The caller selects the pool; the delay is its stored `timelock`, initially factory configured and subsequently changed through the admin's delayed execution path. The current timestamp is execution environment state.
  - **If the return value is controllable, how is it used and how can it go wrong?** The returned `uint40` becomes the pending proposal's execution time. It is computed from the currently active delay, so a proposed new delay cannot shorten its own waiting period. A zero active delay permits execution at the current timestamp.
  - **What happens if it reverts, reenters, or does other unusual control flow?** The helper checks that the sum fits in `uint40` and reverts with `OracleStopLossTimelockOverflow` before storing the proposal if it does not. Checked addition can also revert at the `uint256` arithmetic limit. It performs no external calls.

5.7. PeripheryPayments.sol

Function: `receive() external payable`

This function accepts ETH with empty calldata only when it comes from the configured WETH address.

Inputs

- `msg.value`
  - **Control**: ETH sender.
  - **Constraints**: N/A.
  - **Impact**: Adds received ETH to the contract balance.
- `msg.sender`
  - **Control**: ETH sender.
  - **Constraints**: Must equal the configured WETH address.
  - **Impact**: Restricts which sender can transfer ETH this way.

Branches and code coverage

Intended branches

- Accepts ETH sent from the WETH address.
  - ☑ Test coverage
- Accepts a zero-value transfer when `msg.sender` is WETH.
  - ☐ Test coverage

Negative branches

- Reverts with `NotWETH` when another address sends ETH with empty calldata.
  - ☑ Test coverage
- Reverts with `NotWETH` when `msg.sender` is not WETH, even for zero value.
  - ☐ Test coverage

Function: `function refundETH() external payable override`

This function returns the contract's entire ETH balance to the current caller.

Inputs

- `msg.value`
  - **Control**: Arbitrary.
  - **Constraints**: N/A.
  - **Impact**: Adds to the existing ETH balance and is included in this refund amount.
- `msg.sender`

- **Control**: Arbitrary.
- **Constraints**: The ETH transfer must succeed.
- **Impact**: Becomes the recipient of the contract's entire ETH balance.

Branches and code coverage

Intended branches

- Transfers the entire positive ETH balance read at call time to `msg.sender`.
  - ☑ Test coverage
- Succeeds without an external call when the balance is zero.
  - ☐ Test coverage
- ETH transfer makes a low-level call to the recipient with empty calldata and the specified amount. Recipient code may execute; on success, the function completes without validating return data.
  - ☐ Test coverage
- The transfer amount is the balance read before the external call. If the recipient returns ETH during the callback, for example through the router's empty payable `multicall`, the final balance can be positive; this function does not recheck the balance after transfer.
  - ☐ Test coverage

Negative branches

- The entire call reverts with `ETHTransferFailed` if the caller rejects ETH.
  - ☐ Test coverage
- If the low-level ETH call reverts, replaces the recipient's original error data with `ETHTransferFailed` and reverts.
  - ☐ Test coverage

Function call analysis

- `_transferETH(msg.sender, balance)`
  - **What is controllable?** Any caller can trigger a refund to itself. The value is the entire live contract ETH balance, including this call's `msg.value` and unrelated leftovers; there is no per-user balance accounting. The caller controls its receiving contract's behavior.
  - **If the return value is controllable, how is it used and how can it go wrong?** The low-level CALL returns `(bool ok, bytes return data)`; `_transferETH` keeps only `ok`, ignores return data, and accepts any successful recipient

response; `ok == false` becomes `ETHTransferFailed()`, rather than bubbling the recipient's reason. A zero balance skips the call.

- **What happens if it reverts, reenters, or does other unusual control flow?** The recipient receives ETH before its code runs and can reenter public functions with the remaining gas; there is no refund reentrancy guard. A nested refund sees the reduced live balance, not the original snapshot. An uncaught transfer failure reverts this invocation and its nested effects; earlier batch effects also roll back if the enclosing multicall propagates it.

Function: `function sweepToken(address token, uint256 amountMinimum, address recipient) public payable override`

This function sends the contract's entire balance of the specified token to `recipient`.

Inputs

- `token`
  - **Control**: Any caller.
  - **Constraints**: Must support `balanceOf` and SafeERC20 transfer.
  - **Impact**: Selects the asset to withdraw.
- `amountMinimum`
  - **Control**: Any caller.
  - **Constraints**: The actual token balance must be at least this value.
  - **Impact**: Sets a withdrawal floor, not a transfer cap.
- `recipient`
  - **Control**: Arbitrary.
  - **Constraints**: The token transfer must succeed.
  - **Impact**: Specifies the address that receives the contract's entire balance of the specified token.
- `msg.value`
  - **Control**: Arbitrary.
  - **Constraints**: N/A.
  - **Impact**: Increases the contract's ETH balance but is not included in the token transfer amount and is not forwarded to the token call.

Branches and code coverage

Intended branches

- Safely transfers the entire positive token balance.

☑ Test coverage

- If the balance and minimum are both zero, skips the transfer.

☐ Test coverage

- SafeERC20 treats a successful low-level `transfer` call with a returned word of one as success. If return data is absent, it treats the call as successful only when the token address has code.

☐ Test coverage

Negative branches

- Reverts with `InsufficientToken` if the balance is less than `amountMinimum`.

☑ Test coverage

- The entire call reverts if the `balanceOf` call or return ABI decoding fails or if `safeTransfer` reverts.

☐ Test coverage

- The original `transfer` revert propagates unchanged. If the call succeeds but returns a disallowed value such as `false`, reverts with `SafeERC20FailedOperation`.

☐ Test coverage

Function call analysis

- `IERC20(token).balanceOf(address(this))`
  - **What is controllable?** The caller selects the token and minimum; the token controls the returned balance and may differ from a standard ERC-20.
  - **If the return value is controllable, how is it used and how can it go wrong?** This STATICCALL result is ABI-decoded as `uint256`, checked against `amountMinimum`, and, if positive, used as the whole transfer amount. It is neither a caller-owned deposit balance nor independently verified. A dishonest balance can trigger an incorrect transfer amount or cause `InsufficientToken`; zero skips transfer when the minimum permits it.
  - **What happens if it reverts, reenters, or does other unusual control flow?** A revert or malformed return aborts the call. STATICCALL propagates a static context to reentrant reads and prohibits state changes throughout that subcall, so the query itself cannot perform a successful state-changing reentry.
- `IERC20(token).safeTransfer(recipient, balanceToken)`
  - **What is controllable?** The caller selects token and recipient, and the preceding token balance reply sets the amount. The token receives a zero-ETH CALL from this contract with the ERC-20 transfer selector.
  - **If the return value is controllable, how is it used and how can it go wrong?** The wrapper returns no amount. Its assembly accepts a successful CALL

whose checked return word is `1` or a successful empty response from an address with code; it does not perform a full ABI length check. A false/nonaccepted response or an empty response from a no-code address causes `SafeERC20FailedOperation`. A success indication does not prove the recipient received the stated amount, and there is no posttransfer balance check.

- **What happens if it reverts, reenters, or does other unusual control flow?** Token revert bytes are bubbled by the `bubble = true` path. The token can reenter this contract or invoke recipient hooks during transfer; `sweepToken` has no reentrancy guard or ownership accounting. Any uncaught failure rolls back this sweep's transfer and nested effects.

Function: `function unwrapWETH9(uint256 amountMinimum, address recipient) public payable override`

This function unwraps the contract's entire WETH balance into ETH and sends it to `recipient`. The `amountMinimum` is a balance floor, not the withdrawal amount.

Inputs

- `amountMinimum`
  - **Control**: Any caller.
  - **Constraints**: Must not exceed the WETH balance floor.
  - **Impact**: Sets the minimum WETH balance required to execute the unwrap.
- `recipient`
  - **Control**: Any caller.
  - **Constraints**: The ETH transfer must succeed.
  - **Impact**: Receives the ETH produced by unwrapping the entire WETH balance.
- `msg.value`
  - **Control**: Arbitrary.
  - **Constraints**: N/A.
  - **Impact**: Increases the contract's ETH balance but is not included in the unwrap amount or the initial ETH-transfer amount.

Branches and code coverage

Intended branches

- Withdraws and transfers the entire positive WETH balance as ETH.

☑ Test coverage

- If the WETH balance and minimum are both zero, skips the external withdrawal and transfer.

☐ Test coverage

- ETH transfer makes a low-level call to the recipient with empty calldata and the specified amount. Recipient code may execute; on success, the function completes without validating return data.

☐ Test coverage

- If the recipient calls `refundETH` again during the ETH callback, remaining ETH can also be sent to that recipient. Therefore, `msg.value` or preexisting ETH is not guaranteed to remain in the final balance.

☐ Test coverage

Negative branches

- Reverts with `InsufficientWETH` if the balance is below the minimum.

☑ Test coverage

- The entire call reverts if `balanceOf` or `withdraw` reverts or if the recipient rejects ETH.

☐ Test coverage

- If the low-level ETH call reverts, replaces the recipient's original error data with `ETHTransferFailed` and reverts.

☐ Test coverage

Function call analysis

- `IERC20(WETH).balanceOf(address(this))`
  - **What is controllable?** The caller controls `amountMinimum`; the deployment-selected WETH implementation controls the balance response.
  - **If the return value is controllable, how is it used and how can it go wrong?** The STATICCALL result is decoded as `uint256`, compared to the minimum, and becomes both the withdrawal amount and later ETH payment. It represents all contract WETH, without caller-specific accounting. A dishonest result can skip withdrawal, cause `InsufficientWETH`, request an impossible withdrawal, or cause existing ETH to be paid against a fabricated balance.
  - **What happens if it reverts, reenters, or does other unusual control flow?** Revert or malformed return aborts execution. The read cannot mutate state, including through nested calls, because it executes in a static context.
- `IWETH9(WETH).withdraw(balanceWETH)`
  - **What is controllable?** The target is immutable WETH; its prior balance reply

chooses the amount. The caller cannot change the target in this invocation.

- **If the return value is controllable, how is it used and how can it go wrong?** The ordinary external CALL forwards no ETH and has no return value to inspect. The helper trusts WETH to burn the stated amount and deliver matching ETH; it never checks the ETH balance increase. A normally returning dishonest implementation could leave the next payment using preexisting ETH or failing for lack of funds.
- **What happens if it reverts, reenters, or does other unusual control flow?** Standard withdrawal calls this contract's `receive()`, which accepts ETH from WETH. The WETH implementation can execute other reentrant calls because this helper has no guard. Its revert propagates; a later recipient failure also rolls back the withdrawal and all nested effects in this invocation.

- `_transferETH(recipient, balanceWETH)`
  - **What is controllable?** The caller chooses the recipient; the WETH balance snapshot chooses the value. The recipient controls fallback/receive behavior.
  - **If the return value is controllable, how is it used and how can it go wrong?** Only the low-level success boolean is consumed. Return data is discarded, and a false boolean becomes `ETHTransferFailed()` without the recipient's original reason. A successful call is enough; the recipient need not return anything.
  - **What happens if it reverts, reenters, or does other unusual control flow?** The CALL forwards available gas and may reenter public functions after the ETH transfer reduces the contract balance. There is no reentrancy guard or caller-specific withdrawal entitlement. Failure reverts this unwrap, including WETH withdrawal and any nested effects.

5.8. PriceVelocityGuardExtension.sol

Function: `function beforeSwap( address, address, bool, int128, uint128, uint256, uint128, uint128, uint128 referencePriceX64, bytes calldata ) external override returns (bytes4)`

This function checks the squared relative deviation from the pool-specific reference price. The permitted square is `maxChangePerBlockE18² * (1 + block.number - anchorBlock)`. On success, records the last observed price and block.

Inputs

- `referencePriceX64`
  - **Control**: Caller (the pool in the normal path).
  - **Constraints**: N/A.

    - **Impact**: Used as the mid reference price to initialize the anchor or compare the price change and then stored as the final observed price.
- `msg.sender`
    - **Control**: Caller.
    - **Constraints**: N/A.
    - **Impact**: Selects the caller's own `priceVelocityState`, so it cannot directly change another pool's state.

Branches and code coverage

Intended branches

- If the anchor is zero, initializes it with the input price and current block then returns without a deviation check.
    - ☐ Test coverage
- When `block.number` is in the `uint64` range, the anchor is positive, and the administrator has not changed the anchor or limit, a subsequent swap in the same block uses the same anchor and allowance. The reference price does not change the anchor on every swap.
    - ☑ Test coverage
- A swap with a price inside the permitted range in a new block can proceed.
    - ☑ Test coverage
- At that point, the anchor moves to the previously observed price and block. The test does not query the stored fields directly, and its price would also pass with the existing anchor, so it does not independently validate this update.
    - ☐ Test coverage
- If `maxChange` is zero, skips the deviation check. It initializes a zero anchor; after moving to a new block with a positive anchor, it moves the anchor to the last observation and updates the last observed price and block.
    - ☐ Test coverage
- The same price or a price exactly at the permitted boundary passes; integer division rounds the relative deviation down.
    - ☐ Test coverage
- Calculates absolute deviations for both price increases and decreases; either passes when within the permitted range.
    - ☐ Test coverage
- A non-pool address can initialize and update state under its own key but cannot change another pool's key.
    - ☐ Test coverage

- When the initial anchor is positive, moves the anchor price and block to the last observation only if `block.number > lastObservedBlock`. It then calculates `blockDiff = block.number - anchorBlock`. If the moved anchor remains positive and the administrator does not change the configuration, another call in the same `uint64`-range block preserves the anchor and `blockDiff`. If the stored anchor is zero, the next call uses the initialization path again.

  ☐ Test coverage

- If `block.number` exceeds `uint64.max`, stored `lastObservedBlock` and `anchorBlock` are truncated. A subsequent call in the same block still sees `block.number > lastObservedBlock` and moves the anchor again; the usual same-block invariant no longer holds.

  ☐ Test coverage

- If `maxChange` is positive, calculates `delta = abs(midPrice - anchorMid)`, `changeE18 = floor(delta × 1e18 / anchorMid)`, `actualSq = changeE18²`, and `allowedSq = maxChange² × (1 + blockDiff)` in that order. If `actualSq ≤ allowedSq`, updates the last observed price and block and returns the `beforeSwap` selector.

  ☐ Test coverage

Negative branches

- Reverts with `PriceVelocityExceeded` when the squared deviation exceeds the permitted square.

  ☑ Test coverage

- Reverts with an arithmetic-overflow panic if a small anchor and large input make `changeE18²` exceed `uint256`.

  ☐ Test coverage

- After a zero price is allowed with a positive anchor and execution moves to the next block, the anchor can become zero. A positive limit then reverts because the denominator is zero.

  ☐ Test coverage

- Reverts through checked arithmetic if an extreme block number makes `1 + blockDiff` or `maxChange² × (1 + blockDiff)` exceed `uint256`.

  ☐ Test coverage

- If a deviation check or arithmetic operation fails after moving the anchor in a new block, this call's anchor price and block changes are also reverted, and the entire hook call reverts.

  ☐ Test coverage

Function: `function setAnchorMidPrice(address pool_, uint128 newAnchorMidPriceX64) external onlyPoolAdmin(pool_)`

This function resets the pool-specific anchor and last observed price to the same value and records the current block number in both block fields.

Inputs

- `pool_`
  - **Control**: Pool administrator.
  - **Constraints**: N/A.
  - **Impact**: Selects the pool-specific storage key to modify.
- `newAnchorMidPriceX64`
  - **Control**: Pool administrator.
  - **Constraints**: N/A.
  - **Impact**: Reconfigures the price-check baseline.
- `msg.sender`
  - **Control**: Transaction caller.
  - **Constraints**: Must equal `FACTORY.poolAdmin(pool_)`.
  - **Impact**: Determines authority to change the pool-specific policy.

Branches and code coverage

Intended branches

- Updates both price fields and both block fields together then emits an event.
  - ☐ Test coverage
- A zero price causes the next hook call to take the initial-anchor branch again.
  - ☐ Test coverage
- If `block.number` exceeds the `uint64` range, its high bits are truncated when stored.
  - ☐ Test coverage
- The function body runs only when `msg.sender` equals the result of the `FACTORY.poolAdmin` external query. It stores the configured value and emits the corresponding configuration event; it performs no other internal calculation or external call.
  - ☐ Test coverage

Negative branches

- A non-administrator caller reverts with `OnlyPoolAdmin(pool_, msg.sender,`

`poolAdmin`).

- ☐ Test coverage

- If the `FACTORY.poolAdmin(pool_)` external query in `onlyPoolAdmin` reverts, the original error propagates. An invalid returned address ABI value causes an ABI decoding revert.
  - ☐ Test coverage

Function: `function setMaxChangePerBlock(address pool_, uint64 newMaxPctChangePerBlockE18) external onlyPoolAdmin(pool_)`

This function immediately overwrites the pool-specific price-change limit and emits an event.

Inputs

- `pool_`
  - **Control**: Pool administrator.
  - **Constraints**: N/A.
  - **Impact**: Selects the pool-specific storage key to modify.
- `newMaxPctChangePerBlockE18`
  - **Control**: Pool administrator.
  - **Constraints**: N/A.
  - **Impact**: Changes the permitted squared deviation for subsequent `beforeSwap` calls.
- `msg.sender`
  - **Control**: Transaction caller.
  - **Constraints**: Must equal `FACTORY.poolAdmin(pool_)`.
  - **Impact**: Determines authority to change the pool-specific policy.

Branches and code coverage

Intended branches

- Stores the value without updating the anchor or observed-block fields.
  - ☐ Test coverage
- Configuring zero skips the deviation check, but price-observation state continues to update.
  - ☐ Test coverage
- Values greater than 1e18 are allowed and can permit large price changes.

- ☐ Test coverage

- The function body runs only when `msg.sender` equals the result of the `FACTORY.poolAdmin` external query. It stores the configured value and emits the corresponding configuration event; it performs no other internal calculation or external call.

  - ☐ Test coverage

Negative branches

- A non-administrator caller reverts with `OnlyPoolAdmin(pool_, msg.sender, poolAdmin)`.

  - ☐ Test coverage
- If the `FACTORY.poolAdmin(pool_)` external query in `onlyPoolAdmin` reverts, the original error propagates. An invalid returned address ABI value causes an ABI decoding revert.

  - ☐ Test coverage

5.9. SelfPermit.sol

Function: `function selfPermit(address token, uint256 value, uint256 deadline, uint8 v, bytes32 r, bytes32 s) public payable`

This function calls an EIP-2612 permit to request that the current caller approve this contract to use `value` tokens.

Inputs

- `token`
  - **Control**: Any caller.
  - **Constraints**: Must support the EIP-2612 `IERC20Permit` call.
  - **Impact**: Selects the external token whose approval is updated.
- `value`
  - **Control**: Arbitrary.
  - **Constraints**: N/A.
  - **Impact**: Sets the approved token amount.
- `deadline`
  - **Control**: Arbitrary.
  - **Constraints**: Expiry validation is delegated to the token that executes the permit.
  - **Impact**: Sets the signed approval's validity period.
- `v`

    - **Control**: Arbitrary.
    - **Constraints**: The permit must validate a signature whose owner is `msg.sender` and spender is `address(this)`; validation is delegated to the token.
    - **Impact**: Forwards as the `v` signature argument to the token permit call.
- `r`
    - **Control**: Arbitrary.
    - **Constraints**: The permit must validate a signature whose owner is `msg.sender` and spender is `address(this)`; validation is delegated to the token.
    - **Impact**: Forwards as the `r` signature argument to the token permit call.
- `s`
    - **Control**: Arbitrary.
    - **Constraints**: The permit must validate a signature whose owner is `msg.sender` and spender is `address(this)`; validation is delegated to the token.
    - **Impact**: Forwards as the `s` signature argument to the token permit call.
- `msg.value`
    - **Control**: Arbitrary.
    - **Constraints**: N/A.
    - **Impact**: Increases the contract's ETH balance but is not forwarded in the external permit call.
- `msg.sender`
    - **Control**: Arbitrary.
    - **Constraints**: N/A.
    - **Impact**: Grants this contract permission to use the current caller's tokens.

Branches and code coverage

Intended branches

- In a `multicall` that first executes a valid permit, the following swap has sufficient approval and returns output tokens.
    - ☑ Test coverage
- Always calls `permit`, even when the allowance is already sufficient.
    - ☐ Test coverage
- Does not explicitly forbid `value = 0`; if the token permits it, a zero approval can be requested.
    - ☐ Test coverage

- The permit call uses the original `msg.sender` as owner and this contract as spender, forwarding `value`, `deadline`, and the signature unchanged. This helper records no local approval state and completes if the external token call returns normally.

  - ☐ Test coverage

Negative branches

- Reverts with the original error if `permit` reverts due to an invalid signature, consumed nonce, expired approval, or unsupported token.

  - ☐ Test coverage
- Reverts if the same signature is first submitted to `token.permit` to consume its nonce and then submitted through `selfPermit`.

  - ☐ Test coverage
- Reverts if the permit target has no code or the external permit call reverts.

  - ☐ Test coverage

Function call analysis

- `IERC20Permit(token).permit(msg.sender, address(this), value, deadline, v, r, s)`

  - **What is controllable?** The caller selects token, value, deadline, and signature. The owner is the current `msg.sender`, and the spender is this contract; a self-delegatecall preserves the original batch caller.
  - **If the return value is controllable, how is it used and how can it go wrong?** The ordinary external CALL forwards no ETH and declares no return value. Normal return only means the call completed: no allowance is read afterward, and a token can return normally without establishing the expected approval. Signature, nonce, expiry, and approval semantics belong to the selected token.
  - **What happens if it reverts, reenters, or does other unusual control flow?** The high-level no-return call rejects a no-code target and propagates token reverts. The token can call back into inherited router or payment functions; these helpers have no guard, although they maintain no local permit state. A reentrant call has the token as sender, not the original signer. An uncaught error rolls back this invocation and, when bubbled through multicall, earlier batch effects.

Function: `function selfPermitAllowed(address token, uint256 nonce, uint256 expiry, uint8 v, bytes32 r, bytes32 s) public payable`

This function calls a DAI-style permit with `allowed = true` to request that the current caller allow this contract to use tokens.

Inputs

- `token`
  - **Control**: Any caller.
  - **Constraints**: Must support the DAI-style `IERC20PermitAllowed` call.
  - **Impact**: Selects the external token whose approval is updated.
- `nonce`
  - **Control**: Arbitrary.
  - **Constraints**: Validity during permit execution is checked by the external token.
  - **Impact**: Sets the nonce forwarded to the DAI-style permit.
- `expiry`
  - **Control**: Arbitrary.
  - **Constraints**: Expiry conditions are checked by the external token that executes the permit.
  - **Impact**: Sets the DAI-style permit validity period.
- `v`
  - **Control**: Arbitrary.
  - **Constraints**: The permit must validate a signature whose owner is `msg.sender` and spender is `address(this)`; validation is delegated to the token.
  - **Impact**: Forwards as the `v` signature argument to the token permit call.
- `r`
  - **Control**: Arbitrary.
  - **Constraints**: The permit must validate a signature whose owner is `msg.sender` and spender is `address(this)`; validation is delegated to the token.
  - **Impact**: Forwards as the `r` signature argument to the token permit call.
- `s`
  - **Control**: Arbitrary.
  - **Constraints**: The permit must validate a signature whose owner is `msg.sender` and spender is `address(this)`; validation is delegated to the token.
  - **Impact**: Forwards as the `s` signature argument to the token permit call.
- `msg.value`
  - **Control**: Arbitrary.
  - **Constraints**: N/A.
  - **Impact**: Increases the contract's ETH balance but is not forwarded in the external permit call.

- `msg.sender`
    - **Control**: Arbitrary.
    - **Constraints**: N/A.
    - **Impact**: Grants this contract permission to use the current caller's tokens.

Branches and code coverage

Intended branches

- Calls the external permit every time; it does not skip the call when allowance is already sufficient.
    - ☐ Test coverage
- The approval amount for `allowed = true` and special treatment of `expiry = 0` are token implementation details; this helper does not validate whether approval is unlimited.
    - ☐ Test coverage
- With a valid DAI-style signature, the allowance for `owner = caller` and `spender = router` becomes `uint256.max`.
    - ☐ Test coverage
- The permit call uses the original `msg.sender` as owner and this contract as spender; forwards the nonce, expiry, and signature unchanged; and fixes `allowed = true`. This helper records no local approval state and completes if the external token call returns normally.
    - ☐ Test coverage

Negative branches

- Reverts with the original error if `permit` reverts due to an invalid signature, consumed nonce, expired approval, or unsupported token.
    - ☐ Test coverage
- Reverts if the permit target has no code or the external permit call reverts.
    - ☐ Test coverage

Function call analysis

- `IERC20PermitAllowed(token).permit(msg.sender, address(this), nonce, expiry, true, v, r, s)`
    - **What is controllable?** The caller selects token, nonce, expiry, and signature. The holder is the current `msg.sender`, the spender is this contract, and `allowed` is fixed to `true`. A self-delegatecall preserves the original batch

caller.

- **If the return value is controllable, how is it used and how can it go wrong?** The ordinary external CALL forwards no ETH and declares no return value. Normal return only means the call completed; no allowance is read afterward, and a token can return normally without establishing the expected approval. Signature, nonce, expiry, and approval semantics belong to the selected token.
- **What happens if it reverts, reenters, or does other unusual control flow?** The high-level no-return call rejects a no-code target and propagates token reverts. The token can call back into inherited router or payment functions; these helpers have no guard, although they maintain no local permit state. A reentrant call has the token as sender, not the original signer. An uncaught error rolls back this invocation and, when bubbled through multicall, earlier batch effects.

Function: `function selfPermitAllowedIfNecessary(address token, uint256 nonce, uint256 expiry, uint8 v, bytes32 r, bytes32 s) external payable`

This function checks the current allowance first and calls a DAI-style permit with `allowed = true` only when needed to request that the caller allow this contract to use tokens.

Inputs

- `token`
  - **Control**: Any caller.
  - **Constraints**: Must support DAI-style `IERC20PermitAllowed` when the current allowance is below the required maximum.
  - **Impact**: Selects the external token whose approval is updated.
- `nonce`
  - **Control**: Arbitrary.
  - **Constraints**: Validated by the token when the permit executes.
  - **Impact**: Sets the nonce forwarded to the DAI-style permit.
- `expiry`
  - **Control**: Arbitrary.
  - **Constraints**: Validated by the token when the permit executes.
  - **Impact**: Sets the DAI-style permit validity period.
- `v`
  - **Control**: Arbitrary.
  - **Constraints**: Must form a signature the token validates for `msg.sender` and `address(this)` when the permit executes.

- **Impact**: Forwards as the `v` signature argument to the token permit call.

- `r`
  - **Control**: Arbitrary.
  - **Constraints**: Must form a signature the token validates for `msg.sender` and `address(this)` when the permit executes.
  - **Impact**: Forwards as the `r` signature argument to the token permit call.
- `s`
  - **Control**: Arbitrary.
  - **Constraints**: Must form a signature the token validates for `msg.sender` and `address(this)` when the permit executes.
  - **Impact**: Forwards as the `s` signature argument to the token permit call.
- `msg.value`
  - **Control**: Arbitrary.
  - **Constraints**: N/A.
  - **Impact**: Increases the contract's ETH balance but is not forwarded in the external permit call.
- `msg.sender`
  - **Control**: Arbitrary.
  - **Constraints**: N/A.
  - **Impact**: Grants this contract permission to use the caller's tokens and is also used as the allowance-query owner.

Branches and code coverage

Intended branches

- Calls `permit` when `allowance < type(uint256).max`.
  - ☐ Test coverage
- Skips the call without validating the signature or expiry when `allowance ≥ type(uint256).max`.
  - ☐ Test coverage
- The approval amount for `allowed = true` and special treatment of `expiry = 0` are token implementation details; this helper does not validate whether approval is unlimited.
  - ☐ Test coverage
- When allowance is already `uint256.max`, succeeds by skipping the permit call even if the provided signature has a consumed nonce.
  - ☐ Test coverage

- The permit call uses the original `msg.sender` as owner and this contract as spender; forwards the nonce, expiry, and signature unchanged; and fixes `allowed = true`. This helper records no local approval state and completes if the external token call returns normally.

  - ☐ Test coverage

Negative branches

- When insufficient allowance triggers a permit call, reverts with the original error for an invalid signature, consumed nonce, expired approval, or unsupported token.

  - ☐ Test coverage
- Reverts before permit execution if the allowance query reverts or returns malformed data.

  - ☐ Test coverage
- Reverts if the permit target has no code or the external permit call reverts.

  - ☐ Test coverage

Function call analysis

- `IERC20(token).allowance(msg.sender, address(this))`

  - **What is controllable?** The caller selects token; the token controls its reported allowance. The owner is the current caller, and the spender is this contract.
  - **If the return value is controllable, how is it used and how can it go wrong?** The STATICCALL result is decoded as `uint256`. A result below `type(uint256).max` triggers the internal public-function call; otherwise, the signature and permit fields are never validated. An inflated allowance can skip a needed permit and make a later spend fail; an understated allowance can unnecessarily trigger a consumed or invalid signature. No approval guarantee is established by this read alone.
  - **What happens if it reverts, reenters, or does other unusual control flow?** The allowance query's revert or malformed return aborts execution; its static context prevents state-changing reentry. The internal helper call preserves `msg.sender`, `msg.value`, and this contract's context rather than creating a new message call. Its external permit boundary is described below.
- `IERC20PermitAllowed(token).permit(msg.sender, address(this), nonce, expiry, true, v, r, s)`

  - **What is controllable?** The caller selects token, nonce, expiry, and signature. The holder is the current `msg.sender`, the spender is this contract, and `allowed` is fixed to `true`. A self-delegatecall preserves the original batch caller.
  - **If the return value is controllable, how is it used and how can it go wrong?**

The ordinary external CALL forwards no ETH and declares no return value. Normal return only means the call completed; no allowance is read afterward, and a token can return normally without establishing the expected approval. Signature, nonce, expiry, and approval semantics belong to the selected token.

- **What happens if it reverts, reenters, or does other unusual control flow?** The high-level no-return call rejects a no-code target and propagates token reverts. The token can call back into inherited router or payment functions; these helpers have no guard, although they maintain no local permit state. A reentrant call has the token as sender, not the original signer. An uncaught error rolls back this invocation and, when bubbled through multicall, earlier batch effects.

Function: `function selfPermitIfNecessary(address token, uint256 value, uint256 deadline, uint8 v, bytes32 r, bytes32 s) external payable`

This function checks the current allowance first and calls an EIP-2612 permit only when needed to request that the caller approve this contract to use `value` tokens.

Inputs

- `token`
  - **Control**: Any caller.
  - **Constraints**: Must support EIP-2612 `IERC20Permit` when the current allowance is below `value`.
  - **Impact**: Selects the external token whose approval is updated.
- `value`
  - **Control**: Arbitrary.
  - **Constraints**: N/A.
  - **Impact**: Sets the approved token amount. A permit executes only when the current allowance is below this value.
- `deadline`
  - **Control**: Arbitrary.
  - **Constraints**: Validated by the token when the permit executes.
  - **Impact**: Sets the signed approval's validity period.
- `v`
  - **Control**: Arbitrary.
  - **Constraints**: Must form a signature the token validates for `msg.sender` and `address(this)` when the permit executes.
  - **Impact**: Forwards as the `v` signature argument to the token permit call.

- `r`
  - **Control**: Arbitrary.
  - **Constraints**: Must form a signature the token validates for `msg.sender` and `address(this)` when the permit executes.
  - **Impact**: Forwards as the `r` signature argument to the token permit call.
- `s`
  - **Control**: Arbitrary.
  - **Constraints**: Must form a signature the token validates for `msg.sender` and `address(this)` when the permit executes.
  - **Impact**: Forwards as the `s` signature argument to the token permit call.
- `msg.value`
  - **Control**: Arbitrary.
  - **Constraints**: N/A.
  - **Impact**: Increases the contract's ETH balance but is not forwarded in the external permit call.
- `msg.sender`
  - **Control**: Arbitrary.
  - **Constraints**: N/A.
  - **Impact**: Grants this contract permission to use the caller's tokens and is also used as the allowance-query owner.

Branches and code coverage

Intended branches

- Calls `permit` when `allowance < value`.
  - ☑ Test coverage
- Skips the call without validating the signature or expiry when `allowance ≥ value`.
  - ☑ Test coverage
- Does not explicitly forbid `value = 0`. Because `allowance < 0` is impossible in the if-necessary path, it always skips.
  - ☐ Test coverage
- Succeeds by skipping the permit when allowance is sufficient, even if the signature was previously submitted and its nonce consumed.
  - ☐ Test coverage
- The permit call uses the original `msg.sender` as owner and this contract as spender, forwarding `value`, `deadline`, and the signature unchanged. This helper records no local approval state and completes if the external token call returns normally.

- ☐ Test coverage

Negative branches

- When insufficient allowance triggers a permit call, reverts with the original error for an invalid signature, consumed nonce, expired approval, or unsupported token.
  - ☐ Test coverage
- Reverts before permit execution if the allowance query reverts or returns malformed data.
  - ☐ Test coverage
- Reverts if the permit target has no code or the external permit call reverts.
  - ☐ Test coverage

Function call analysis

- `IERC20(token).allowance(msg.sender, address(this))`
  - **What is controllable?** The caller selects token and the requested value; the token controls its reported allowance. The owner is the current caller and the spender is this contract.
  - **If the return value is controllable, how is it used and how can it go wrong?** The STATICCALL result is decoded as `uint256`. A result below `value` triggers the internal public-function call; otherwise, the signature and permit fields are never validated. An inflated allowance can skip a needed permit and make a later spend fail; an understated allowance can unnecessarily trigger a consumed or invalid signature. No approval guarantee is established by this read alone.
  - **What happens if it reverts, reenters, or does other unusual control flow?** The allowance query's revert or malformed return aborts execution; its static context prevents state-changing reentry. The internal helper call preserves `msg.sender`, `msg.value`, and this contract's context rather than creating a new message call. Its external permit boundary is described below.
- `IERC20Permit(token).permit(msg.sender, address(this), value, deadline, v, r, s)`
  - **What is controllable?** The caller selects token, value, deadline, and signature. The owner is the current `msg.sender`, and the spender is this contract; a self-delegatecall preserves the original batch caller.
  - **If the return value is controllable, how is it used and how can it go wrong?** The ordinary external CALL forwards no ETH and declares no return value. Normal return only means the call completed: no allowance is read afterward, and a token can return normally without establishing the expected approval. Signature, nonce, expiry, and approval semantics belong to the selected token.

- **What happens if it reverts, reenters, or does other unusual control flow?** The high-level no-return call rejects a no-code target and propagates token reverts. The token can call back into inherited router or payment functions; these helpers have no guard, although they maintain no local permit state. A reentrant call has the token as sender, not the original signer. An uncaught error rolls back this invocation and, when bubbled through multicall, earlier batch effects.

5.10. SwapAllowlistExtension.sol

Function: `function setAllowAllSwappers(address pool_, bool allowed) external onlyPoolAdmin(pool_)`

The pool administrator immediately changes the global swapper permission flag and emits a configuration event. No timelock or additional signature is required.

Inputs

- `pool_`
  - **Control**: Pool administrator.
  - **Constraints**: N/A.
  - **Impact**: Selects the pool-specific storage key to modify.
- `allowed`
  - **Control**: Pool administrator.
  - **Constraints**: N/A.
  - **Impact**: Changes the global permission flag.
- `msg.sender`
  - **Control**: Transaction caller.
  - **Constraints**: Must equal `FACTORY.poolAdmin(pool_)`.
  - **Impact**: Determines authority to change the pool-specific policy.

Branches and code coverage

Intended branches

- Setting `true` enables global access regardless of the individual allowlist.
  - ☑ Test coverage
- Setting `false` clears only global access and preserves individual permission mappings.
  - ☐ Test coverage
- If `msg.sender` matches the pool admin returned by `FACTORY`, stores `allowed` unchanged

in the pool's global permission flag and emits the corresponding Set event with `pool_` and the configured value. It does not change other permission mappings or flags.

- ☐ Test coverage

Negative branches

- A non-administrator reverts with `OnlyPoolAdmin`.
  - ☑ Test coverage
- If the `FACTORY.poolAdmin(pool_)` external query in `onlyPoolAdmin` reverts, the original error propagates. An invalid returned address ABI value causes an ABI decoding revert.
  - ☐ Test coverage

Function: `function setAllowedToSwap(address pool_, address swapper, bool allowed) external onlyPoolAdmin(pool_)`

The pool administrator immediately updates an individual address's permission status and emits a configuration event. No timelock or additional signature is required.

Inputs

- `pool_`
  - **Control**: Pool administrator.
  - **Constraints**: N/A.
  - **Impact**: Selects the pool-specific storage key to modify.
- `swapper`
  - **Control**: Pool administrator.
  - **Constraints**: N/A.
  - **Impact**: Records the permission status of the direct pool caller that the next `beforeSwap` call will check. For swaps through a public router, this address is the router, so allowing the router lets its users pass this allowlist check if they meet other execution conditions such as funds and approvals.
- `allowed`
  - **Control**: Pool administrator.
  - **Constraints**: N/A.
  - **Impact**: Overwrites the individual permission mapping.
- `msg.sender`
  - **Control**: Transaction caller.
  - **Constraints**: Must equal `FACTORY.poolAdmin(pool_)`.
  - **Impact**: Determines authority to change the pool-specific policy.

Branches and code coverage

Intended branches

- Setting `true` allows the target address.
  - ☑ Test coverage
- Setting `false` clears only the individual permission. Effective access remains if the global permission flag is `true`.
  - ☐ Test coverage
- If `msg.sender` matches the pool admin returned by `FACTORY`, stores `allowed` unchanged in the specified address's individual permission mapping and emits the corresponding Set event with `pool_` and the configured value. It does not change other permission mappings or flags.
  - ☐ Test coverage

Negative branches

- A non-administrator reverts with `OnlyPoolAdmin`.
  - ☑ Test coverage
- If the `FACTORY.poolAdmin(pool_)` external query in `onlyPoolAdmin` reverts, the original error propagates. An invalid returned address ABI value causes an ABI decoding revert.
  - ☐ Test coverage

5.11. AnchoredPriceProvider.sol

Function: `function setConfidenceParam(uint256 newValue) external override`

This function updates the confidence parameter used by this anchored price provider.

Inputs

- `newValue`
  - **Control**: The immutable factory supplies the value; its batch setter permits the recorded provider owner or a delegated updater.
  - **Constraints**: The `newValue` must not exceed `CONFIDENCE_MAX`.
  - **Impact**: Sets the confidence configuration.

Branches and code coverage

Intended branches

- Stores a valid confidence parameter.
  - ☐ Test coverage

Negative branches

- Unauthorized access or an invalid value causes the call to revert.
  - ☐ Test coverage

Function: `function setMaxStaleness(uint256 newBase, uint256 newQuote) external`

This function configures maximum permitted staleness for base and quote sources.

Inputs

- `newBase`
  - **Control**: The immutable factory supplies the value; its batch setter permits the recorded provider owner or a delegated updater.
  - **Constraints**: Each value must not exceed `MAX_REF_STALENESS`.
  - **Impact**: Stores base and quote staleness limits.
- `newQuote`
  - **Control**: The immutable factory supplies the value; its batch setter permits the recorded provider owner or a delegated updater.
  - **Constraints**: Each value must not exceed `MAX_REF_STALENESS`.
  - **Impact**: Stores base and quote staleness limits.

Branches and code coverage

Intended branches

- Updates both staleness limits.
  - ☐ Test coverage

Negative branches

- Unauthorized access causes the call to revert.
  - ☐ Test coverage

Function call analysis

- `_applyStaleness(newBase, newQuote)`
  - **What is controllable?** The immutable `factory` supplies both per-leg bounds; its owner/updater authorization determines who may choose them. The helper reads this provider's immutable `MAX_REF_STALENESS` and rejects either argument above it. The helper target is fixed internal code.
  - **If the return value is controllable, how is it used and how can it go wrong?** No value is returned. The helper stores `baseMaxStaleness` and `quoteMaxStaleness` and emits `MaxStalenessSet`; the caller then stores `lastStalenessUpdate`. Zero imposes same-block freshness on later reads. The helper neither queries a feed nor evaluates whether a current quote is fresh.
  - **What happens if it reverts, reenters, or does other unusual control flow?** The `OnlyFactory()`, `ImmutableProvider()`, and `CooldownNotElapsed()` are checked before the helper. Its `StalenessOutOfBounds()` propagates without a catch and before either bound is stored. There are no external calls or callbacks between the two bound writes, event, and timestamp write; any enclosing revert rolls all of them back.

Function: `function setSource(address newSource) external`

This function replaces the provider's optional custom quote source; the immutable reference oracle is unchanged.

Inputs

- `newSource`
  - **Control**: The immutable factory supplies the value; its `setSource` entry point requires the recorded provider owner.
  - **Constraints**: N/A.
  - **Impact**: Stores the new source address.

Branches and code coverage

Intended branches

- Stores the supplied source address and emits `SourceSet`.
  - ☐ Test coverage

Negative branches

- A caller other than the immutable factory causes `OnlyFactory()` to revert.

- ☐ Test coverage

Function call analysis

- `_applySource(newSource)`
  - **What is controllable?** The immediate caller must equal the immutable `factory`; otherwise, `OnlyFactory()` reverts. That factory supplies `newSource`; through the anchored factory, only the recorded provider owner may choose it. The helper is fixed internal code.
  - **If the return value is controllable, how is it used and how can it go wrong?** The helper has no return value. It writes `source = newSource` and emits `SourceSet`; zero selects reference mode for later quotes. It does not validate or invoke the supplied source, so successful configuration does not establish that the source will quote successfully.
  - **What happens if it reverts, reenters, or does other unusual control flow?** The helper performs only a storage write and an event. Neither the source nor the oracle receives control, and there is no callback or reentrancy path. The factory check precedes the write; a transaction revert rolls the write and event back.

Function: `function setSpreadMultipliers(uint256 newBase, uint256 newQuote) external`

This function updates spread multipliers for base and quote prices.

Inputs

- `newBase`
  - **Control**: The immutable factory supplies the value; its batch setter permits the recorded provider owner or a delegated updater.
  - **Constraints**: Each value must not exceed `maxSpreadMult`.
  - **Impact**: Stores the two spread multipliers.
- `newQuote`
  - **Control**: The immutable factory supplies the value; its batch setter permits the recorded provider owner or a delegated updater.
  - **Constraints**: Each value must not exceed `maxSpreadMult`.
  - **Impact**: Stores the two spread multipliers.

Branches and code coverage

Intended branches

- Updates both spread multipliers.
  - ☐ Test coverage

Negative branches

- Unauthorized access or invalid values causes the call to revert.
  - ☐ Test coverage

Function call analysis

- `_applySpreadMultipliers(newBase, newQuote)`
  - **What is controllable?** The immutable `factory` supplies both per-leg multipliers; its owner/updater authorization determines who may choose them. The fixed internal helper compares each argument against this provider's immutable `maxSpreadMult`. Values below 1e18, including zero, pass that upper-bound check.
  - **If the return value is controllable, how is it used and how can it go wrong?** No value is returned. The helper stores `baseSpreadMult` and `quoteSpreadMult` and emits `SpreadMultipliersSet`; the caller then stores `lastSpreadMultUpdate`. These values scale later reference spreads. The helper does not read a price or establish that the resulting interval will pass the quote-time checks.
  - **What happens if it reverts, reenters, or does other unusual control flow?** The `OnlyFactory()`, `ImmutableProvider()`, and `CooldownNotElapsed()` are checked before the helper. Its `SpreadMultOutOfBounds()` propagates before either multiplier write. No external contract receives control between the multiplier writes and the final timestamp write; all changes and the event roll back on an enclosing revert.

5.12. AnchoredProviderFactory.sol

Function: `function addOracle(address oracle) external override onlyRole(ADMIN_ROLE)`

This function adds an oracle to the factory allowlist.

Inputs

- `oracle`

- **Control**: The `ADMIN_ROLE` holder.
- **Constraints**: The `oracle` must be nonzero and not already allowed.
- **Impact**: Enables this oracle for anchored-provider creation.

Branches and code coverage

Intended branches

- Adds a valid oracle.
  - ☐ Test coverage

Negative branches

- Unauthorized access or an invalid oracle causes the call to revert.
  - ☐ Test coverage

Function call analysis

- `_oracles.add(oracle)`
  - **What is controllable?** The authorized admin supplies the nonzero `oracle` address; the set and implementation are fixed factory storage and internal library code. This records an address without invoking it or checking its code or oracle interface.
  - **If the return value is controllable, how is it used and how can it go wrong?** The Boolean is derived from set membership: `_add` returns false if already present and otherwise appends the address and records its position. Then, `require(..., OracleAlreadyAllowed(oracle))` consumes that result. Successful insertion permits future creations to select this oracle, so its behavior is an admin-curated trust decision.
  - **What happens if it reverts, reenters, or does other unusual control flow?** A duplicate returns false and is converted into `OracleAlreadyAllowed(oracle)` by the caller. There is no external control transfer or reentrancy during insertion. A revert rolls back insertion and the later `OracleAdded` event.

Function: `function createAnchoredProvider(address oracle, bytes32 baseFeedId, bytes32 quoteFeedId, uint256 minMargin, uint256 maxRefStaleness, uint256 maxSpreadMult, bool mutableParams, int256 marginStep, address baseToken, address quoteToken, InitialConfig calldata init) external override returns (address provider)`

This function deploys and initializes an anchored provider using an approved oracle and configuration envelope.

Inputs

- `oracle`
  - **Control**: Arbitrary.
  - **Constraints**: The `oracle` must be allowed; the selected feed class must have an envelope; `minMargin`, `maxRefStaleness`, and `maxSpreadMult` must fall within that envelope; `baseToken` and `quoteToken` must be nonzero and distinct; `marginStep` must be strictly between `-BPS_BASE` and `BPS_BASE`; and each `init` staleness or multiplier must be within its corresponding bound.
  - **Impact**: Defines the new provider and its initial parameters.
- `baseFeedId`
  - **Control**: Arbitrary.
  - **Constraints**: The `oracle` must be allowed; the selected feed class must have an envelope; `minMargin`, `maxRefStaleness`, and `maxSpreadMult` must fall within that envelope; `baseToken` and `quoteToken` must be nonzero and distinct; `marginStep` must be strictly between `-BPS_BASE` and `BPS_BASE`; and each `init` staleness or multiplier must be within its corresponding bound.
  - **Impact**: Defines the new provider and its initial parameters.
- `quoteFeedId`
  - **Control**: Arbitrary.
  - **Constraints**: The `oracle` must be allowed; the selected feed class must have an envelope; `minMargin`, `maxRefStaleness`, and `maxSpreadMult` must fall within that envelope; `baseToken` and `quoteToken` must be nonzero and distinct; `marginStep` must be strictly between `-BPS_BASE` and `BPS_BASE`; and each `init` staleness or multiplier must be within its corresponding bound.
  - **Impact**: Defines the new provider and its initial parameters.
- `minMargin`
  - **Control**: Arbitrary.
  - **Constraints**: The `oracle` must be allowed; the selected feed class must have an envelope; `minMargin`, `maxRefStaleness`, and `maxSpreadMult` must fall within that envelope; `baseToken` and `quoteToken` must be nonzero and distinct; `marginStep` must be strictly between `-BPS_BASE` and `BPS_BASE`; and

each `init` staleness or multiplier must be within its corresponding bound.
    - **Impact**: Defines the new provider and its initial parameters.
- `maxRefStaleness`
    - **Control**: Arbitrary.
    - **Constraints**: The `oracle` must be allowed; the selected feed class must have an envelope; `minMargin`, `maxRefStaleness`, and `maxSpreadMult` must fall within that envelope; `baseToken` and `quoteToken` must be nonzero and distinct; `marginStep` must be strictly between `-BPS_BASE` and `BPS_BASE`; and each `init` staleness or multiplier must be within its corresponding bound.
    - **Impact**: Defines the new provider and its initial parameters.
- `maxSpreadMult`
    - **Control**: Arbitrary.
    - **Constraints**: The `oracle` must be allowed; the selected feed class must have an envelope; `minMargin`, `maxRefStaleness`, and `maxSpreadMult` must fall within that envelope; `baseToken` and `quoteToken` must be nonzero and distinct; `marginStep` must be strictly between `-BPS_BASE` and `BPS_BASE`; and each `init` staleness or multiplier must be within its corresponding bound.
    - **Impact**: Defines the new provider and its initial parameters.
- `mutableParams`
    - **Control**: Arbitrary.
    - **Constraints**: The `oracle` must be allowed; the selected feed class must have an envelope; `minMargin`, `maxRefStaleness`, and `maxSpreadMult` must fall within that envelope; `baseToken` and `quoteToken` must be nonzero and distinct; `marginStep` must be strictly between `-BPS_BASE` and `BPS_BASE`; and each `init` staleness or multiplier must be within its corresponding bound.
    - **Impact**: Defines the new provider and its initial parameters.
- `marginStep`
    - **Control**: Arbitrary.
    - **Constraints**: The `oracle` must be allowed; the selected feed class must have an envelope; `minMargin`, `maxRefStaleness`, and `maxSpreadMult` must fall within that envelope; `baseToken` and `quoteToken` must be nonzero and distinct; `marginStep` must be strictly between `-BPS_BASE` and `BPS_BASE`; and each `init` staleness or multiplier must be within its corresponding bound.
    - **Impact**: Defines the new provider and its initial parameters.
- `baseToken`
    - **Control**: Arbitrary.
    - **Constraints**: The `oracle` must be allowed; the selected feed class must have an envelope; `minMargin`, `maxRefStaleness`, and `maxSpreadMult` must fall within that envelope; `baseToken` and `quoteToken` must be nonzero and distinct; `marginStep` must be strictly between `-BPS_BASE` and `BPS_BASE`; and each `init` staleness or multiplier must be within its corresponding bound.

- **Impact**: Defines the new provider and its initial parameters.

- `quoteToken`
  - **Control**: Arbitrary.
  - **Constraints**: The `oracle` must be allowed; the selected feed class must have an envelope; `minMargin`, `maxRefStaleness`, and `maxSpreadMult` must fall within that envelope; `baseToken` and `quoteToken` must be nonzero and distinct; `marginStep` must be strictly between `-BPS_BASE` and `BPS_BASE`; and each `init` staleness or multiplier must be within its corresponding bound.
  - **Impact**: Defines the new provider and its initial parameters.
- `init`
  - **Control**: Arbitrary.
  - **Constraints**: The `oracle` must be allowed; the selected feed class must have an envelope; `minMargin`, `maxRefStaleness`, and `maxSpreadMult` must fall within that envelope; `baseToken` and `quoteToken` must be nonzero and distinct; `marginStep` must be strictly between `-BPS_BASE` and `BPS_BASE`; and each `init` staleness or multiplier must be within its corresponding bound.
  - **Impact**: Defines the new provider and its initial parameters.

Branches and code coverage

Intended branches

- Deploys a valid provider, initializes it, and records it in the factory.
  - ☐ Test coverage

Negative branches

- An unapproved oracle, missing envelope, out-of-envelope parameters, invalid constructor configuration, or deployment failure causes creation to revert.
  - ☐ Test coverage

Function call analysis

- `_oracles.contains(oracle)`
  - **What is controllable?** The permissionless caller selects `oracle`; the internal library reads membership in the admin-managed `_oracles` set. It does not call the selected oracle. The later class lookup uses `baseFeedId`, falling back to `DEFAULT_CLASS` when no explicit class is stored.
  - **If the return value is controllable, how is it used and how can it go wrong?** The library's boolean is consumed by the `OracleNotAllowed(oracle)` check. It establishes allowlist membership only. The subsequent in-line envelope checks validate `minMargin`, `maxRefStaleness`, and

`maxSpreadMult`; neither feed's existence nor a current quote is checked.

- **What happens if it reverts, reenters, or does other unusual control flow?** False membership causes `OracleNotAllowed(oracle)` before deployment. Both `EnvelopeNotFound(classId)` and `ParamsOutOfEnvelope()` also occur before deployment. No external control transfer occurs in these registry and envelope checks.

- `new AnchoredPriceProvider(address(this), oracle, baseFeedId, quoteFeedId, minMargin, maxRefStaleness, maxSpreadMult, mutableParams, marginStep, baseToken, quoteToken, init)`

  - **What is controllable?** The caller supplies every constructor argument except `_factory`, which is fixed to `address(this)`. The oracle and envelope constraints already apply. And `CREATE` executes the statically selected `AnchoredPriceProvider` constructor; caller-supplied oracle, source, and token addresses do not choose constructor code.
  - **If the return value is controllable, how is it used and how can it go wrong?** The returned contract address becomes `provider`, is registered under `msg.sender`, and is returned to the caller. The constructor stores the immutable oracle, feeds, tokens, and band configuration and then initializes the source and per-leg knobs through the helpers below. It fixes `MUTABLE_PARAMS`, `marginStep`, and the two-step factors. It checks nonzero distinct tokens and `-BPS_BASE < marginStep < BPS_BASE`. The factory address and envelope already ensure its nonzero-factory, positive-margin, maximum-staleness, and minimum-multiplier-ceiling checks pass.
  - **What happens if it reverts, reenters, or does other unusual control flow?** A constructor revert or failed creation propagates and leaves no deployment or registration. Token-pair validation uses a plain `require`; an out-of-range margin step reverts with `MarginStepOutOfBounds()`. The constructor calls only its fixed internal helpers and never invokes the oracle, feeds, tokens, or source, so there is no constructor callback into the factory.

- `_applySource(_init.source)`

  - **What is controllable?** The creator controls all five `init` fields. The three helpers execute in this order within the newly created provider. Source is stored without interface validation; each staleness bound must be at most the immutable `MAX_REF_STALENESS`, and each multiplier must be at most the immutable `maxSpreadMult`.
  - **If the return value is controllable, how is it used and how can it go wrong?** The helpers return no values. They store the source pointer and both per-leg pairs and emit their respective events. These initial settings apply to both mutable and immutable variants. The helpers do not set update timestamps or confidence; those storage fields begin at zero, and constructor initialization does not invoke the setter cooldown gates.
  - **What happens if it reverts, reenters, or does other unusual control flow?** `StalenessOutOfBounds()` or `SpreadMultOutOfBounds()` propagates through the constructor and reverts creation, including any earlier

initialization writes and events. The later margin-step check can also roll them back. No helper makes an external call, so the unvalidated source cannot reenter during deployment.

- `_providers.add(provider)`
  - **What is controllable?** The inserted address comes from the successful `CREATE`, and `creator` is the original `msg.sender`. Both sets are factory storage and all library calls are internal. The caller cannot replace the inserted address with an unrelated implementation.
  - **If the return value is controllable, how is it used and how can it go wrong?** The insertion booleans are ignored. Each helper appends an absent address and records its position; the fresh provider address is then assigned `providerOwner[provider] = creator`. This establishes recognition and owner authority after construction succeeds.
  - **What happens if it reverts, reenters, or does other unusual control flow?** There is no external control transfer during registry updates. The following event-argument getters run only after both insertions and the ownership write; any later revert rolls back those writes and the deployment.
- `p.baseToken()`
  - **What is controllable?** The target `p` is the just-deployed fixed provider implementation. Both argument-free calls are `STATICCALL`s to immutable getters; the creator influenced their values through the validated constructor token addresses.
  - **If the return value is controllable, how is it used and how can it go wrong?** The returned addresses are used only as the event's base and quote fields. They do not select another call target, affect registration, or validate token behavior. The getters return the immutable addresses without calling either token.
  - **What happens if it reverts, reenters, or does other unusual control flow?** There is no try-catch; a getter revert or return-data decoding failure would revert deployment and all registry writes in this transaction. The actual getters only return immutables and offer no callback; the static context also forbids state-changing reentrancy.

Function: `function grantUpdater(address provider, address updater) external override onlyProviderOwner(provider)`

This function grants an updater role for a provider.

Inputs

- `provider`
  - **Control**: Provider owner.

- **Constraints**: The `provider` must be tracked.
- **Impact**: Authorizes `updater` to update the provider.

- `updater`
  - **Control**: Provider owner.
  - **Constraints**: The `provider` must be tracked.
  - **Impact**: Authorizes `updater` to update the provider.

Branches and code coverage

Intended branches

- Grants the updater role.
  - ☐ Test coverage

Negative branches

- A non-owner caller causes the call to revert.
  - ☐ Test coverage

Function call analysis

- `_providers.contains(provider)`
  - **What is controllable?** The caller selects `provider`. The fixed internal EnumerableSet implementation reads that address's position in the factory's registry; it does not query the provider.
  - **If the return value is controllable, how is it used and how can it go wrong?** The returned boolean reflects registry membership and is consumed by `require(..., ProviderNotTracked())` before the operation. Membership comes from factory storage and is restricted to providers deployed by `createAnchoredProvider`; it is not a caller-supplied response.
  - **What happens if it reverts, reenters, or does other unusual control flow?** The `ProviderNotTracked()` is defensive on this owner-authorized path: the factory records owners only for its registered deployments and never removes those providers, so an authorized entry cannot fail membership under the implementation at the time of writing. The lookup is internal and has no callback or reentrancy boundary. The body then writes `isUpdater[provider][updater] = true` and emits `UpdaterGranted`; neither the provider nor the updater is invoked.

Function: `function removeEnvelope(bytes32 classId) external override onlyRole(ADMIN_ROLE)`

This function deletes configuration limits for a feed class.

Inputs

- `classId`
  - **Control**: The `ADMIN_ROLE` holder.
  - **Constraints**: The `classId` must identify an existing envelope.
  - **Impact**: Removes the class envelope.

Branches and code coverage

Intended branches

- Removes the envelope entry.
  - ☐ Test coverage

Negative branches

- Unauthorized access causes the call to revert.
  - ☐ Test coverage

Function: `function removeOracle(address oracle) external override onlyRole(ADMIN_ROLE)`

This function removes an oracle from the factory allowlist.

Inputs

- `oracle`
  - **Control**: The `ADMIN_ROLE` holder.
  - **Constraints**: The `oracle` must be currently allowed.
  - **Impact**: Revokes factory-level oracle authorization.

Branches and code coverage

Intended branches

- Removes the oracle entry.

- ☐ Test coverage

Negative branches

- Unauthorized access causes the call to revert.
  - ☐ Test coverage

Function call analysis

- `_oracles.remove(oracle)`
  - **What is controllable?** The authorized admin selects `oracle`; `_remove` reads its position in the factory's fixed allowlist. The oracle address is only a set element, not an external call target.
  - **If the return value is controllable, how is it used and how can it go wrong?** The library returns false for an absent element, which the caller converts to `OracleNotFound(oracle)`. For a present element, it swaps the last element into the vacated position when necessary, pops the array, clears the position, and returns true. Removal blocks new anchored deployments; it does not change existing providers' immutable oracle or registration.
  - **What happens if it reverts, reenters, or does other unusual control flow?** No oracle code executes, and no reentrancy point exists. The `OracleNotFound(oracle)` or a later transaction revert aborts the operation; successful removal and `OracleRemoved` are atomic.

Function: `function revokeUpdater(address provider, address updater) external override onlyProviderOwner(provider)`

This function revokes an updater role for a provider.

Inputs

- `provider`
  - **Control**: Provider owner.
  - **Constraints**: The `provider` must be tracked.
  - **Impact**: Removes update authority.
- `updater`
  - **Control**: Provider owner.
  - **Constraints**: The `provider` must be tracked.
  - **Impact**: Removes update authority.

Branches and code coverage

Intended branches

- Revokes the updater role.
  - ☐ Test coverage

Negative branches

- A non-owner caller causes the call to revert.
  - ☐ Test coverage

Function call analysis

- `_providers.contains(provider)`
  - **What is controllable?** The caller selects `provider`. The fixed internal EnumerableSet implementation reads that address's position in the factory's registry; it does not query the provider.
  - **If the return value is controllable, how is it used and how can it go wrong?** The returned boolean reflects registry membership and is consumed by `require(..., ProviderNotTracked())` before the operation. Membership comes from factory storage and is restricted to providers deployed by `createAnchoredProvider`; it is not a caller-supplied response.
  - **What happens if it reverts, reenters, or does other unusual control flow?** The `ProviderNotTracked()` is defensive on this owner-authorized path: the factory records owners only for its registered deployments and never removes those providers, so an authorized entry cannot fail membership under the implementation at the time of writing. The lookup is internal and has no callback or reentrancy boundary. The body then writes `isUpdater[provider][updater] = false` and emits `UpdaterRevoked`; neither the provider nor the updater is invoked.

Function: `function setConfidence(address[] calldata providers, uint256[] calldata values) external override`

This function sets confidence parameters for a batch of providers.

Inputs

- `providers`
  - **Control**: A caller authorized as the recorded owner or a delegated updater for every selected provider.

  - **Constraints**: The arrays must have equal length, every provider must be tracked, and each value must not exceed that provider's `CONFIDENCE_MAX`.
  - **Impact**: Applies each confidence update.
- `values`
  - **Control**: A caller authorized as the recorded owner or a delegated updater for every selected provider.
  - **Constraints**: The arrays must have equal length, every provider must be tracked, and each value must not exceed that provider's `CONFIDENCE_MAX`.
  - **Impact**: Applies each confidence update.

Branches and code coverage

Intended branches

- Updates valid provider-confidence pairs.
  - ☐ Test coverage

Negative branches

- Mismatched arrays, an untracked provider, a caller lacking owner/updater authority, an immutable provider, an out-of-range value, or an active cooldown causes the batch to revert.
  - ☐ Test coverage

Function call analysis

- `_providers.contains(providers[i])`
  - **What is controllable?** The caller chooses all array entries and their order. After the array-length checks, each iteration reads the selected provider's registry membership, owner, and `isUpdater[provider][msg.sender]` entry from factory storage. These are fixed internal checks.
  - **If the return value is controllable, how is it used and how can it go wrong?** The membership boolean gates the update with `ProviderNotTracked()` — `_requireUpdater` returns no value and accepts either the current owner or a delegated updater, otherwise reverting with `NotProviderUpdater()`. The registry contains only this factory's own anchored deployments; choosing an address cannot select arbitrary deployed code.
  - **What happens if it reverts, reenters, or does other unusual control flow?** Neither check calls the provider or updater. A failure in any iteration propagates and rolls back earlier provider updates in the same batch. Empty arrays reach no calls; mismatched lengths revert before this loop.
- `AnchoredPriceProvider(providers[i]).setConfidenceParam(values[i])`

- **What is controllable?** The authorized owner/updater supplies `values[i]` and selects a registered provider. The external `CALL` uses the factory as `msg.sender`. The fixed provider code verifies that sender, `MUTABLE_PARAMS`, `values[i] <= CONFIDENCE_MAX`, and the provider's confidence cooldown before writing.
- **If the return value is controllable, how is it used and how can it go wrong?** The setter has no return value. It stores the new confidence and `lastConfidenceUpdate = block.timestamp` and emits `ConfidenceParamSet`. These affect later quotes; the factory does not read a quote or inspect a success value.
- **What happens if it reverts, reenters, or does other unusual control flow?** The provider performs no downstream calls or callbacks. The immutable factory check necessarily passes for these tracked deployments. The `ImmutableProvider()`, `ConfidenceParamOutOfBounds()`, or `CooldownNotElapsed()` propagates without a catch, reverting all earlier batch updates and events. A repeated provider after its first successful update hits the cooldown on the second occurrence in the same transaction.

Function: `function setEnvelope(bytes32 classId, Envelope calldata envelope) external override onlyRole(ADMIN_ROLE)`

This function stores deployment limits for a feed class.

Inputs

- `classId`
  - **Control**: The `ADMIN_ROLE` holder.
  - **Constraints**: The `classId` must be nonzero; `minMarginMin` must be nonzero and no greater than `minMarginMax`; `stalenessMin <= stalenessMax <= MAX_STALENESS`; and `spreadMultMax >= BPS_BASE_U`.
  - **Impact**: Defines allowed configuration bounds for the class.
- `envelope`
  - **Control**: The `ADMIN_ROLE` holder.
  - **Constraints**: The `classId` must be nonzero; `minMarginMin` must be nonzero and no greater than `minMarginMax`; `stalenessMin <= stalenessMax <= MAX_STALENESS`; and `spreadMultMax >= BPS_BASE_U`.
  - **Impact**: Defines allowed configuration bounds for the class.

Branches and code coverage

Intended branches

- Stores a valid class envelope.
  - ☐ Test coverage

Negative branches

- Unauthorized access or invalid bounds causes the call to revert.
  - ☐ Test coverage

Function: `function setFeedClass(bytes32 feedId, bytes32 classId) external override onlyRole(ADMIN_ROLE)`

This function assigns a feed to a deployment class.

Inputs

- `feedId`
  - **Control**: The `ADMIN_ROLE` holder.
  - **Constraints**: The `classId` must be zero or identify an existing envelope.
  - **Impact**: Associates the feed with its class envelope.
- `classId`
  - **Control**: The `ADMIN_ROLE` holder.
  - **Constraints**: The `classId` must be zero or identify an existing envelope.
  - **Impact**: Associates the feed with its class envelope.

Branches and code coverage

Intended branches

- Stores the feed-class association.
  - ☐ Test coverage

Negative branches

- Unauthorized access causes the call to revert.
  - ☐ Test coverage

Function: `function setMaxStaleness(address[] calldata providers, uint256[] calldata baseStaleness, uint256[] calldata quoteStaleness) external override`

This function sets base and quote staleness limits for providers.

Inputs

- `providers`
  - **Control**: A caller authorized as the recorded owner or a delegated updater for every selected provider.
  - **Constraints**: The arrays must have equal length, every provider must be tracked, and each staleness value must not exceed that provider's `MAX_REF_STALENESS`.
  - **Impact**: Applies staleness settings.
- `baseStaleness`
  - **Control**: A caller authorized as the recorded owner or a delegated updater for every selected provider.
  - **Constraints**: The arrays must have equal length, every provider must be tracked, and each staleness value must not exceed that provider's `MAX_REF_STALENESS`.
  - **Impact**: Applies staleness settings.
- `quoteStaleness`
  - **Control**: A caller authorized as the recorded owner or a delegated updater for every selected provider.
  - **Constraints**: The arrays must have equal length, every provider must be tracked, and each staleness value must not exceed that provider's `MAX_REF_STALENESS`.
  - **Impact**: Applies staleness settings.

Branches and code coverage

Intended branches

- Updates valid staleness settings.
  - ☐ Test coverage

Negative branches

- Mismatched arrays, an untracked provider, a caller lacking owner/updater authority, an immutable provider, an out-of-range staleness bound, or an active cooldown causes the batch to revert.

- ☐ Test coverage

Function call analysis

- `_providers.contains(providers[i])`
    - **What is controllable?** The caller chooses all array entries and their order. After the array-length checks, each iteration reads the selected provider's registry membership, owner, and `isUpdater[provider][msg.sender]` entry from factory storage. These are fixed internal checks.
    - **If the return value is controllable, how is it used and how can it go wrong?** The membership boolean gates the update with `ProviderNotTracked()` — `_requireUpdater` returns no value and accepts either the current owner or a delegated updater, otherwise reverting with `NotProviderUpdater()`. The registry contains only this factory's own anchored deployments; choosing an address cannot select arbitrary deployed code.
    - **What happens if it reverts, reenters, or does other unusual control flow?** Neither check calls the provider or updater. A failure in any iteration propagates and rolls back earlier provider updates in the same batch. Empty arrays reach no calls; mismatched lengths revert before this loop.
- `AnchoredPriceProvider(providers[i]).setMaxStaleness(baseStaleness[i], quoteStaleness[i])`
    - **What is controllable?** The authorized owner/updater supplies each pair of bounds for a registered provider. This external `CALL` arrives from the immutable factory. The provider checks the factory, `MUTABLE_PARAMS`, and its staleness cooldown before the internal helper verifies both arguments against `MAX_REF_STALENESS`.
    - **If the return value is controllable, how is it used and how can it go wrong?** Neither call returns a value. The helper stores both per-leg staleness limits and emits `MaxStalenessSet`, then the setter stores `lastStalenessUpdate`. The limits constrain later price reads; no feed or oracle is queried during this update.
    - **What happens if it reverts, reenters, or does other unusual control flow?** The fixed provider/helper path has no external callbacks. The immutable factory check necessarily passes for these tracked deployments. `ImmutableProvider()`, `CooldownNotElapsed()`, or `StalenessOutOfBounds()` propagates without a catch and rolls back all earlier batch writes and events. The helper validates before its writes; a duplicate provider after a successful update encounters the cooldown in the same batch.

Function: `function setSource(address provider, address newSource) external override onlyProviderOwner(provider)`

This function updates an anchored provider's source.

Inputs

- `provider`
  - **Control**: Provider owner.
  - **Constraints**: The `provider` must be tracked.
  - **Impact**: Forwards the source update to the provider.
- `newSource`
  - **Control**: Provider owner.
  - **Constraints**: The `provider` must be tracked.
  - **Impact**: Forwards the source update to the provider.

Branches and code coverage

Intended branches

- Sets a valid source.
  - ☐ Test coverage

Negative branches

- A caller other than the recorded provider owner causes `NotProviderOwner()` to revert.
  - ☐ Test coverage

Function call analysis

- `_providers.contains(provider)`
  - **What is controllable?** The caller selects `provider`. The fixed internal EnumerableSet implementation reads that address's position in the factory's registry; it does not query the provider.
  - **If the return value is controllable, how is it used and how can it go wrong?** The returned boolean reflects registry membership and is consumed by `require(..., ProviderNotTracked())` before the operation. Membership comes from factory storage and is restricted to providers deployed by `createAnchoredProvider`; it is not a caller-supplied response.
  - **What happens if it reverts, reenters, or does other unusual control flow?** The `ProviderNotTracked()` is defensive on this owner-authorized path; the

factory records owners only for its registered deployments and never removes those providers, so an authorized entry cannot fail membership under the implementation at the time of writing. The lookup is internal and has no callback or reentrancy boundary. The following provider setter is analyzed separately.

- `AnchoredPriceProvider(provider).setSource(newSource)`
  - **What is controllable?** The owner chooses `newSource` and a provider already deployed by this factory. This is a normal external `CALL`, with the factory as the callee's `msg.sender`; the provider's immutable `factory` check therefore binds the update to this factory. The `newSource` is stored without code or interface validation.
  - **If the return value is controllable, how is it used and how can it go wrong?** Neither the external setter nor the internal helper returns a value. The `_applySource` writes the provider's source pointer and emits its event; after it returns, the factory emits `SourceSet(provider, newSource)`. Zero selects reference mode for later quotes. A successful setter does not prove the source can quote.
  - **What happens if it reverts, reenters, or does other unusual control flow?** The known provider implementation checks `OnlyFactory()` and then only writes/emits; it does not invoke the new source, oracle, or any callback. There is no catch; any callee revert propagates, and a later transaction revert also rolls back the provider write and both events.

Function: `function setSpreadMultipliers(address[] calldata providers, uint256[] calldata baseMults, uint256[] calldata quoteMults) external override`

This function sets base and quote spread multipliers for a provider batch.

Inputs

- `providers`
  - **Control**: A caller authorized as the recorded owner or a delegated updater for every selected provider.
  - **Constraints**: The arrays must have equal length, every provider must be tracked, and each multiplier must not exceed that provider's `maxSpreadMult`.
  - **Impact**: Applies spread-multiplier settings.
- `baseMults`
  - **Control**: A caller authorized as the recorded owner or a delegated updater for every selected provider.
  - **Constraints**: The arrays must have equal length, every provider must be tracked, and each multiplier must not exceed that provider's `maxSpreadMult`.

    - **Impact**: Applies spread-multiplier settings.
- `quoteMults`
    - **Control**: A caller authorized as the recorded owner or a delegated updater for every selected provider.
    - **Constraints**: The arrays must have equal length, every provider must be tracked, and each multiplier must not exceed that provider's `maxSpreadMult`.
    - **Impact**: Applies spread-multiplier settings.

Branches and code coverage

Intended branches

- Updates valid multiplier settings.
    - ☐ Test coverage

Negative branches

- Mismatched arrays, an untracked provider, a caller lacking owner/updater authority, an immutable provider, an out-of-range multiplier, or an active cooldown causes the batch to revert.
    - ☐ Test coverage

Function call analysis

- `_providers.contains(providers[i])`
    - **What is controllable?** The caller chooses all array entries and their order. After the array-length checks, each iteration reads the selected provider's registry membership, owner, and `isUpdater[provider][msg.sender]` entry from factory storage. These are fixed internal checks.
    - **If the return value is controllable, how is it used and how can it go wrong?** The membership boolean gates the update with `ProviderNotTracked()` — `_requireUpdater` returns no value and accepts either the current owner or a delegated updater, otherwise reverting with `NotProviderUpdater()`. The registry contains only this factory's own anchored deployments; choosing an address cannot select arbitrary deployed code.
    - **What happens if it reverts, reenters, or does other unusual control flow?** Neither check calls the provider or updater. A failure in any iteration propagates and rolls back earlier provider updates in the same batch. Empty arrays reach no calls; mismatched lengths revert before this loop.
- `AnchoredPriceProvider(providers[i]).setSpreadMultipliers(baseMults[i], quoteMults[i])`
    - **What is controllable?** The authorized owner/updater supplies each

multiplier pair for a registered provider. This external `CALL` arrives from the immutable factory. The provider checks the factory, `MUTABLE_PARAMS`, and its multiplier cooldown before the internal helper checks both multipliers against `maxSpreadMult`; values below 1e18 are allowed.

- **If the return value is controllable, how is it used and how can it go wrong?** Neither call returns a value. The helper stores both multipliers and emits `SpreadMultipliersSet`, and then the setter stores `lastSpreadMultUpdate`. The values shape subsequent reference intervals; no quote is fetched, and quote-time interval validity is not established here.
- **What happens if it reverts, reenters, or does other unusual control flow?** The fixed provider/helper path makes no external callbacks. The immutable factory check necessarily passes for these tracked deployments. `ImmutableProvider()`, `CooldownNotElapsed()`, or `SpreadMultOutOfBounds()` propagates without a catch and rolls back all earlier batch writes and events. Bounds are checked before storage updates; repeating a provider after its first successful update hits the cooldown.

Function: `function transferProviderOwnership(address provider, address newOwner) external override onlyProviderOwner(provider)`

This function transfers ownership of an anchored provider.

Inputs

- `provider`
  - **Control**: Current provider owner.
  - **Constraints**: The `provider` must be tracked, and `newOwner` must be nonzero.
  - **Impact**: Changes the provider owner.
- `newOwner`
  - **Control**: Current provider owner.
  - **Constraints**: The `provider` must be tracked, and `newOwner` must be nonzero.
  - **Impact**: Changes the provider owner.

Branches and code coverage

Intended branches

- Transfers provider ownership.
  - ☐ Test coverage

Negative branches

- A non-owner caller or invalid new owner causes the call to revert.

  - ☐ Test coverage

Function call analysis

- `_providers.contains(provider)`
  - **What is controllable?** The caller selects `provider`. The fixed internal EnumerableSet implementation reads that address's position in the factory's registry; it does not query the provider.
  - **If the return value is controllable, how is it used and how can it go wrong?** The returned boolean reflects registry membership and is consumed by `require(..., ProviderNotTracked())` before the operation. Membership comes from factory storage and is restricted to providers deployed by `createAnchoredProvider`; it is not a caller-supplied response.
  - **What happens if it reverts, reenters, or does other unusual control flow?** The `ProviderNotTracked()` is defensive on this owner-authorized path: the factory records owners only for its registered deployments and never removes those providers, so an authorized entry cannot fail membership under the implementation at the time of writing. The lookup is internal and has no callback or reentrancy boundary. The nonzero `newOwner` check runs before the ownership mapping is changed.
- `_providersByCreator[previousOwner].remove(provider)`
  - **What is controllable?** The current owner supplies `newOwner` and the tracked `provider`; `previousOwner` is loaded from the checked ownership mapping. The body first writes `providerOwner[provider] = newOwner` then updates these two fixed internal sets. Neither owner address is called.
  - **If the return value is controllable, how is it used and how can it go wrong?** The remove/add boolean results are ignored. Removal updates the old owner's array and positions using swap-and-pop; addition inserts into the new owner's set only if absent. These operations maintain enumeration alongside the authoritative ownership mapping. If the owner transfers to itself, removal followed by addition retains its membership, possibly changing enumeration order. Existing `isUpdater` grants are not cleared by this path.
  - **What happens if it reverts, reenters, or does other unusual control flow?** The libraries do not transfer control externally, so the intermediate ownership/set order is not exposed to a callback. Any revert rolls back the ownership write, both set updates, and the final event together.

5.13. ChainlinkOracle.sol

Function: `receive() external payable override`

This function accepts and retains ETH sent to the oracle. The current Chainlink verifier path forwards zero ETH.

Inputs

- `msg.value`
  - **Control**: External sender.
  - **Constraints**: N/A.
  - **Impact**: Increases the oracle's ETH balance.

Branches and code coverage

Intended branches

- Accepts a direct ETH transfer.
  - ☐ Test coverage

Negative branches

- No function-specific failure branch exists.
  - ☐ Test coverage

Function: `function removeVerifier(uint8 tag) external onlyRole(ADMIN_ROLE)`

This function removes the verifier assigned to a report tag.

Inputs

- `tag`
  - **Control**: The `ADMIN_ROLE` holder.
  - **Constraints**: The `tag` must identify a registered verifier.
  - **Impact**: Deletes the tag's verifier configuration.

Branches and code coverage

Intended branches

- Removes an existing verifier entry and emits the removal event.

- ☐ Test coverage

Negative branches

- A non-admin caller or an unregistered `tag` causes the call to revert.
  - ☐ Test coverage

Function call analysis

- `_tags.remove(tag)`
  - **What is controllable?** The authorized caller supplies `tag`; `_remove` reads its position in the oracle's set. It swaps the last tag into a removed nonfinal position, fixes that position mapping, pops the array, and deletes the removed tag's position.
  - **If the return value is controllable, how is it used and how can it go wrong?** The boolean return is checked by `require(_tags.remove(tag), UnknownVerifierTag(tag))`, and `false` means the tag was absent. Only a successful removal is followed by `delete verifierByTag[tag]` and `VerifierRemoved(tag)`.
  - **What happens if it reverts, reenters, or does other unusual control flow?** The `UnknownVerifierTag(tag)` aborts an absent-tag removal. All work is internal storage manipulation, with no call to the removed proxy and no callback. A revert rolls back the whole removal.

Function: `function setVerifier(uint8 tag, address proxy, bool supportsView) external onlyRole(ADMIN_ROLE)`

This function registers or replaces the verifier configuration for a report tag.

Inputs

- `tag`
  - **Control**: The `ADMIN_ROLE` holder.
  - **Constraints**: The `proxy` must be nonzero.
  - **Impact**: Stores the verifier proxy and whether `verifyView` is supported.
- `proxy`
  - **Control**: The `ADMIN_ROLE` holder.
  - **Constraints**: The `proxy` must be nonzero.
  - **Impact**: Stores the verifier proxy and whether `verifyView` is supported.
- `supportsView`

- **Control**: The `ADMIN_ROLE` holder.
- **Constraints**: The `proxy` must be nonzero.
- **Impact**: Stores the verifier proxy and whether `verifyView` is supported.

Branches and code coverage

Intended branches

- Stores a valid verifier configuration and emits the configuration event.
  - ☐ Test coverage

Negative branches

- A non-admin caller or zero `proxy` causes the call to revert.
  - ☐ Test coverage

Function call analysis

- `_setVerifier(tag, proxy, supportsView)`
  - **What is controllable?** The authorized caller supplies the tag, proxy address, and mode; `_setVerifier` rejects a zero proxy then writes `verifierByTag[tag]` before adding the tag to the local set. It makes no call to `proxy` and does not inspect its code or interface.
  - **If the return value is controllable, how is it used and how can it go wrong?** `_contains` reads the set's position mapping; `add` returns whether a new tag was inserted. That boolean is ignored, so an existing tag's verifier configuration is replaced without duplicating its set entry. Afterwards, `VerifierSet` is emitted.
  - **What happens if it reverts, reenters, or does other unusual control flow?** A zero proxy triggers a bare `require` revert. The helpers operate on the oracle's own storage and cannot reenter. Any revert rolls back both mapping and set changes; no verifier is invoked during configuration.

Function: `function updateReport(bytes calldata taggedReport) external`

This function verifies and normalizes one tagged Chainlink report then stores nonzero normalized prices with a valid timestamp strictly newer than the stored timestamp.

Inputs

- `taggedReport`

- **Control**: Arbitrary.
- **Constraints**: The `taggedReport` must be nonempty; its leading tag must identify a configured verifier; and that verifier must accept the remaining report bytes.
- **Impact**: Supplies the price report to verify and store.

Branches and code coverage

Intended branches

- Calls `verifyView` or `verify` with the configured zero ETH fee, normalizes the returned report, and stores it if the normalized price is nonzero and the valid timestamp is strictly newer. A zero normalized price returns before the store-time timestamp checks; equal or older timestamps leave storage unchanged.

  ☐ Test coverage

Negative branches

- Empty data, an unknown tag, unsupported report schema, invalid report encoding, verifier failure, malformed verifier return data, or a reached price, bid/ask, cast, or timestamp validation failure causes the update to revert.

  ☐ Test coverage

Function call analysis

- `_store(_verifyReport(taggedReport))`
  - **What is controllable?** The caller controls the leading tag and report suffix. The entry point is nonpayable and has no caller-role or feed-registration modifier.
  - **If the return value is controllable, how is it used and how can it go wrong?** `_verifyReport` requires at least one byte, reads `verifierByTag[tag]` into memory, and rejects a zero proxy. The caller chooses among routes; prior ADMIN configuration controls each proxy and `supportsView`. The suffix `taggedReport[1:]`, not the tag, becomes `report`. The returned bytes are trusted as the verifier's report and are not cryptographically reverified in the oracle.
  - **What happens if it reverts, reenters, or does other unusual control flow?** The `EmptyPayload` or `UnknownVerifierTag(tag)` stops the current update. Any later uncaught verifier, decoder, or time-check error reverts the invocation and its nested effects.
- `v.proxy.verifyView(report)`
  - **What is controllable?** The target and mode come from the selected stored route; the report suffix is caller supplied. The verifier sees the oracle as its

caller. This path transfers no ETH and does not pass `feeToken`.

- **If the return value is controllable, how is it used and how can it go wrong?** The proxy controls the ABI-encoded bytes returned to `_verifyReport`. Those bytes determine the feed ID and all normalized data; syntactically valid but fabricated data can pass the subsequent checks if the selected verifier is malicious. The repository defines this boundary through `IVerifierProxy`; no particular deployed proxy's signature policy is enforced by this oracle.
- **What happens if it reverts, reenters, or does other unusual control flow?** There is no try-catch or low-level success flag; verifier revert propagates, and missing/malformed returned bytes fail high-level ABI decoding. STATICCALL permits read-only callbacks but prevents state changes, logs, and state-changing reentrancy throughout its static call tree. The outer entry point has made no oracle storage write before this verification call.

- `v.proxy.verify{value: VERIFICATION_FEE}(report, abi.encode(feeToken))`
  - **What is controllable?** The selected stored proxy receives the caller-supplied report and an ABI-encoded immutable fee-token address. The `VERIFICATION_FEE` is exactly zero wei, independent of oracle balance; the caller cannot set a nonzero verification payment through this entry point.
  - **If the return value is controllable, how is it used and how can it go wrong?** The returned bytes are used exactly like the view-path report. There is no independent signer check or comparison against an externally requested feed ID — the returned report's own feed ID selects the storage key after schema validation. A verifier requiring nonzero ETH can reject this zero-value call.
  - **What happens if it reverts, reenters, or does other unusual control flow?** No try-catch or reentrancy lock exists. Unlike `verifyView`, this path can change external state or call the oracle again, subject to the called entry point's checks. The outer entry point has made no oracle storage write before this verification call. After return, `_store` reads the then-current timestamp, so a reentrant newer store can cause the outer report to be skipped. A revert or malformed bytes return aborts the entire invocation; revert data is not deliberately replaced.
- `_store(reportData)`
  - **What is controllable?** The verified bytes returned by the selected proxy control this internal path. The `_peekFeedId` performs an assembly memory load of the first word; it does not itself check byte length; `_feedResolution` returns the high nibble of feed-ID byte zero, while the low 12 bits of the upper 16 bits identify the base schema.
  - **If the return value is controllable, how is it used and how can it go wrong?** Resolution zero selects base schema 3 or 4; resolution one permits only base schema 3 for HFS. The selected normalizer uses `abi.decode` into its concrete report struct and returns `(feedId, OracleData)`. Other schema/resolution combinations revert with `UnsupportedReportSchema`.

The locally decoded feed ID determines the eventual `oracleData` key; there is no `priceGuard` or `stateGuard` call.

- **What happens if it reverts, reenters, or does other unusual control flow?** These calls are internal and pure. Unsupported schemas and invalid struct encodings revert, without callbacks or writes. They do not establish report authenticity beyond the external verifier's assertion.

- `_toMid8(price)`

  - **What is controllable?** The returned report controls signed raw price, bid/ask or v4 market status; `_toMid8` requires a positive raw price, divides by `PRICE_SCALE = 1e10` with floor rounding, then casts to `uint64`.
  - **If the return value is controllable, how is it used and how can it go wrong?** V3/HFS require `ask >= bid`, compute `max(ask-price, price-bid)` in `int256`, and use `Math.ceilDiv(1e6 * deviation, price)`, saturating at 65,534 before converting to `uint16`. V4 sets `spread0` to zero for status 2 and 65 and 535 otherwise. Every schema sets `spread1` to 65,535. The `ceilDiv` uses internal `SafeCast.toUint(a > 0)` to turn the numerator-nonzero flag into 0 or 1; its denominator is positive here. The `int192` input bounds keep deviation arithmetic and its scale product within range.
  - **What happens if it reverts, reenters, or does other unusual control flow?** The `InvalidReportPrice` or `uint64` `SafeCastOverflowedUintDowncast` propagates. The capped spread makes the `uint16` conversion safe on this path; a division-by-zero branch in Math is unreachable after the price check. These internal library calls cannot reenter and make no storage changes.

- `toTimeMs(...)`

  - **What is controllable?** Report observation times supply `uint256(r.observationsTimestamp) * 1000` for second-resolution v3/v4 and `uint256(r.observationsTimestamp)` for HFS. The allowed drift is fixed at deployment. The timestamp compared against is the current stored value after verification returns.
  - **If the return value is controllable, how is it used and how can it go wrong?** `toTimeMs` enforces `uint56`: v3/v4 `uint32` seconds multiplied by 1,000 fit; an HFS `uint64` value may overflow. All normalization, including this conversion and bid/ask checks, completes before `_store` returns early for a floored price of zero. Otherwise, `revertIfZero` uses `isZero`, drift uses `toSeconds` (floor division by 1,000), and `isAfter` compares raw milliseconds. Only strictly newer data is stored with `FeedUpdated`; equal/older data is ignored.
  - **What happens if it reverts, reenters, or does other unusual control flow?** The `TimeMsOverflow`, `ZeroTimestamp`, `FutureTimestamp`, or checked overflow of `block.timestamp + MAX_TIME_DRIFT` propagates when its call is reached. A zero normalized price skips the last three checks but cannot skip preceding normalization. These are internal calls without external execution; writes/events occur only after all checks. Any revert rolls back the whole update.

Function: `function updateReports(bytes[] calldata taggedReports) external`

This function processes tagged reports in order, storing only those that pass normalization and the conditional timestamp checks.

Inputs

- `taggedReports`
  - **Control**: Arbitrary.
  - **Constraints**: Each item must be nonempty, use a configured verifier tag, and be accepted by that verifier.
  - **Impact**: Defines the ordered batch of report updates.

Branches and code coverage

Intended branches

- Verifies and normalizes each report in order. A zero normalized price is skipped before store-time timestamp checks; a valid nonzero price is stored only with a strictly newer timestamp. An empty batch is a no-op.
  - ☐ Test coverage

Negative branches

- A verifier failure, ABI decoding failure, unsupported schema, or reached normalization or timestamp validation failure reverts the entire batch, including earlier updates.
  - ☐ Test coverage

Function call analysis

- `_store(_verifyReport(taggedReports[i]))`
  - **What is controllable?** The caller controls the array, its order, each leading tag, and each report suffix. The loop processes one complete verify/decode/store path per element; an empty array makes no calls.
  - **If the return value is controllable, how is it used and how can it go wrong?** `_verifyReport` requires at least one byte, reads `verifierByTag[tag]` into memory, and rejects a zero proxy. The caller chooses among routes; prior ADMIN configuration controls each proxy and `supportsView`. The suffix `taggedReport[1:]`, not the tag, becomes `report`. The returned bytes are trusted as the verifier's report and are not cryptographically reverified in the oracle.

- **What happens if it reverts, reenters, or does other unusual control flow?** The `EmptyPayload` or `UnknownVerifierTag(tag)` stops the current update. Any uncaught error in any element reverts the whole batch, including earlier stores, events, and state changes made by nonstatic verifier calls.

- `v.proxy.verifyView(report)`
  - **What is controllable?** The target and mode come from the selected stored route; the report suffix is caller supplied. The verifier sees the oracle as its caller. This path transfers no ETH and does not pass `feeToken`.
  - **If the return value is controllable, how is it used and how can it go wrong?** The proxy controls the ABI-encoded bytes returned to `_verifyReport`. Those bytes determine the feed ID and all normalized data; syntactically valid but fabricated data can pass the subsequent checks if the selected verifier is malicious. The repository defines this boundary through `IVerifierProxy`; no particular deployed proxy's signature policy is enforced by this oracle.
  - **What happens if it reverts, reenters, or does other unusual control flow?** There is no try-catch or low-level success flag; a verifier revert propagates, and missing/malformed returned bytes fail high-level ABI decoding. STATICCALL permits read-only callbacks but prevents state changes, logs, and state-changing reentrancy throughout its static call tree. Earlier batch elements have already completed their verification and storage, so their changes are visible to later verifier calls. Each element reloads its route; reentrant ADMIN-authorized route changes can affect subsequent elements.
- `v.proxy.verify{value: VERIFICATION_FEE}(report, abi.encode(feeToken))`
  - **What is controllable?** The selected stored proxy receives the caller-supplied report and an ABI-encoded immutable fee-token address. The `VERIFICATION_FEE` is exactly zero wei, independent of oracle balance; the caller cannot set a nonzero verification payment through this entry point.
  - **If the return value is controllable, how is it used and how can it go wrong?** The returned bytes are used exactly like the view-path report. There is no independent signer check or comparison against an externally requested feed ID; the returned report's own feed ID selects the storage key after schema validation. A verifier requiring nonzero ETH can reject this zero-value call.
  - **What happens if it reverts, reenters, or does other unusual control flow?** No try-catch or reentrancy lock exists. Unlike `verifyView`, this path can change external state or call the oracle again, subject to the called entry point's checks. Earlier batch elements have already completed their verification and storage, so their changes are visible to later verifier calls. Each element reloads its route; reentrant ADMIN-authorized route changes can affect subsequent elements. After return, `_store` reads the then-current timestamp, so a reentrant newer store can cause the outer report to be skipped. A revert or malformed bytes return aborts the entire invocation; revert data is not deliberately replaced.

- `_store(reportData)`
    - **What is controllable?** The verified bytes returned by the selected proxy control this internal path. The `_peekFeedId` performs an assembly memory load of the first word; it does not itself check byte length. The `_feedResolution` returns the high nibble of feed-ID byte zero, while the low 12 bits of the upper 16 bits identify the base schema.
    - **If the return value is controllable, how is it used and how can it go wrong?** Resolution zero selects base schema 3 or 4; resolution one permits only base schema 3 for HFS. The selected normalizer uses `abi.decode` into its concrete report struct and returns `(feedId, OracleData)`. Other schema/resolution combinations revert with `UnsupportedReportSchema`. The locally decoded feed ID determines the eventual `oracleData` key; there is no `priceGuard` or `stateGuard` call.
    - **What happens if it reverts, reenters, or does other unusual control flow?** These calls are internal and pure. Unsupported schemas and invalid struct encodings revert, without callbacks or writes. They do not establish report authenticity beyond the external verifier's assertion.
- `_toMid8(price)`
    - **What is controllable?** The returned report controls signed raw price, bid/ask, or v4 market status; `_toMid8` requires a positive raw price, divides by `PRICE_SCALE = 1e10` with floor rounding, and then casts to `uint64`.
    - **If the return value is controllable, how is it used and how can it go wrong?** V3/HFS require `ask >= bid`, compute `max(ask-price, price-bid)` in `int256`, and use `Math.ceilDiv(1e6 * deviation, price)`, saturating at 65,534 before converting to `uint16`. V4 sets `spread0` to zero for status 2 and 65 and 535 otherwise. Every schema sets `spread1` to 65,535. The `ceilDiv` uses internal `SafeCast.toUint(a > 0)` to turn the numerator-nonzero flag into 0 or 1; its denominator is positive here. The `int192` input bounds keep deviation arithmetic and its scale product within range.
    - **What happens if it reverts, reenters, or does other unusual control flow?** The `InvalidReportPrice` or `uint64` `SafeCastOverflowedUintDowncast` propagates. The capped spread makes the `uint16` conversion safe on this path; a division-by-zero branch in Math is unreachable after the price check. These internal library calls cannot reenter and make no storage changes.
- `toTimeMs(...)`
    - **What is controllable?** Report observation times supply `uint256(r.observationsTimestamp) * 1000` for second-resolution v3/v4 and `uint256(r.observationsTimestamp)` for HFS. The allowed drift is fixed at deployment. The timestamp compared against is the current stored value after verification returns.
    - **If the return value is controllable, how is it used and how can it go wrong?** `toTimeMs` enforces `uint56`: v3/v4 `uint32` seconds multiplied by 1,000 fit; an HFS uint64 value may overflow. All normalization, including this conversion and bid/ask checks, completes before `_store` returns early for a floored

price of zero. Otherwise, `revertIfZero` uses `isZero`, drift uses `toSeconds` (floor division by 1,000), and `isAfter` compares raw milliseconds. Only strictly newer data is stored with `FeedUpdated`; equal/older data is ignored.

- **What happens if it reverts, reenters, or does other unusual control flow?** The `TimeMsOverflow`, `ZeroTimestamp`, `FutureTimestamp`, or checked overflow of `block.timestamp + MAX_TIME_DRIFT` propagates when its call is reached. A zero normalized price skips the last three checks but cannot skip preceding normalization. These are internal calls without external execution; writes/events occur only after all checks. Later batch failure rolls them back.

5.14. CompressedOracle.sol

Function: `function allowContractPushers(address[] calldata pushers) external`

This function authorizes contract accounts to push compressed oracle updates.

Inputs

- `pushers`
  - **Control**: Arbitrary.
  - **Constraints**: Each pusher must differ from the caller, successfully answer `isPusher(caller)`, and return `true`.
  - **Impact**: Marks listed contract addresses as authorized pushers.

Branches and code coverage

Intended branches

- Authorizes every pusher that proves consent through `isPusher(caller)`.
  - ☐ Test coverage

Negative branches

- A self-remapping pusher, failed call, malformed return value, or `false` result causes the call to revert.
  - ☐ Test coverage

Function call analysis

- `pusher.staticcall(abi.encodeWithSignature("isPusher(address)", msg.sender))`

- **What is controllable?** The caller chooses each target pusher and the creator argument through its own identity. The callee sees the oracle as `msg.sender` and the creator as the encoded argument. No ETH is sent. The creator cannot change the selector, and authorization has no deadline/signature check.
- **If the return value is controllable, how is it used and how can it go wrong?** The low-level result is `(bool ok, bytes memory res)` — `require(ok)` is followed by `abi.decode(res, (bool))` and `require(allowed)`. Thus, a callee-controlled ABI true permits `namespaceRemapping[pusher] = msg.sender` and `PusherAuthorized`, even if the pusher was mapped to another creator. ABI false reverts — empty, short, or noncanonical boolean return data fails decoding. No exact return-length check is made; ABI-compatible trailing data is not rejected.
- **What happens if it reverts, reenters, or does other unusual control flow?** A callee revert produces `ok == false`; the bare `require(ok)` discards its original revert data. Static context propagates to nested calls; read-only callbacks are possible, but any attempted state write, log, or nonzero-value call in that context cannot succeed. A later pusher can observe earlier successful remappings via read-only calls. No reentrancy guard is installed. Any uncaught call/decode/check failure reverts the entire batch and its earlier writes; an empty array makes no calls.

Function: `function allowPushers(uint256 deadline, address[] calldata pushers, bytes[] memory signatures) external`

This function authorizes pusher remapping using signatures from each pusher.

Inputs

- `deadline`
  - **Control**: Arbitrary.
  - **Constraints**: The `deadline` must not have passed, the arrays must have equal length, each pusher must differ from the caller, each signature must recover its corresponding pusher, and `deadline` must be newer than that pusher's previously used deadline for the caller.
  - **Impact**: Adds each recovered pusher authorization.
- `pushers`
  - **Control**: Arbitrary.
  - **Constraints**: The `deadline` must not have passed, the arrays must have equal length, each pusher must differ from the caller, each signature must recover its corresponding pusher, and `deadline` must be newer than that pusher's previously used deadline for the caller.
  - **Impact**: Adds each recovered pusher authorization.

- `signatures`
    - **Control**: Arbitrary.
    - **Constraints**: The `deadline` must not have passed, the arrays must have equal length, each pusher must differ from the caller, each signature must recover its corresponding pusher, and `deadline` must be newer than that pusher's previously used deadline for the caller.
    - **Impact**: Adds each recovered pusher authorization.

Branches and code coverage

Intended branches

- Accepts valid pusher signatures and authorizes their corresponding remappings.
    - ☐ Test coverage

Negative branches

- An expired or reused deadline, mismatched arrays, self-remapping pusher, or invalid signature causes the call to revert.
    - ☐ Test coverage

Function call analysis

- `_ensureDeadline(deadline)`
    - **What is controllable?** The caller chooses `deadline`. The helper compares it to `block.timestamp`; it does not call a clock or another contract.
    - **If the return value is controllable, how is it used and how can it go wrong?** The helper returns no value. Its `require(block.timestamp <= deadline, DeadlineExceeded())` must pass before array lengths or signatures are processed.
    - **What happens if it reverts, reenters, or does other unusual control flow?** `DeadlineExceeded()` propagates before any delegation changes, including for an empty array. The call has no callback.
- `MessageHashUtils.toEthSignedMessageHash(keccak256(abi.encode(block.chainid, address(this), deadline, pusher, msg.sender)))`
    - **What is controllable?** The caller controls the deadline, both arrays, and creator identity; each array element selects the claimed pusher. The digest also binds the current chain and oracle address. ABI encoding and Keccak-256 are local operations; the OpenZeppelin helpers are internal, not calls to the pusher.
    - **If the return value is controllable, how is it used and how can it go wrong?**

The hash helper returns the EIP-191 digest; `ECDSA.recover` calls `tryRecover`, which accepts a 65-byte signature, extracts r/s/v, rejects high-s, and calls the fixed `ecrecover(hash, v, r, s)` precompile; `_throwError` converts recovery errors to custom reverts. The recovered nonzero signer must equal `pusher`; there is no ERC-1271 call. Only then is `deadline > usedPusherDeadline[pusher][msg.sender]` enforced, the deadline consumed, and `namespaceRemapping[pusher]` overwritten with the caller.

- **What happens if it reverts, reenters, or does other unusual control flow?** Array mismatch or a recovered different signer triggers a bare require; self-remapping triggers `NoSelfRemapping`. Recovery can revert with `ECDSAInvalidSignatureLength`, `ECDSAInvalidSignatureS` or `ECDSAInvalidSignature`; stale consent triggers `StaleConsent`. The precompile cannot call back. Earlier elements' mappings, consumed deadlines, and events roll back if a later element reverts; duplicate pushers with valid signatures hit the same consumed deadline on their second occurrence.

Function: `fallback() external override`

This function processes packed compressed-price updates.

Inputs

- `msg.data`
  - **Control**: Arbitrary.
  - **Constraints**: The calldata must contain a leading byte followed by one or more complete 32-byte slot words; each accepted word's timestamp must not exceed the permitted future drift.
  - **Impact**: Updates newer slot words in the caller's mapped namespace.

Branches and code coverage

Intended branches

- Stores each newer, valid slot word.
  - ☐ Test coverage

Negative branches

- Invalid calldata length or a timestamp beyond the permitted future drift causes the call to revert.
  - ☐ Test coverage

Function call analysis

- `toTimeMs(word >> 8 & X56)`
    - **What is controllable?** The sender chooses the word data. The creator is its existing `namespaceRemapping[msg.sender]` or the sender itself if unmapped; `key` is `bytes32((uint256(uint160(creator)) << 96) | uint256(uint8(word)))`. The prefix is skipped rather than validated, and the length must satisfy the fallback's one-byte-plus-whole-words format.
    - **If the return value is controllable, how is it used and how can it go wrong?** The `_loadStorage` returns an `sload` from the derived local key. Both `toTimeMs` arguments are masked to 56 bits, so their range check cannot fail here. The drift helper uses `toSeconds()` / `TimeMs.unwrap()` to compare the floored seconds to `block.timestamp + MAX_TIME_DRIFT`; `isAfter` compares unwrapped millisecond timestamps, and false skips the word. Repeated slot IDs use the state written by earlier iterations.
    - **What happens if it reverts, reenters, or does other unusual control flow?** The `BadCalldataLength` rejects invalid lengths before the loop. The drift helper can revert with `FutureTimestamp` or checked-addition overflow before the monotonicity check, even for a word that would later be skipped. These are internal storage/time operations, with no external return data or callback.
- `_writeStorage(key, bytes32(bytes32(word & ~uint256(0xff))))`
    - **What is controllable?** The accepted caller-controlled word supplies all four 48-bit positions; the low eight slot-ID bits are cleared in storage. Creator and slot are derived as above.
    - **If the return value is controllable, how is it used and how can it go wrong?** Neither helper returns a value. The `_writeStorage` executes `sstore`. The `_notarizeSlot` extracts positions 0..3 with `(newWord >> (208 - 48 * i)) & X48`; it skips packed zero and low-16-bit `0xffff` stall markers, and emits `FeedUpdated` for the remaining creator/chain/slot/position IDs. It performs no U64x32, Codebook256, price-provider, or price-guard call.
    - **What happens if it reverts, reenters, or does other unusual control flow?** The write precedes the events, with no external execution between them. A later word's revert rolls back every earlier word and event in this fallback invocation. There is no external reentrancy point.

Function: `function removePushers(address[] calldata pushers) external`

This function revokes pusher authorization for the listed addresses.

Inputs

- `pushers`

- **Control**: Arbitrary.
- **Constraints**: Each pusher must differ from the caller and be currently mapped to the caller.
- **Impact**: Clears each pusher's namespace remapping to this caller, restoring pushes into the pusher's own namespace.

Branches and code coverage

Intended branches

- Revokes each listed authorization.
  - ☐ Test coverage

Negative branches

- A pusher equal to the caller reverts with `NoSelfRemapping`; an unmapped pusher or one mapped to another creator reverts with `InvalidManager`.
  - ☐ Test coverage

Function: `function updateBySignature(address feedCreator, uint256 newSlotValue, bytes calldata signature) external override returns (bool)`

This function updates a compressed slot when the feed creator signs the update.

Inputs

- `feedCreator`
  - **Control**: Arbitrary.
  - **Constraints**: Must be nonzero — a newer update must recover this address from its signature.
  - **Impact**: Selects the namespace whose slot can be updated.
- `newSlotValue`
  - **Control**: Arbitrary.
  - **Constraints**: Its encoded timestamp must not exceed the permitted future drift. Storage changes require a timestamp strictly newer than the selected slot's stored timestamp.
  - **Impact**: Encodes the slot ID, timestamp, and packed price data; the slot-ID byte is cleared in storage.
- `signature`

- **Control**: Arbitrary.
- **Constraints**: For a newer update, must recover `feedCreator` over the contract's domain-separated signed slot hash.
- **Impact**: Authorizes a newer slot update — it is not examined when the timestamp is not newer.

Branches and code coverage

Intended branches

- Verifies the creator signature and stores a newer update.
  - ☐ Test coverage
- Returns `false` without checking the signature when the timestamp is not newer, after the creator and future-time checks pass.
  - ☐ Test coverage

Negative branches

- A zero creator or excessive future timestamp reverts. If the update is newer, malformed signature recovery or recovery of a different signer also reverts.
  - ☐ Test coverage

Function call analysis

- `toTimeMs(newSlotValue >> 8 & X56)`
  - **What is controllable?** The caller chooses a nonzero `feedCreator` and the whole slot word. The local storage key combines `feedCreator << 96` with `uint8(newSlotValue)`; namespace remapping is not consulted.
  - **If the return value is controllable, how is it used and how can it go wrong?** The `_loadStorage` performs local `sload`. Masking makes both TimeMs conversions fit `uint56`. The drift helper compares floored seconds to `block.timestamp + MAX_TIME_DRIFT`; `isAfter` then returns whether the incoming millisecond timestamp strictly exceeds the stored one. If false, the entry point immediately returns false without recovering any signature, writing storage, or notarizing positions.
  - **What happens if it reverts, reenters, or does other unusual control flow?** A zero creator reverts with `InvalidNamespace` before these calls. The `FutureTimestamp` or checked-addition overflow propagates before the monotonicity check. There is no external contract call or callback in this stage.
- `MessageHashUtils.toEthSignedMessageHash(keccak256(abi.encode(block.chainid, address(this), feedCreator, newSlotValue)))`

- **What is controllable?** The caller supplies the creator, entire word, and signature. The digest binds that word and creator to the current chain and oracle. No external call is made to `feedCreator`.
- **If the return value is controllable, how is it used and how can it go wrong?** The internal hash helper returns the EIP-191 digest — `recover` → `tryRecover` extracts r/s/v from a 65-byte signature, enforces low-s, and calls the fixed ecrecover precompile; `_throwError` turns recovery errors into reverts. The recovered address must equal `feedCreator`. There is no ERC-1271 signature validation.
- **What happens if it reverts, reenters, or does other unusual control flow?** Invalid length, high-s, or failed recovery propagates the corresponding OpenZeppelin ECDSA error; a valid different signer fails the bare equality require. No state was written, and the precompile cannot call back.

- `_writeStorage(key, bytes32(newSlotValue & ~uint256(0xff)))`
  - **What is controllable?** The signed word controls all four positions at the derived creator/slot key. Clearing the low eight slot-ID bits does not change the signed data that was verified.
  - **If the return value is controllable, how is it used and how can it go wrong?** Both helpers return nothing. The first performs `sstore`; the second checks four 48-bit positions, skips zero and low-16-bit `0xffff` stall markers, and emits a creator/chain/slot/position `FeedUpdated` ID for each remaining position. The entry point then returns true. No price or spread decoding or price-guard check occurs.
  - **What happens if it reverts, reenters, or does other unusual control flow?** There is no external execution between verification, the write, and event emission. Any revert aborts the write and logs together; no callee-controlled return value exists in these helpers.

5.15. PriceProviderFactory.sol

Function: `function addProvider(address provider) external override onlyRole(ADMIN_ROLE)`

This function adds an existing price provider to the factory registry.

Inputs

- `provider`
  - **Control**: The `ADMIN_ROLE` holder.
  - **Constraints**: The `provider.factory()` must equal this factory, and `provider` must not already be tracked.
  - **Impact**: Registers the provider.

Branches and code coverage

Intended branches

- Adds a valid provider.
  - ☐ Test coverage

Negative branches

- Unauthorized access or invalid provider configuration causes the call to revert.
  - ☐ Test coverage

Function call analysis

- `PriceProvider(provider).factory()`
  - **What is controllable?** The admin selects an arbitrary `provider` address. This view call is a `STATICCALL` to that address; the Solidity cast does not establish that it contains the repository's `PriceProvider` bytecode. A custom implementation can choose the returned address.
  - **If the return value is controllable, how is it used and how can it go wrong?** The returned address is compared with `address(this)` in a plain `require`. Returning this factory passes the check even if the implementation has different setter behavior; this is an admin-controlled admission decision, not code-provenance verification. A different return value prevents registration.
  - **What happens if it reverts, reenters, or does other unusual control flow?** A revert or invalid/missing ABI return data propagates without a catch before any registry write. Static context prevents the target or nested callbacks from changing state, although arbitrary code can perform reads, consume gas, or revert. The repository's genuine `factory()` getter merely returns its immutable factory address.
- `_providers.add(provider)`
  - **What is controllable?** The successfully queried admin-selected address is inserted into the factory's fixed internal registry. Existing registry state determines whether insertion is possible.
  - **If the return value is controllable, how is it used and how can it go wrong?** False insertion is converted to `ProviderAlreadyTracked()`; true insertion grants `isProvider` recognition to the address. The boolean comes from local membership, not the provider. A successful insertion therefore inherits the limited assurance supplied by the preceding getter check.
  - **What happens if it reverts, reenters, or does other unusual control flow?** The helper contains only local membership checks and storage updates. Duplicate registration reverts without an external callback; any subsequent transaction revert also undoes insertion.
- `_providersByCreator[owner].add(provider)`

- **What is controllable?** The `owner` is taken from `providerOwner[provider]`; if it was zero, the admin caller becomes owner and that mapping is written before this helper. If a prior owner remains recorded, that owner is retained. The provider and owner addresses are set keys, not called contracts.
- **If the return value is controllable, how is it used and how can it go wrong?** The insertion boolean is ignored. The helper inserts only if absent and keeps enumeration aligned with the chosen owner; no external return determines ownership. Adopting an unowned address therefore gives the calling admin authority over its factory controls.
- **What happens if it reverts, reenters, or does other unusual control flow?** No callback occurs between global registry insertion, owner adoption, creator-set insertion, and `ProviderAdded`. Any revert rolls all these changes back together.

Function: `function createPriceProvider(address _oracle, bytes32 _feedId, int256 _marginStep, uint256 _maxTimeDelta, address _baseToken, address _quoteToken) external override returns (address provider)`

This function deploys and registers a price provider for an oracle feed.

Inputs

- `_oracle`
  - **Control**: Arbitrary.
  - **Constraints**: Both `_baseToken` and `_quoteToken` must be nonzero and distinct; `_marginStep` must be strictly between `-BPS_BASE` and `BPS_BASE`; and `_maxTimeDelta` must be greater than zero and at most seven days.
  - **Impact**: Defines and deploys the provider.
- `_feedId`
  - **Control**: Arbitrary.
  - **Constraints**: Both `_baseToken` and `_quoteToken` must be nonzero and distinct; `_marginStep` must be strictly between `-BPS_BASE` and `BPS_BASE`; and `_maxTimeDelta` must be greater than zero and at most seven days.
  - **Impact**: Defines and deploys the provider.
- `_marginStep`
  - **Control**: Arbitrary.
  - **Constraints**: Both `_baseToken` and `_quoteToken` must be nonzero and distinct; `_marginStep` must be strictly between `-BPS_BASE` and `BPS_BASE`; and `_maxTimeDelta` must be greater than zero and at most seven days.
  - **Impact**: Defines and deploys the provider.
- `_maxTimeDelta`

- **Control**: Arbitrary.
- **Constraints**: Both `_baseToken` and `_quoteToken` must be nonzero and distinct; `_marginStep` must be strictly between `-BPS_BASE` and `BPS_BASE`; and `_maxTimeDelta` must be greater than zero and at most seven days.
- **Impact**: Defines and deploys the provider.

- `_baseToken`
  - **Control**: Arbitrary.
  - **Constraints**: Both `_baseToken` and `_quoteToken` must be nonzero and distinct; `_marginStep` must be strictly between `-BPS_BASE` and `BPS_BASE`; and `_maxTimeDelta` must be greater than zero and at most seven days.
  - **Impact**: Defines and deploys the provider.
- `_quoteToken`
  - **Control**: Arbitrary.
  - **Constraints**: Both `_baseToken` and `_quoteToken` must be nonzero and distinct; `_marginStep` must be strictly between `-BPS_BASE` and `BPS_BASE`; and `_maxTimeDelta` must be greater than zero and at most seven days.
  - **Impact**: Defines and deploys the provider.

Branches and code coverage

Intended branches

- Deploys and registers a valid price provider.
  - ☐ Test coverage

Negative branches

- Invalid configuration or deployment failure causes creation to revert.
  - ☐ Test coverage

Function call analysis

- `new PriceProvider(address(this), _oracle, _feedId, _marginStep, _maxTimeDelta, _baseToken, _quoteToken)`
  - **What is controllable?** The permissionless caller supplies the oracle, feed, margin step, staleness bound, and token pair; `_factory` is fixed to this factory. `CREATE` executes the fixed `PriceProvider` constructor. Neither the oracle nor the feed is allowlisted or queried by this path.
  - **If the return value is controllable, how is it used and how can it go wrong?** The returned address becomes `provider`, is registered, and is returned to the caller. The constructor stores the factory, oracle, feed, and tokens as

immutables; checks nonzero distinct tokens; enforces `-BPS_BASE < _marginStep < BPS_BASE` and `0 < _maxTimeDelta <= 7 days`; and derives immutable bid/ask step factors. Confidence and its update timestamp remain zero. A successful deployment establishes configuration validity under these checks, not live oracle or token validity.

- **What happens if it reverts, reenters, or does other unusual control flow?** A token-pair `require`, `MarginStepOutOfBounds()`, `MaxTimeDeltaOutOfBounds()`, or failed creation propagates before registry writes. The fixed nonzero factory argument satisfies the constructor's factory check. The constructor has no internal helper or external calls and never invokes the supplied oracle or tokens, so it cannot call back into the factory. Creation is rolled back if the outer transaction later reverts.

- `_providers.add(provider)`
  - **What is controllable?** The `provider` is the fresh address produced by `CREATE`, and `creator` is the original caller. Both set targets and the implementation are fixed internal factory state/code.
  - **If the return value is controllable, how is it used and how can it go wrong?** The returned insertion booleans are ignored. Each helper inserts an absent address into its array/position mapping; the factory then records `providerOwner[provider] = creator`. This grants registry recognition and owner authority to the deployed instance.
  - **What happens if it reverts, reenters, or does other unusual control flow?** These internal library operations make no callback. Registry writes follow successful construction and precede the event getters; an ensuing revert rolls back the sets, ownership mapping, events, and deployment together.
- `p.baseToken()`
  - **What is controllable?** The argument-free `STATICCALL`s target only the just-deployed `PriceProvider`. The caller determines the returned immutable token addresses through the validated constructor inputs.
  - **If the return value is controllable, how is it used and how can it go wrong?** Both addresses are consumed solely as event fields. They do not validate token contracts, control registration, or select further external calls.
  - **What happens if it reverts, reenters, or does other unusual control flow?** Failures are uncaught, so a getter revert or decoding failure would undo this entire creation transaction. These fixed getters only return immutables and have no callbacks; static execution also prohibits state mutation.

Function: `function setConfidence(address[] calldata providers, uint256[] calldata values) external override`

This function updates confidence settings for a batch of registered providers.

Inputs

- `providers`
  - **Control**: A caller authorized as the recorded owner or a delegated updater for every selected provider.
  - **Constraints**: Must match `values.length` — every entry must be tracked and permit the caller as its owner or updater.
  - **Impact**: Selects the contracts called in order. Registry membership can include custom implementations admitted by ADMIN.
- `values`
  - **Control**: A caller authorized as the recorded owner or a delegated updater for every selected provider.
  - **Constraints**: Must match `providers.length`. Standard providers enforce `newValue <= CONFIDENCE_MAX`; an admitted custom implementation determines its own value validation.
  - **Impact**: Supplies each target's new confidence setting.

Branches and code coverage

Intended branches

- Updates valid provider-confidence pairs.
  - ☐ Test coverage

Negative branches

- Mismatched arrays, unauthorized callers, or invalid values causes the batch to revert.
  - ☐ Test coverage

Function call analysis

- `_providers.contains(providers[i])`
  - **What is controllable?** The caller supplies the provider/value arrays and order. Equal lengths are checked before iteration. Each internal check reads registry membership, the recorded owner, and `isUpdater[provider][msg.sender]` from factory storage.
  - **If the return value is controllable, how is it used and how can it go wrong?** The membership boolean gates the update with `ProviderNotTracked()`. The `_requireUpdater` has no return and permits the owner or a delegated updater, otherwise reverting with `NotProviderUpdater()`. Membership may originate from this factory's deployment or admin `addProvider`, so it does not by itself prove the target uses the standard `PriceProvider`

implementation.

- **What happens if it reverts, reenters, or does other unusual control flow?** These checks make no callbacks. Their failures propagate and roll back all earlier updates in the batch. Empty arrays make no calls; length mismatch reverts before any provider is examined.

- `PriceProvider(providers[i]).setConfidenceParam(values[i])`

  - **What is controllable?** The authorized caller selects the tracked target and value. This is an ordinary external `CALL`, with this factory as `msg.sender`, and no explicit gas cap or reentrancy guard. Factory-created providers have the fixed implementation; an admin-adopted address can contain arbitrary code despite having returned this factory from `factory()`.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed. The standard implementation checks its immutable factory, `newValue <= CONFIDENCE_MAX`, and the confidence cooldown; then stores confidence and the update timestamp; and emits an event. An admitted custom target can return successfully without applying those checks or writing confidence, and this function has no postcall check to detect that.
  - **What happens if it reverts, reenters, or does other unusual control flow?** For the standard implementation, the immutable factory check necessarily passes for factory-created or genuinely admitted instances, and `ConfidenceParamOutOfBounds()` or `CooldownNotElapsed()` propagates. It makes no callbacks, and repeating it after a successful update hits the cooldown. An admitted custom target can reenter, consume gas, or revert. A callback reaches factory entry points with the provider as `msg.sender`, so their owner/updater/admin checks still apply; membership alone grants none of those permissions. The factory makes no own storage update before or after this call. Any uncaught failure reverts the entire batch, including earlier provider and nested-call state changes; a malicious target that returns normally can still make the batch appear successful.

5.16. PythOracle.sol

Function: `fallback() external payable override`

This function processes Pyth Lazer price-update calldata.

Inputs

- `msg.data`

  - **Control**: Arbitrary.
  - **Constraints**: The first two bytes declare the feed count, and calldata must be at least `2 + 4 * feedCount` bytes; the remaining update payload must pass

Pyth Lazer verification.
- **Impact**: Verifies and stores valid price updates.

- `msg.value`
  - **Control**: Arbitrary.
  - **Constraints**: The oracle's total available ETH balance must cover the one-wei verification call.
  - **Impact**: Adds to the balance used for the verification fee — excess attached ETH remains in the oracle.

Branches and code coverage

Intended branches

- Verifies and stores a valid Pyth Lazer update.
  - ☐ Test coverage

Negative branches

- A feed-count length mismatch or failed Pyth Lazer verification causes the call to revert.
  - ☐ Test coverage

Function call analysis

- `_verifyAndStore(oracleData, updateFeedIds, priceUpdate)`
  - **What is controllable?** The caller supplies the two-byte count, four-byte IDs, and remaining update bytes. The fallback first requires `2 + feedsLength * 4 <= calldatasize()` and then constructs the ID array and payload slice. The storage mapping passed to the helper is fixed to this oracle's `oracleData`. Attached ETH increases the balance available for verification.
  - **If the return value is controllable, how is it used and how can it go wrong?** `_verifyPayload` returns packed per-feed values, the final parse position, and payload length. All feeds are parsed before the storage loop begins; this is not a separate verification call per feed. The returned feed IDs, prices, and timestamps come from the external verifier's payload.
  - **What happens if it reverts, reenters, or does other unusual control flow?** The upfront bound reverts with `FeedsLengthMismatch` before allocation/slicing if it fails. Any subsequent uncaught error rolls back the whole fallback invocation, its payment, nested calls, earlier stores, and events. Neither helper has a reentrancy lock.
- `pythLazer.verifyUpdate{value: PYTH_VERIFICATION_FEE}(priceUpdate)`
  - **What is controllable?** The verifier address is immutable from deployment, not an address in the current calldata. The caller supplies `priceUpdate`; the

oracle pays exactly 1 wei from its balance, which may include the current `msg.value`. The callee sees the oracle as `msg.sender`; a separate signer value returned by the verifier is ignored by LazerConsumer.

- **If the return value is controllable, how is it used and how can it go wrong?** The returned bytes become the sole payload parsed for feed IDs and values. The consumer does not independently recover a signer — an arbitrary or upgraded verifier can return fabricated but well-formed data that passes all parser checks. ABI decoding still requires a usable returned payload.
- **What happens if it reverts, reenters, or does other unusual control flow?** A verifier revert or malformed returned ABI propagates without try-catch; insufficient oracle balance also prevents the call. A verifier requiring more than 1 wei rejects it. This non-static call can execute code and reenter other oracle entry points subject to their checks. The outer invocation has not stored feed data yet; its later timestamp comparisons use any state left by successful nested calls. Any outer revert rolls back the verifier's state changes and fee transfer.

- `payable(msg.sender).transfer(msg.value - verification_fee)`

  - **What is controllable?** At this boundary, `msg.sender` is the oracle and the payment is the fixed 1 wei. The refund amount depends on verifier storage; it is positive if that stored fee is zero. This behavior describes the checked-in verifier implementation, not every possible contract at the configured address.
  - **If the return value is controllable, how is it used and how can it go wrong?** `transfer` has no Solidity return value and enforces successful delivery. It uses empty calldata and a 2,300-gas stipend, so the oracle's empty payable `receive()` handles the refund. Refund processing occurs before envelope and signature checks.
  - **What happens if it reverts, reenters, or does other unusual control flow?** A failed refund reverts verification. The checked-in oracle receive body makes no further call or storage write. A later envelope/signature/parser failure also rolls back this refund and the original fee transfer.

- `ECDSA.tryRecover(keccak256(payload), uint8(update[68]) + 27, bytes32(update[4:36]), bytes32(update[36:68]))`

  - **What is controllable?** The caller supplies the envelope's magic, signature bytes, and `uint16` payload length. The verifier checks a minimum 71-byte envelope, EVM magic, and declared payload extent, and then hashes `update[71:71 + payload_len]`. Bytes beyond that declared payload are not included in its hash or returned payload. Trusted-signer expirations are verifier storage set by its authority.
  - **If the return value is controllable, how is it used and how can it go wrong?** The `tryRecover` rejects high-s and invokes the fixed `ecrecover(hash, v, r, s)` precompile. It returns `(signer, error, errorArg)`; this verifier ignores the error fields and rejects a zero signer with `"invalid signature"`. And `isValidSigner` returns `block.timestamp <`

`trustedSignerToExpiresAtMapping[signer]`; false becomes `"invalid signer"`. No EIP-191 prefix is added to this Pyth payload hash.

- **What happens if it reverts, reenters, or does other unusual control flow?** The envelope checks can revert with their string errors. Checked `uint16 71 + payload_len` or `uint8 update[68] + 27` arithmetic can panic. The precompile and local signer lookup cannot callback; `tryRecover` errors are handled through the zero-signer check rather than `ECDSA.recover` / `_throwError`. Any verifier failure propagates through the high-level call.

- `parseAndPack2(base, pos, propCfg, expMask, expCount)`

  - **What is controllable?** The externally returned payload controls the magic, one-byte feed count, per-feed ID, property count, and property bytes. Caller-supplied IDs must match in the same order. Allowed property sizes, expected mask, and expected count are immutable deployment configuration. The packet-header timestamp is not used for freshness.
  - **If the return value is controllable, how is it used and how can it go wrong?** The helper returns the parsed ID, packed value, and next position. It checks property count and allowed IDs, accumulates the found mask, and extracts `Price`, `Exponent`, `Confidence`, and `FeedUpdateTimestamp`; other configured properties are skipped by their fixed size. Property 12 consumes one exists byte plus eight timestamp bytes only when exists is nonzero. Its microseconds are floored to milliseconds; zero or > 48-bit milliseconds and nonpositive price yield a skipped marker. Missing price/exponent/confidence begin at zero. Duplicate configured properties are not rejected at deployment — payloads matching that count/mask can repeat fields; last price/exponent/confidence values win, while a timestamp with exists zero leaves an earlier parsed timestamp intact.
  - **What happens if it reverts, reenters, or does other unusual control flow?** Invalid magic, feed count/ID mismatch, zero feeds, unexpected property count/mask or unknown property reverts through the consumer's named errors. The helper performs raw memory loads rather than individual field bounds checks; the final `pos != payloadLen` check is deferred until after the store loop and can then revert with `PayloadLengthMismatch`. There is no external call or callback during parsing.

- `_normalize(v)`

  - **What is controllable?** The verifier's parsed positive raw price, `int16` exponent, and `uint64` confidence control normalization; the parsed `uint32` ID supplies the eventual mapping key. Invalid-price or absent-timestamp markers are skipped before `_normalize` is called.
  - **If the return value is controllable, how is it used and how can it go wrong?** `_normalize` returns `(uint64 normPrice, uint256 spreadU, bytes32 feedId)`. It requires `totalExpo = exponent + 8` within [-57,57], multiplies by `10 ** totalExpo.toUint256()` when nonnegative, or divides by `10 ** (-totalExpo).toUint256()` otherwise, and it converts to `uint64`. The extracted masked fields fit their preliminary casts; the scaled price can overflow `uint64`. Spread is the ceiling of `1e6 * confidence / rawPrice`,

capped at 65,534. Internally, `Math.ceilDiv` uses `SafeCast.toUint(a > 0)` to convert the numerator-nonzero flag to 0 or 1. A zero normalized price is subsequently skipped.

- **What happens if it reverts, reenters, or does other unusual control flow?** The exponent require or scaled-price SafeCast can revert before any future-time or stored-time comparison. Positive price and the exponent bound prevent zero division and unchecked multiplication wrap; nonnegative cast arguments are established before their `toUint256` calls. These internal library calls cannot call back.

- `toTimeMs(tsMs)`

  - **What is controllable?** The parsed timestamp was reduced to 48 bits, so it fits TimeMs's `uint56`. Drift comes from immutable deployment configuration. A previous successful store, including an earlier occurrence of the same feed ID in this packet, determines the current stored timestamp.
  - **If the return value is controllable, how is it used and how can it go wrong?** `toSeconds` floors milliseconds by 1,000. If it exceeds `block.timestamp + MAX_TIME_DRIFT`, the loop emits `FeedSkippedFutureTimestamp` and continues. This addition is inside an unchecked block and can wrap. Otherwise, only a strictly greater millisecond timestamp writes price, `spread0`, `spread1 = 65,535`, and time and then emits `FeedUpdated`. Equal/older timestamps are skipped. The capped spread always fits `uint16`; no `priceGuard` / `stateGuard` check occurs.
  - **What happens if it reverts, reenters, or does other unusual control flow?** These helpers are internal and cannot reenter. A future timestamp is a per-feed skip, not a revert. The final payload-position equality check runs after every store, and its failure rolls back all of them and all verification/refund effects. There is no separately reachable TimeMsOverflow or spread-cast overflow on this masked/capped path.

Function: `receive() external payable override`

This function accepts ETH used to pay Pyth update fees.

Inputs

- `msg.value`

  - **Control**: External sender.
  - **Constraints**: N/A.
  - **Impact**: Increases ETH available for Pyth calls.

Branches and code coverage

Intended branches

- Accepts ETH.
  - ☐ Test coverage

Negative branches

- No function-specific failure branch exists.
  - ☐ Test coverage

5.17. compressed-OracleBase.sol

Function: `function acceptStateGuardRole(bytes32 feedId) external`

This function accepts a pending guardian role for a compressed feed.

Inputs

- `feedId`
  - **Control**: Pending guardian.
  - **Constraints**: N/A.
  - **Impact**: Promotes the pending guardian to active guardian.

Branches and code coverage

Intended branches

- Transfers the feed guardian role to the pending guardian.
  - ☐ Test coverage

Negative branches

- A caller other than the pending guardian causes the call to revert.
  - ☐ Test coverage

Function: `function purgePendingStateGuardRole(bytes32 feedId) external checkRole(feedId)`

This function clears a pending guardian nomination for a compressed feed.

Inputs

- `feedId`
  - **Control**: The active guardian or the feed's default authority when no guardian is set (the encoded creator in CompressedOracle).
  - **Constraints**: N/A.
  - **Impact**: Deletes the pending guardian.

Branches and code coverage

Intended branches

- Clears the current pending guardian.
  - ☐ Test coverage

Negative branches

- A caller that does not match the active guardian, or the default authority when no guardian is set, reverts with `InvalidGuard`.
  - ☐ Test coverage

Function: `function purgeStateGuardRole(bytes32 feedId) external`

This function removes the active guardian role from a compressed feed.

Inputs

- `feedId`
  - **Control**: The active guardian or the feed's default authority (the encoded creator in CompressedOracle).
  - **Constraints**: N/A.
  - **Impact**: Clears the active and pending guardian state.

Branches and code coverage

Intended branches

- Removes the guardian role for the feed.
  - ☐ Test coverage

Negative branches

- A caller that is neither the active guardian nor the default authority reverts with `InvalidGuard`.

☐ Test coverage

Function call analysis

- `_defaultGuard(feedId)`
  - **What is controllable?** The caller supplies `feedId`. If it is already the active guard, short-circuit evaluation skips the default lookup. Otherwise, the compressed override validates the embedded chain, position < 4 and nonzero creator, and returns that creator; the standalone base returns address zero.
  - **If the return value is controllable, how is it used and how can it go wrong?** The returned default guard is compared to `msg.sender`, allowing the feed creator to purge another active guard. The slot and position returns from `_unpackFeedId` are ignored. Successful authorization deletes the active guard and any nonzero pending guard, emitting the corresponding deletion events.
  - **What happens if it reverts, reenters, or does other unusual control flow?** `FeedNotFound(feedId)` from the default lookup propagates before the outer check can finish. If both caller comparisons are false, `InvalidGuard(msg.sender)` reverts. An active guard can purge without default-ID validation. There are no external calls or callbacks, and all deletions are atomic.

Function: `function setPendingStateGuardRole(bytes32 feedId, address newGuard) external checkRole(feedId)`

This function nominates a new guardian for a compressed feed.

Inputs

- `feedId`
  - **Control**: The active guardian or the feed's default authority when no guardian is set (the encoded creator in CompressedOracle).
  - **Constraints**: N/A.
  - **Impact**: Replaces the pending guardian address.
- `newGuard`
  - **Control**: The active guardian or the feed's default authority when no guardian is set (the encoded creator in CompressedOracle).
  - **Constraints**: N/A.

- **Impact**: Replaces the pending guardian address.

Branches and code coverage

Intended branches

- Stores a new pending guardian.
  - ☐ Test coverage

Negative branches

- A caller that does not match the active guardian, or the default authority when no guardian is set, reverts with `InvalidGuard`.
  - ☐ Test coverage

Function: `function setPriceGuard(bytes32 feedId, uint128 minPrice, uint128 maxPrice) external checkRole(feedId)`

This function configures the accepted price interval for a compressed feed.

Inputs

- `feedId`
  - **Control**: The active guardian or the feed's default authority when no guardian is set (the encoded creator in CompressedOracle).
  - **Constraints**: The `minPrice` must be less than `maxPrice`.
  - **Impact**: Stores feed-specific lower and upper price guards.
- `minPrice`
  - **Control**: The active guardian or the feed's default authority when no guardian is set (the encoded creator in CompressedOracle).
  - **Constraints**: The `minPrice` must be less than `maxPrice`.
  - **Impact**: Stores feed-specific lower and upper price guards.
- `maxPrice`
  - **Control**: The active guardian or the feed's default authority when no guardian is set (the encoded creator in CompressedOracle).
  - **Constraints**: The `minPrice` must be less than `maxPrice`.
  - **Impact**: Stores feed-specific lower and upper price guards.

Branches and code coverage

Intended branches

- Stores a valid inclusive price range.
  - ☐ Test coverage

Negative branches

- Unauthorized access or an invalid range causes the call to revert.
  - ☐ Test coverage

5.18. providers-OracleBase.sol

Function: `function acceptStateGuardRole(bytes32 feedId) external`

This function accepts a pending guardian role for a provider feed.

Inputs

- `feedId`
  - **Control**: Pending guardian.
  - **Constraints**: N/A.
  - **Impact**: Promotes the caller to active guardian.

Branches and code coverage

Intended branches

- Transfers the active feed guardian role.
  - ☐ Test coverage

Negative branches

- A nonpending caller causes the call to revert.
  - ☐ Test coverage

Function: `function purgePendingStateGuardRole(bytes32 feedId) external checkRole(feedId)`

This function clears a pending provider-feed guardian.

Inputs

- `feedId`
  - **Control**: The active guardian — or ADMIN when no guardian is set.
  - **Constraints**: N/A.
  - **Impact**: Deletes the pending nomination.

Branches and code coverage

Intended branches

- Clears the pending guardian field.
  - ☐ Test coverage

Negative branches

- Unauthorized access causes the call to revert.
  - ☐ Test coverage

Function: `function purgeStateGuardRole(bytes32 feedId) external`

This function removes the active guardian role from a provider feed.

Inputs

- `feedId`
  - **Control**: The active guardian or ADMIN.
  - **Constraints**: N/A.
  - **Impact**: Clears active and pending guardian state.

Branches and code coverage

Intended branches

- Purges the feed guardian role.
  - ☐ Test coverage

Negative branches

- Unauthorized access causes the call to revert.
  - ☐ Test coverage

Function call analysis

- `hasRole(ADMIN_ROLE, msg.sender)`
    - **What is controllable?** The caller supplies `feedId` and its own identity. The required ADMIN role is fixed; its membership is stored in this contract. The active guard comparison is evaluated first.
    - **If the return value is controllable, how is it used and how can it go wrong?** `hasRole` is skipped when the caller is the active guard. Otherwise, its local boolean authorizes an ADMIN to recover the feed even when another active guard exists. A successful check precedes deletion of the active guard and any nonzero pending guard.
    - **What happens if it reverts, reenters, or does other unusual control flow?** If both comparisons are false, the outer `require` reverts with `InvalidGuard(msg.sender)`, not AccessControl's unauthorized-account error. There is no external call, `_checkRole` call, or callback. Guard deletions and their events are atomic.

Function: `function register(bytes32 feedId, address pool, address factory) external payable`

This function pays to authorize a pool to read a feed and clears any blacklist entry for that pool.

Inputs

- `feedId`
    - **Control**: Arbitrary.
    - **Constraints**: N/A.
    - **Impact**: Selects the feed key for `registeredPool[feedId][pool]`; no feed data is created.
- `pool`
    - **Control**: Arbitrary.
    - **Constraints**: Must be nonzero and recognized by the approved factory's `isPool(pool)` query.
    - **Impact**: Receives feed-read authorization and has its blacklist entry cleared if set.
- `factory`
    - **Control**: Arbitrary.
    - **Constraints**: Must belong to `approvedFactories` and return true from `isPool(pool)`.
    - **Impact**: Supplies the pool-membership check; the factory address is not stored for this registration.

- `msg.value`
  - **Control**: Arbitrary.
  - **Constraints**: The `msg.value` must be at least `registrationFee`.
  - **Impact**: Adds ETH to the oracle balance; overpayment is retained without a refund.

Branches and code coverage

Intended branches

- Sets `registeredPool[feedId][pool]` to true and emits `PoolRegistered`, including on repeated registration. If the pool was blacklisted, clears that entry and emits `BlacklistUpdated` first.
  - ☐ Test coverage

Negative branches

- Insufficient fee reverts with `InsufficientFee`, a zero pool fails a bare `require`, an unapproved factory reverts with `FactoryNotApproved`, and a false membership response reverts with `NotAPool`. A factory revert or malformed ABI response also reverts registration.
  - ☐ Test coverage

Function call analysis

- `approvedFactories.contains(factory)`
  - **What is controllable?** The caller chooses `factory`, `pool`, `feedId`, and payment. The fee and nonzero-pool requires run first. Set membership is controlled by prior ADMIN factory configuration, not by the submitted call.
  - **If the return value is controllable, how is it used and how can it go wrong?** The local storage boolean must be true or `FactoryNotApproved(factory)` reverts. No code at the factory or pool runs during this lookup.
  - **What happens if it reverts, reenters, or does other unusual control flow?** The `InsufficientFee`, the nonzero-pool bare require, or `FactoryNotApproved` stops execution before external validation. These checks do not write registration state or trigger callbacks.
- `IPoolFactory(factory).isPool(pool)`
  - **What is controllable?** The caller selects an approved factory target and supplies the nonzero pool argument. The factory sees the oracle as the caller. The oracle sends no ETH to the factory and does not call the pool itself.
  - **If the return value is controllable, how is it used and how can it go wrong?**

The ABI-decoded bool gates `require(..., NotAPool(pool))`. A true response permits clearing `blacklisted[pool]` when set and then writing `registeredPool[feedId][pool] = true` and emitting `PoolRegistered`. The factory's statement is trusted; feed data and duplicate registration are not checked, and the factory address is not stored as a feed configuration. All attached ETH, including excess over the fee, stays in the oracle.

- **What happens if it reverts, reenters, or does other unusual control flow?** No try-catch exists, a factory revert propagates, false becomes `NotAPool(pool)`, and missing or malformed bool return data causes ABI decoding to revert. STATICCALL allows read-only callbacks but prevents state-changing reentrancy throughout the nested static context. Registration and blacklist writes occur only after a true return, although the incoming ETH is already reflected in the oracle balance during the call. Any revert rolls back both payment and state changes.

Function: `function setPriceGuard(bytes32 feedId, uint128 minPrice, uint128 maxPrice) external checkRole(feedId)`

This function sets the permitted price range for a provider feed.

Inputs

- `feedId`
  - **Control**: The active guardian — or ADMIN when no guardian is set.
  - **Constraints**: The `minPrice` must be less than `maxPrice`.
  - **Impact**: Stores lower and upper price bounds.
- `minPrice`
  - **Control**: The active guardian — or ADMIN when no guardian is set.
  - **Constraints**: The `minPrice` must be less than `maxPrice`.
  - **Impact**: Stores lower and upper price bounds.
- `maxPrice`
  - **Control**: The active guardian — or ADMIN when no guardian is set.
  - **Constraints**: The `minPrice` must be less than `maxPrice`.
  - **Impact**: Stores lower and upper price bounds.

Branches and code coverage

Intended branches

- Stores a valid price guard.

- ☐ Test coverage

Negative branches

- Unauthorized access or invalid bounds causes the call to revert.
  - ☐ Test coverage

Function: `function setStateGuardRole(bytes32 feedId, address newGuard) external checkRole(feedId)`

This function proposes a provider feed guardian by setting `pendingStateGuard`; the proposed guardian must accept before becoming active.

Inputs

- `feedId`
  - **Control**: Current feed guardian — or ADMIN when no guardian is set.
  - **Constraints**: N/A.
  - **Impact**: Selects the feed's pending guardian entry.
- `newGuard`
  - **Control**: Current feed guardian — or ADMIN when no guardian is set.
  - **Constraints**: N/A.
  - **Impact**: Becomes the pending guardian without changing the active guardian.

Branches and code coverage

Intended branches

- Stores the pending guardian and emits `StateGuardPending`.
  - ☐ Test coverage

Negative branches

- If an active guardian exists, another caller reverts with `InvalidGuard`. Otherwise, a caller without `ADMIN_ROLE` reverts with `AccessControlUnauthorizedAccount`.
  - ☐ Test coverage

Function: `function withdrawEth() external onlyRole(ADMIN_ROLE)`

This function withdraws the contract's accumulated ETH to the admin-controlled recipient.

Inputs

- `msg.sender`
  - **Control**: Caller.
  - **Constraints**: N/A.
  - **Impact**: Authorizes the ETH withdrawal.

Branches and code coverage

Intended branches

- Transfers the available ETH balance through the configured withdrawal path.
  - ☐ Test coverage

Negative branches

- A non-admin caller or failed ETH transfer causes the call to revert.
  - ☐ Test coverage

Function call analysis

- `payable(msg.sender).call{value: amount}("")`
  - **What is controllable?** The authorized caller is the recipient; it is not a separately configured address or parameter. The amount is the entire balance sampled after the role check, including registration fees and other ETH. Calldata is empty, and the call has no explicit gas cap.
  - **If the return value is controllable, how is it used and how can it go wrong?** The EVM success flag `ok` is enforced by `require(ok)`. Return bytes are discarded and not ABI-decoded, so a successful receiver can return any bytes, and `EthWithdrawn(msg.sender, amount)` is emitted only after the call returns successfully.
  - **What happens if it reverts, reenters, or does other unusual control flow?** The recipient's receive/fallback can execute and reenter; there is no reentrancy lock. The value transfer has already debited the oracle and credited the recipient while its code runs, although no oracle storage field is updated first. Reentrant calls face their own authorization and balance checks; ETH sent back during the callback is not included in the outer sampled amount. A failed call is converted to a bare require revert, discarding the recipient's revert data and rolling back transfer and nested effects. A successful call's nested effects persist.

6. Assessment Results

During our assessment on the scoped OMM Reaudit contracts, we discovered eight findings. No critical issues were found. Seven findings were of low impact and the remaining finding was informational in nature.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.